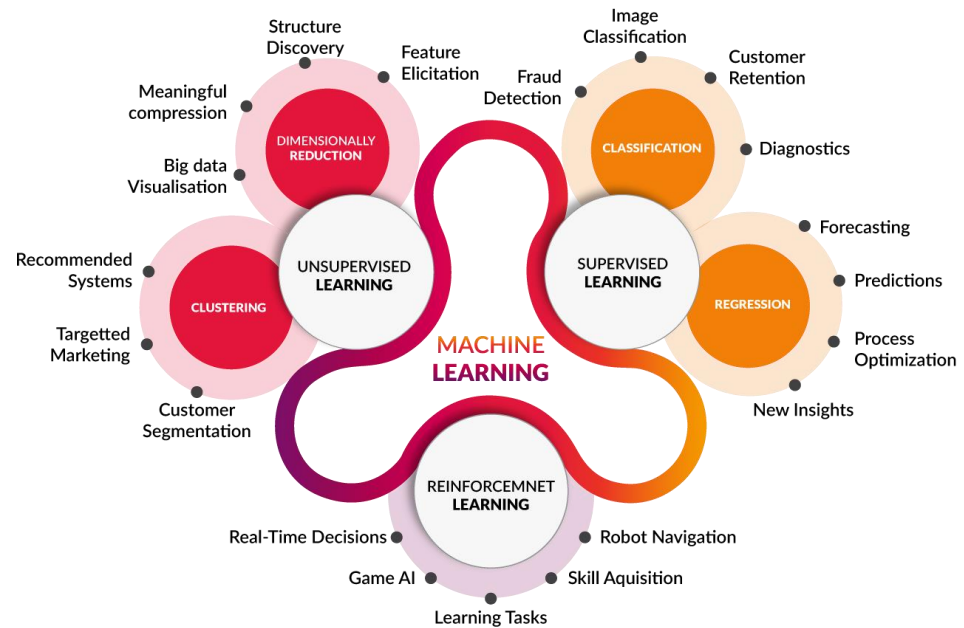




순환신경망 기반의 시계열 회귀

강필성

고려대학교 산업경영공학부

pilsung_kang@korea.ac.kr

Non-Sequential vs. Sequential (Time-Series) Data

- Non-Sequential Data: 시간 정보를 포함하지 않고 생성되는 데이터
 - ✓ 순차 데이터가 아닌 경우 데이터는 N by D 행렬(N: 관측치 수, D: 변수 수)로 표현됨
 - 예) 특정 고객의 금융상품 이용 현황을 바탕으로 대출 상품 추천
 - X: 고객별 금융 상품 이용 현황
 - Y: 대출 상품 이용 유무 (1: 사용, 0: 미사용)

	Var 1	Var 2	Var 3	...	Var D
Obs 1			X		
Obs 2					
Obs 3					
...					
...					
...					
...					
Obs N-1					
Obs N					

Y
Y

Non-Sequential vs. Sequential (Time-Series) Data

- Non-Sequential Data: 시간 정보를 포함하지 않고 생성되는 데이터
 - ✓ 순차 데이터가 아닌 경우 데이터는 N by D 행렬(N: 관측치 수, D: 변수 수)로 표현됨
 - 예) 특정 고객의 금융상품 이용 현황을 바탕으로 대출 상품 추천
 - X: 고객별 금융 상품 이용 현황
 - Y: 대출 상품 이용 유무 (1: 사용, 0: 미사용)

	Var 1	Var 2	Var 3	...	Var D
Obs 1					
Obs 2			X		
Obs 3					
...					
...					
...					
...					
Obs N-1					
Obs N					

Y
Y

Non-Sequential vs. Sequential (Time-Series) Data

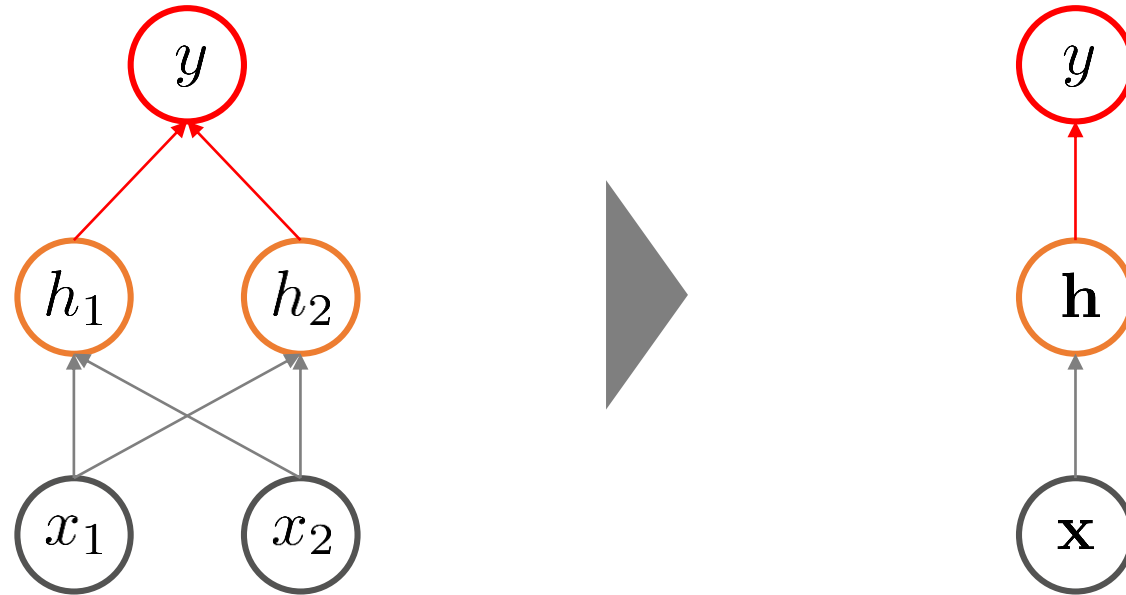
- Non-Sequential Data: 시간 정보를 포함하지 않고 생성되는 데이터
 - ✓ 순차 데이터가 아닌 경우 데이터는 N by D 행렬(N: 관측치 수, D: 변수 수)로 표현됨
 - 예) 특정 고객의 금융상품 이용 현황을 바탕으로 대출 상품 추천
 - X: 고객별 금융 상품 이용 현황
 - Y: 대출 상품 이용 유무 (1: 사용, 0: 미사용)

	Var 1	Var 2	Var 3	...	Var D
Obs 1					
Obs 2					
Obs 3					
...					
...					
...					
...					
Obs N-1					
Obs N			X		

Y
Y

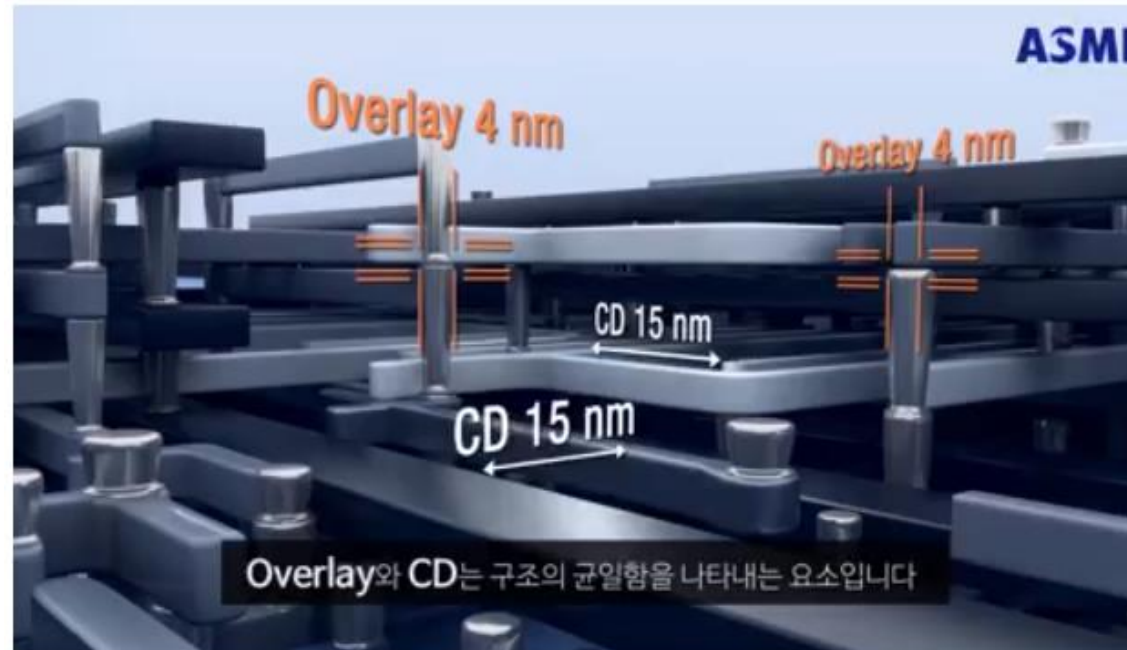
Non-Sequential vs. Sequential (Time-Series) Data

- 순서(sequence)가 없는 인공신경망 구조



Non-Sequential vs. Sequential (Time-Series) Data

- Sequential Data: 시간 정보를 포함하여 순차적으로 생성되는 데이터
 - ✓ 순차데이터의 경우 데이터는 (N) by T by D Tensor (T는 측정 시점 수)로 표현됨
 - 예) 반도체 공정에서 주기적으로 측정되는 여러 센서 값을 이용한 Critical Dimension 예측
 - X: 반도체 공정 설비에서 시간별 측정되는 다양한 센서(변수) 값
 - Y: 웨이퍼의 Critical Dimension (CD)



출처: ASML Facebook official page (<https://www.facebook.com/ASMLKR/videos/1615526971830085/>)

Non-Sequential vs. Sequential (Time-Series) Data

- Sequential Data: 시간 정보를 포함하여 순차적으로 생성되는 데이터
 - ✓ 순차데이터의 경우 데이터는 (N) by T by D Tensor (T는 측정 시점 수)로 표현됨
 - 예) 반도체 공정에서 주기적으로 측정되는 여러 센서 값을 이용한 Critical Dimension 예측
 - X: 반도체 공정 설비에서 시간별 측정되는 다양한 센서(변수) 값
 - Y: 웨이퍼의 Critical Dimension (CD)

Obs I	Var 1	Var 2	Var 3	...	Var D
Time 1					
Time 2					
Time 3					
...					
...					
...					
...					
Time T-1					
Time T					

Y
Y

Non-Sequential vs. Sequential (Time-Series) Data

- Sequential Data: 시간 정보를 포함하여 순차적으로 생성되는 데이터
 - ✓ 순차데이터의 경우 데이터는 (N) by T by D Tensor (T는 측정 시점 수)로 표현됨
 - 예) 반도체 공정에서 주기적으로 측정되는 여러 센서 값을 이용한 Critical Dimension 예측
 - X: 반도체 공정 설비에서 시간별 측정되는 다양한 센서(변수) 값
 - Y: 웨이퍼의 Critical Dimension (CD)

Obs 1		Var 1	Var 2	Var 3	...	Var D
Ti	Obs 2	Var 1	Var 2	Var 3	...	Var D
Ti	Time 1					
Ti	Time 2					
	Time 3					
	...					
	...					
	...					
Tim	...					
Ti	Time T-1					
	Time T					

Y
Y
Y

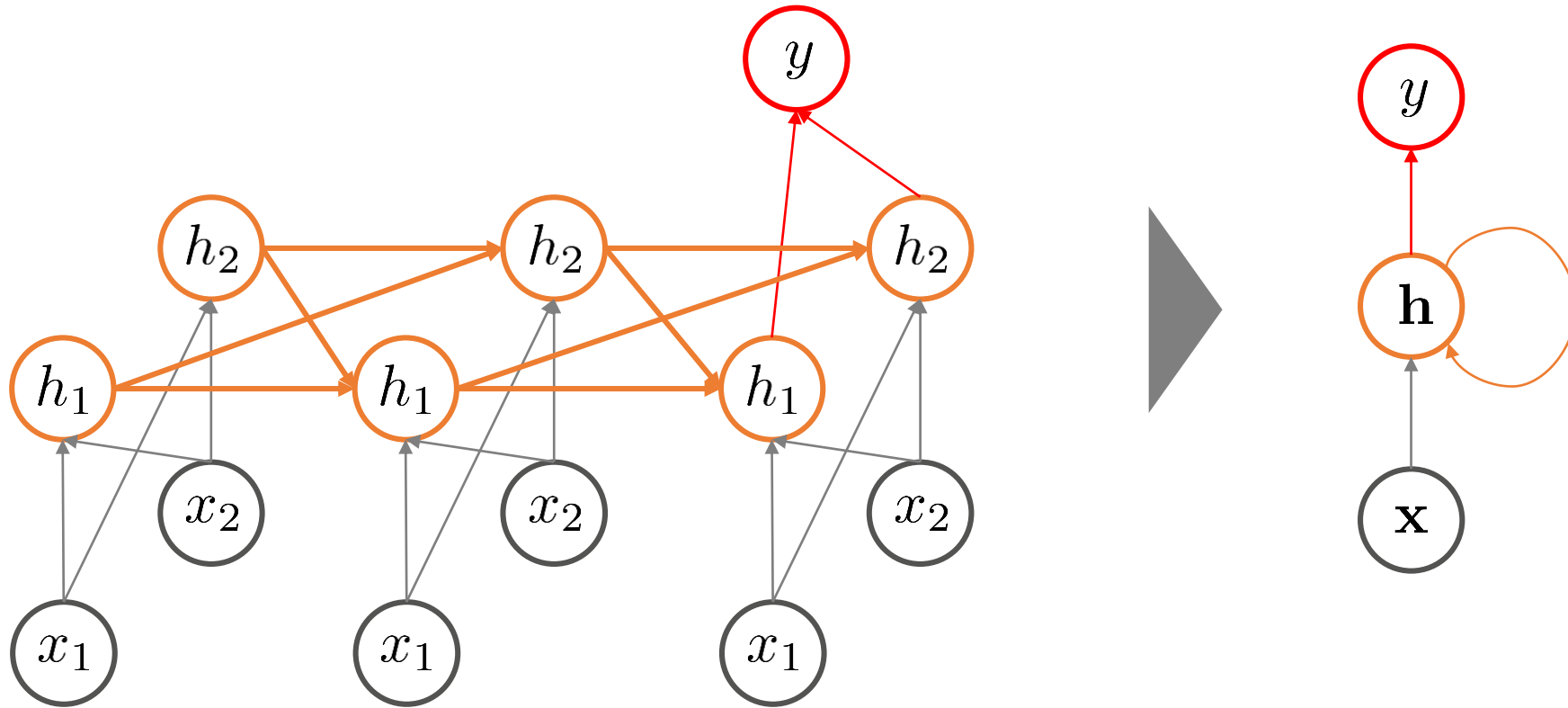
Non-Sequential vs. Sequential (Time-Series) Data

- Sequential Data: 시간 정보를 포함하여 순차적으로 생성되는 데이터
 - ✓ 순차데이터의 경우 데이터는 (N) by T by D Tensor (T는 측정 시점 수)로 표현됨
 - 예) 반도체 공정에서 주기적으로 측정되는 여러 센서 값을 이용한 Critical Dimension 예측
 - X: 반도체 공정 설비에서 시간별 측정되는 다양한 센서(변수) 값
 - Y: 웨이퍼의 Critical Dimension (CD)

Obs 1	Var 1	Var 2	Var 3	...	Var D		Y
Ti	Obs 2	Var 1	Var 2	Var 3	...	Var D	Y
Ti	Time 1						
Ti	Time 2						
	Time	Obs N	Var 1	Var 2	Var 3	...	Var D
	...	Time 1					
	...	Time 2					
	...	Time 3					
Tim	...	...					
Ti	Time T	...					
	Time	...					
	...	...					
	Time T-1						
	Time T						Y

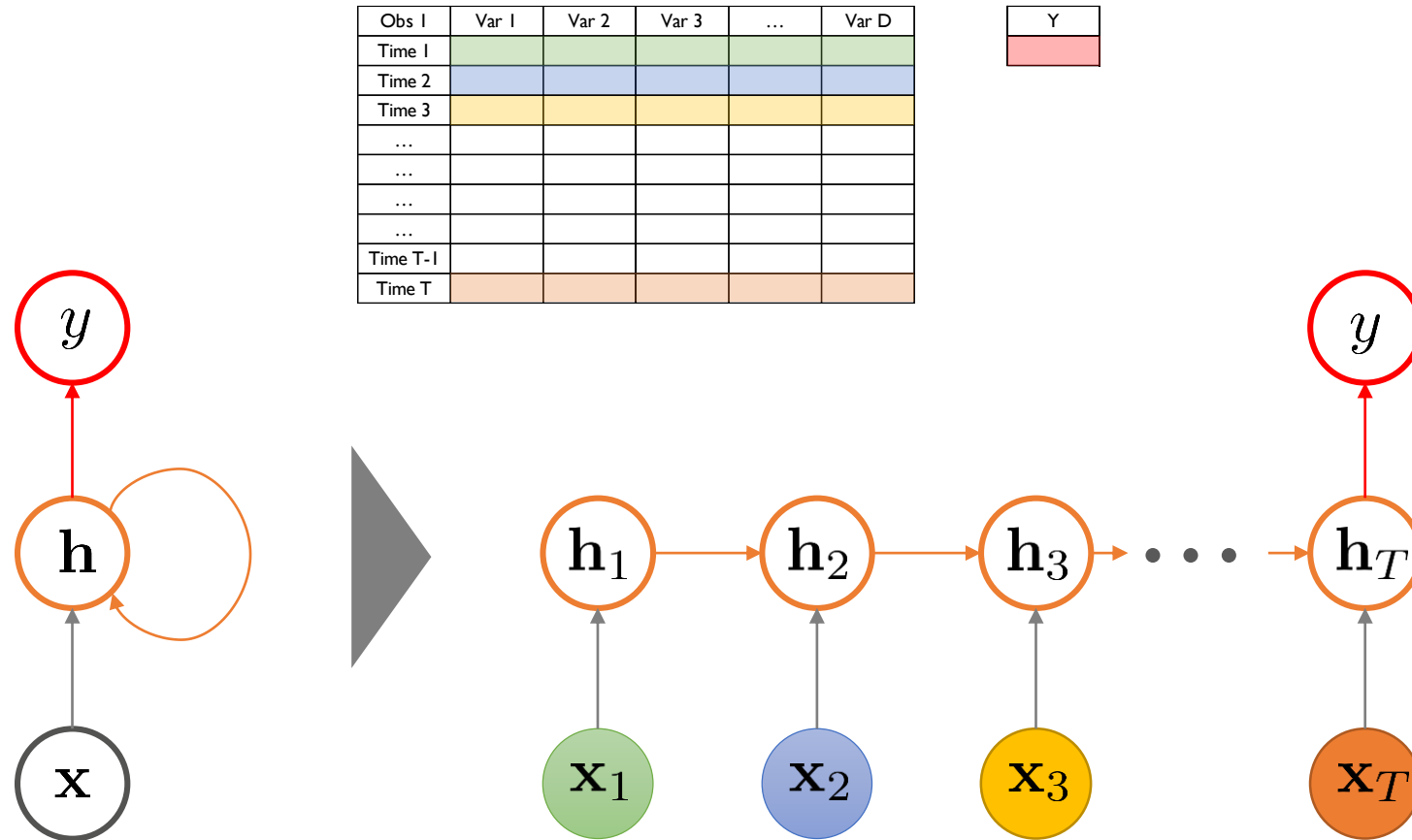
Non-Sequential vs. Sequential (Time-Series) Data

- 순서(sequence)가 있는 인공신경망 구조



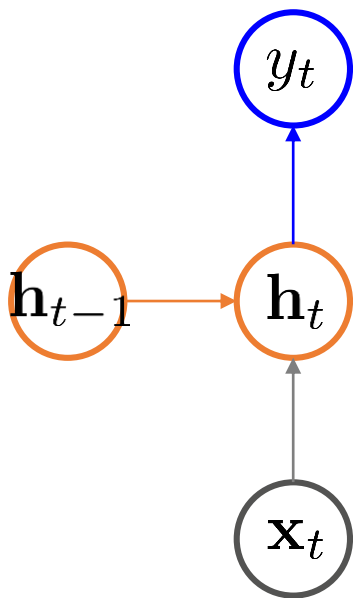
Non-Sequential vs. Sequential (Time-Series) Data

- 순서(sequence)가 있는 인공지능망 구조 (벡터 표현)



RNN Basics: Forward Path

- 기본 RNN(Vanilla RNN) 구조에서 정보의 흐름
 - ✓ $f()$ 와 $g()$ 는 활성화 함수



$$y_t = g(\mathbf{W}_{hy} \mathbf{h}_t + \mathbf{b}_y)$$

$$\mathbf{h}_t = f(\mathbf{W}_{hh} \mathbf{h}_{t-1} + \mathbf{W}_{xh} \mathbf{x}_t + \mathbf{b}_x)$$

t시점에서의 은닉 노드의
값은

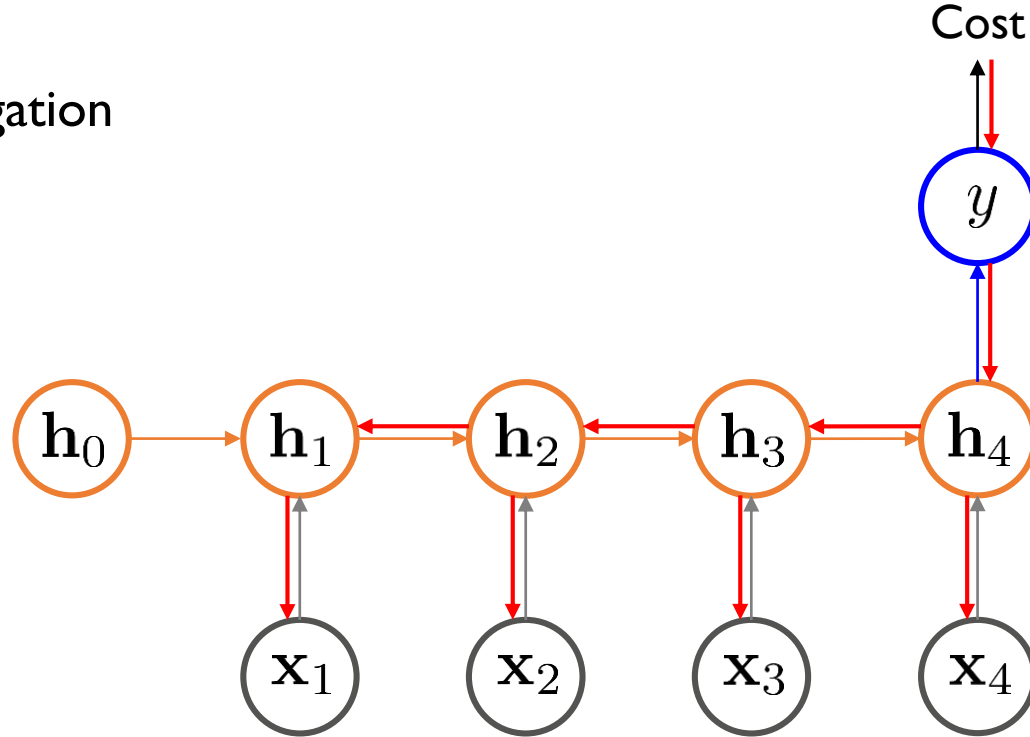
t-1 시점까지 은닉 노드에 저장된 정보와

t 시점에서 새롭게 제공되는 입력 정보에 영향을 받는다

RNN Basics: Gradient Vanishing/Exploding Problem

- RNN에서의 Backpropagation

✓ $f()$ 는 Tanh로 가정



$$\begin{aligned} \frac{\partial Cost}{\partial \mathbf{W}_{xh}} = & \frac{\partial Cost}{\partial y} \times \frac{\partial y}{\partial h_4} \times \frac{\partial h_4}{\partial \mathbf{W}_{xh}} + \frac{\partial Cost}{\partial y} \times \frac{\partial y}{\partial h_4} \times \frac{\partial h_4}{\partial h_3} \times \frac{\partial h_3}{\partial \mathbf{W}_{xh}} \\ & + \frac{\partial Cost}{\partial y} \times \frac{\partial y}{\partial h_4} \times \frac{\partial h_4}{\partial h_3} \times \frac{\partial h_3}{\partial h_2} \times \frac{\partial h_2}{\partial \mathbf{W}_{xh}} \\ & + \frac{\partial Cost}{\partial y} \times \frac{\partial y}{\partial h_4} \times \frac{\partial h_4}{\partial h_3} \times \frac{\partial h_3}{\partial h_2} \times \frac{\partial h_2}{\partial h_1} \times \frac{\partial h_1}{\partial \mathbf{W}_{xh}} \end{aligned}$$

RNN Basics: Gradient Vanishing/Exploding Problem

- RNN에서의 Backpropagation

✓ 일반적으로

$$\frac{\partial Cost}{\partial \mathbf{W}_{xh}} = \sum_{i=1}^n \left(\frac{\partial Cost}{\partial y} \cdot \frac{\partial y}{\partial \mathbf{h}_n} \cdot \left(\prod_{j=i}^{n-1} \frac{\partial \mathbf{h}_{j+1}}{\partial \mathbf{h}_j} \right) \cdot \frac{\partial \mathbf{h}_i}{\partial \mathbf{W}_{xh}} \right)$$

✓ $f(\cdot)$ 는 Tanh이고

$$\mathbf{h}_t = f(\mathbf{W}_{hh} \mathbf{h}_{t-1} + \mathbf{W}_{xh} \mathbf{x}_t + \mathbf{b}_x) = \tanh(\mathbf{z}_t)$$

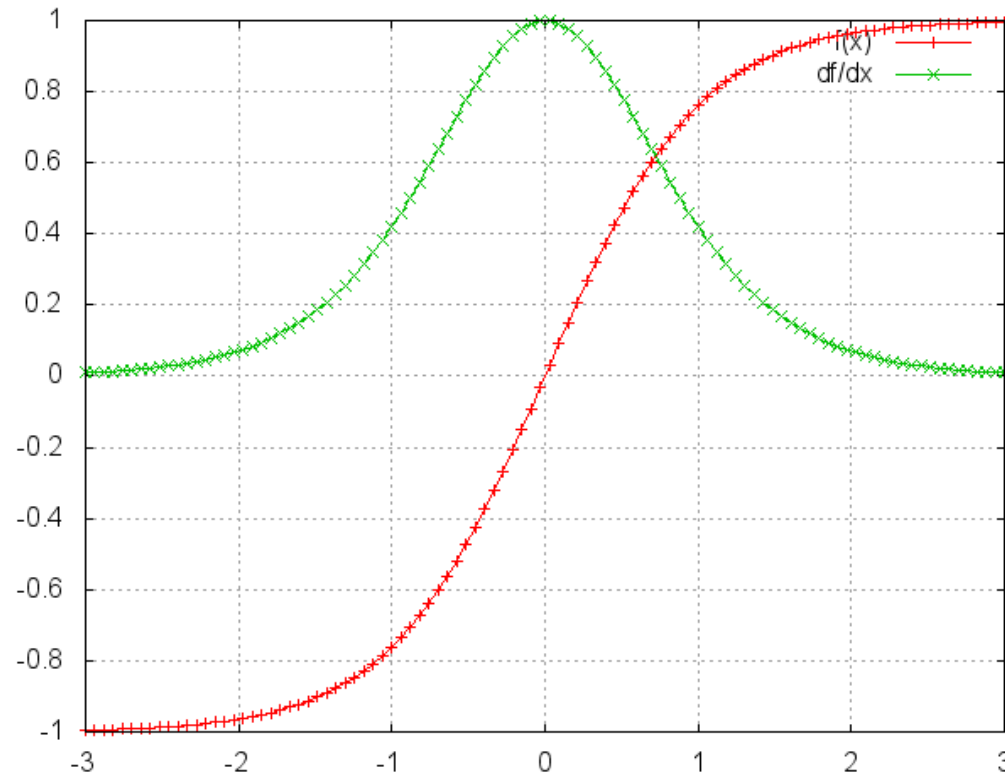
$$\frac{\partial \mathbf{h}_t}{\partial \mathbf{h}_{t-1}} = \frac{\partial \mathbf{h}_t}{\partial \mathbf{z}_t} \times \frac{\partial \mathbf{z}_t}{\partial \mathbf{h}_{t-1}} = (1 - \tanh^2(\mathbf{z}_t)) \times \mathbf{W}_{hh}$$

$$\frac{\partial \mathbf{h}_t}{\partial \mathbf{W}_{xh}} = \frac{\partial \mathbf{h}_t}{\partial \mathbf{z}_t} \times \frac{\partial \mathbf{z}_t}{\partial \mathbf{W}_{xh}} = (1 - \tanh^2(\mathbf{z}_t)) \times \mathbf{x}_t$$

RNN Basics: Gradient Vanishing/Exploding Problem

- RNN에서의 Backpropagation

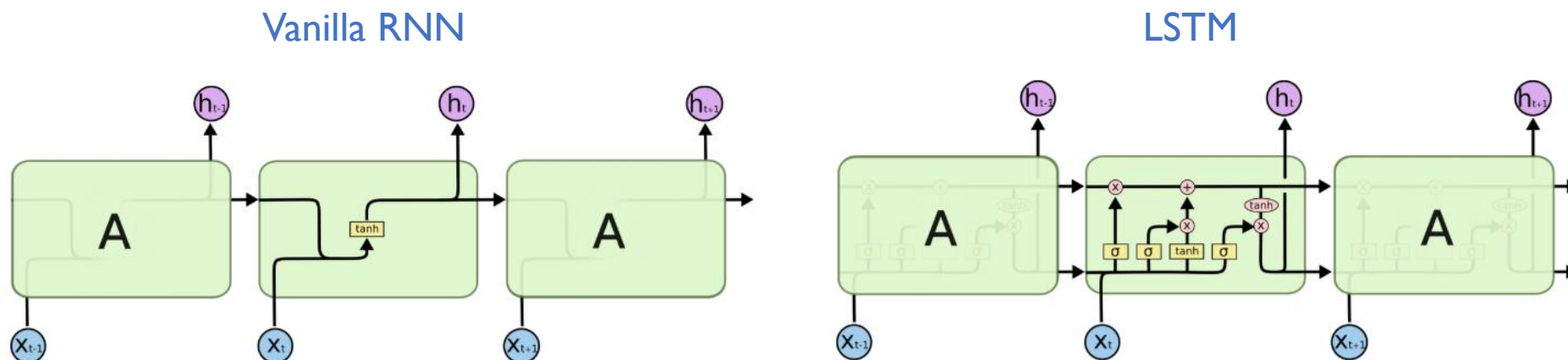
$$\frac{\partial \mathbf{h}_t}{\partial \mathbf{h}_{t-1}} = \frac{\partial \mathbf{h}_t}{\partial \mathbf{z}_t} \times \frac{\partial \mathbf{z}_t}{\partial \mathbf{h}_{t-1}} = (1 - \tanh^2(\mathbf{z}_t)) \times \mathbf{W}_{hh}$$



RNN Hidden Unit: LSTM

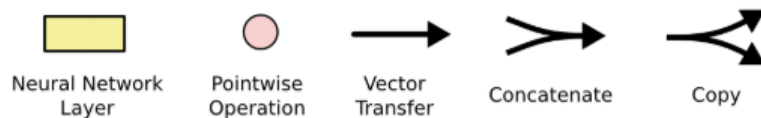
- LSTM: Long Short-Term Memory

✓ Gradient exploding/vanishing 문제를 해결하여 Long-term dependency 학습 가능



<http://colah.github.io/posts/2015-08-Understanding-LSTMs/>

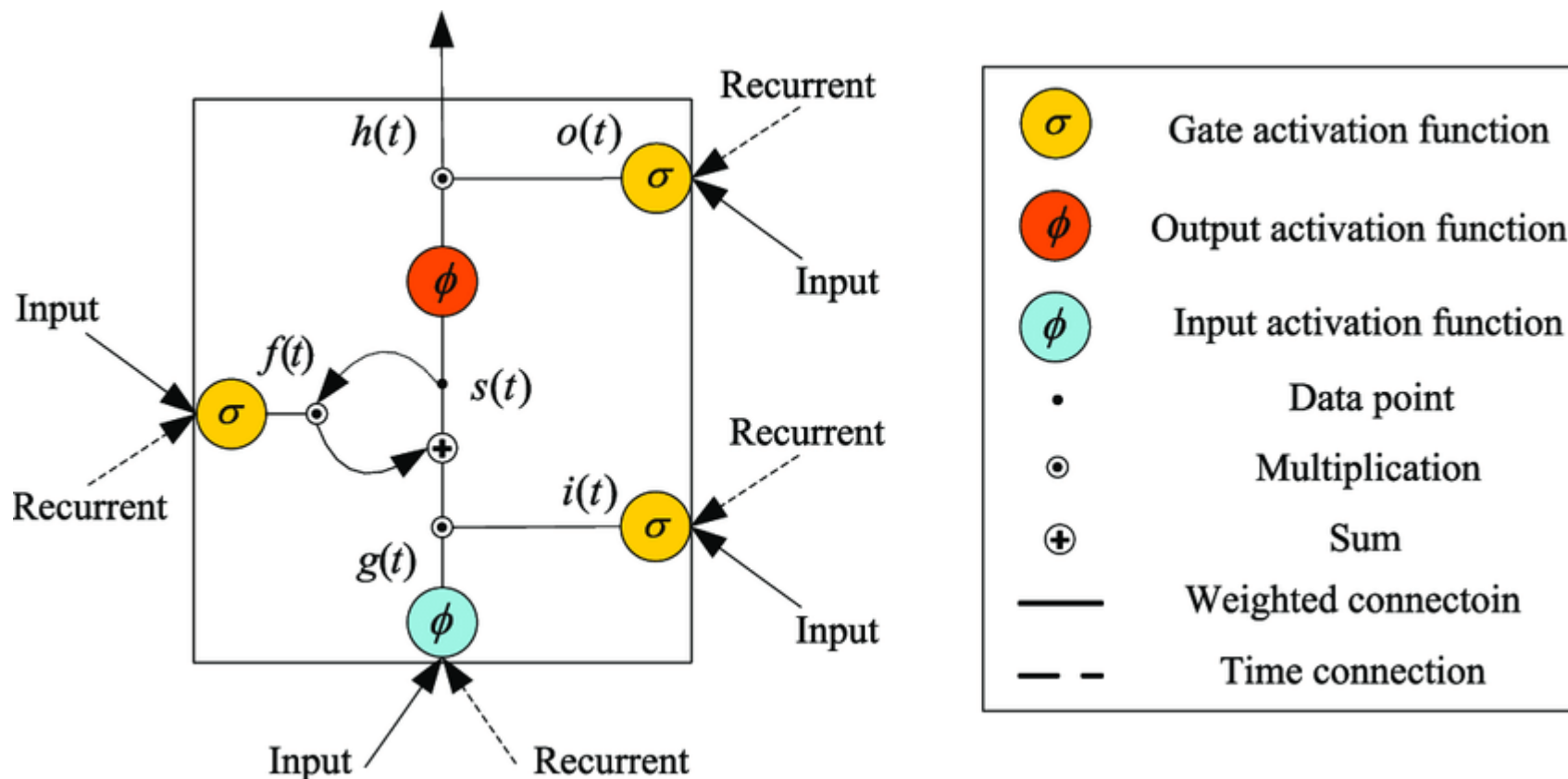
Operation symbols



RNN Hidden Unit: LSTM

- LSTM: Long Short-Term Memory

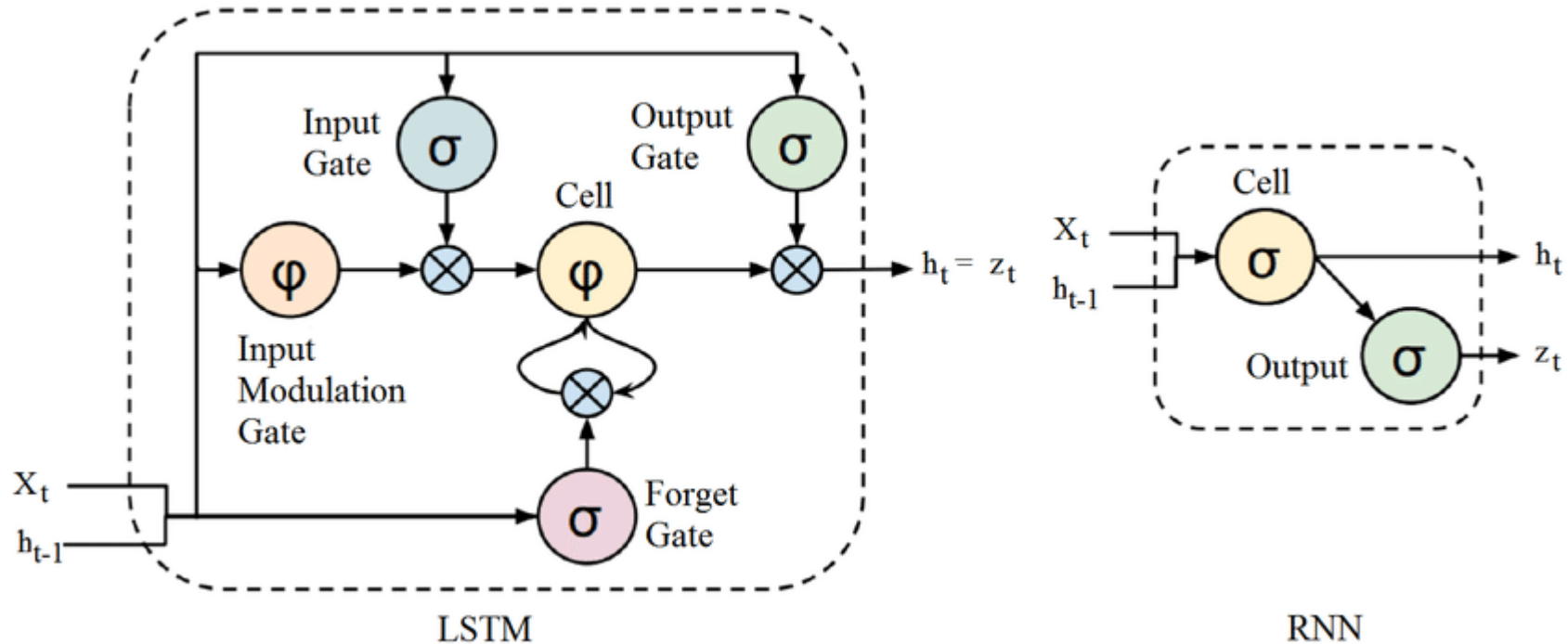
✓ LSTM의 구조가 복잡하여 다양한 diagram들이 존재



RNN Hidden Unit: LSTM

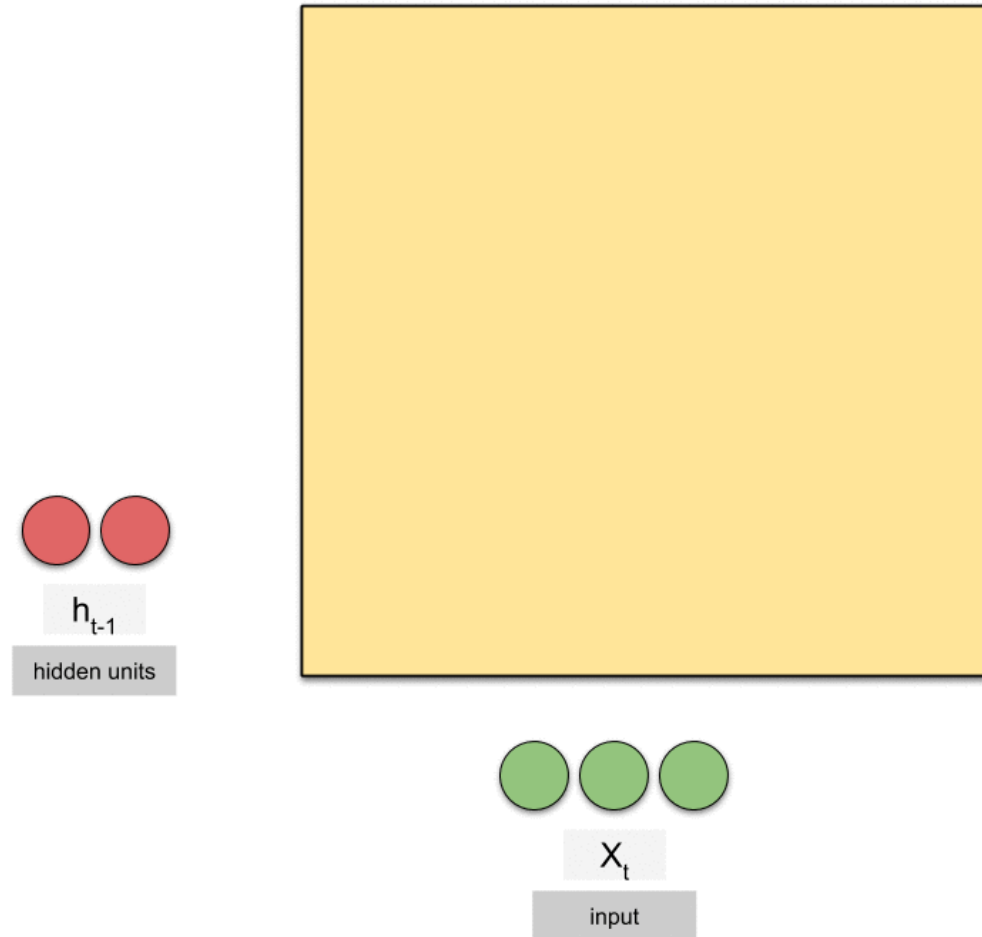
- LSTM: Long Short-Term Memory

✓ LSTM의 구조가 복잡하여 다양한 diagram들이 존재



RNN Hidden Unit: LSTM

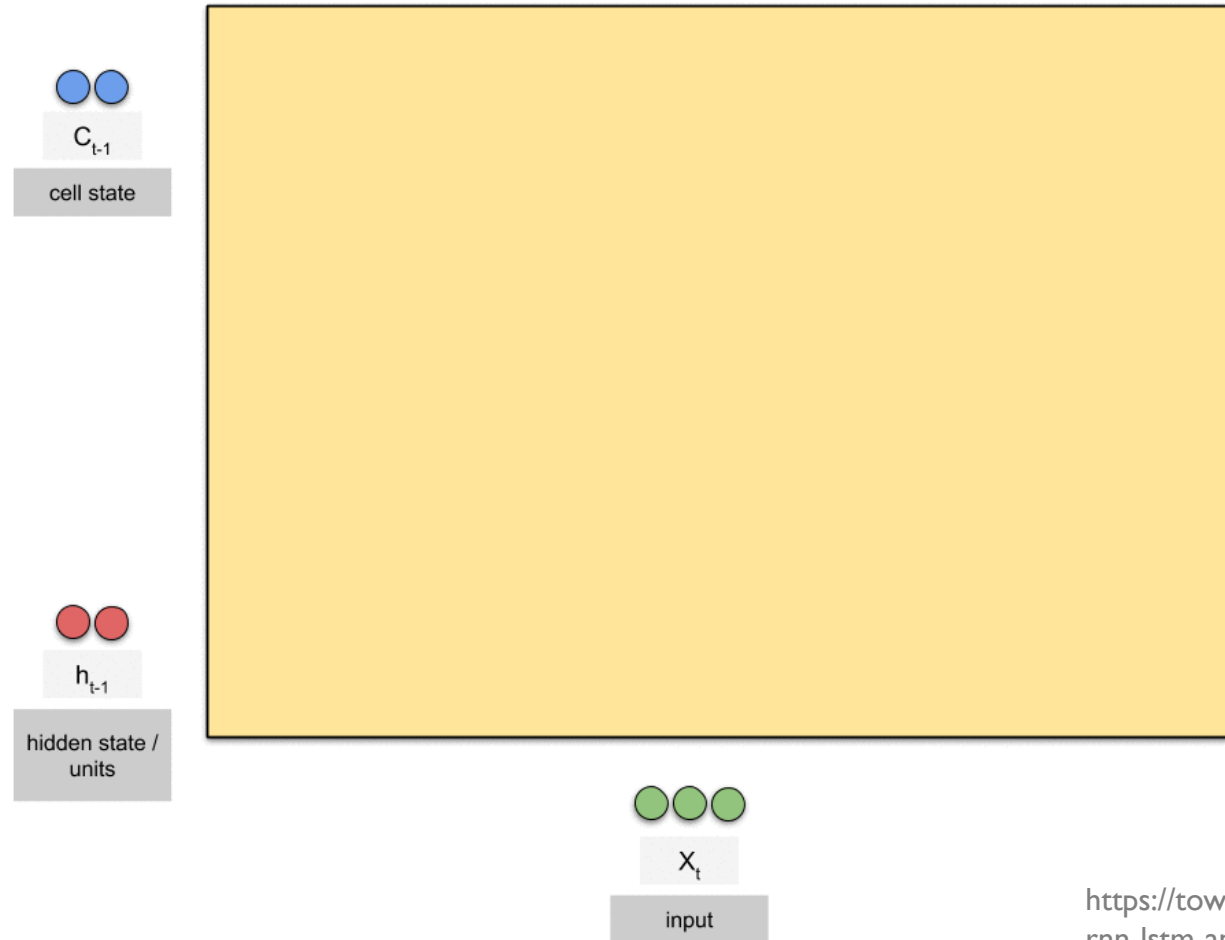
- Vanilla RNN vs LSTM



<https://towardsdatascience.com/animated-rnn-lstm-and-gru-ef124d06cf45>

RNN Hidden Unit: LSTM

- Vanilla RNN vs LSTM

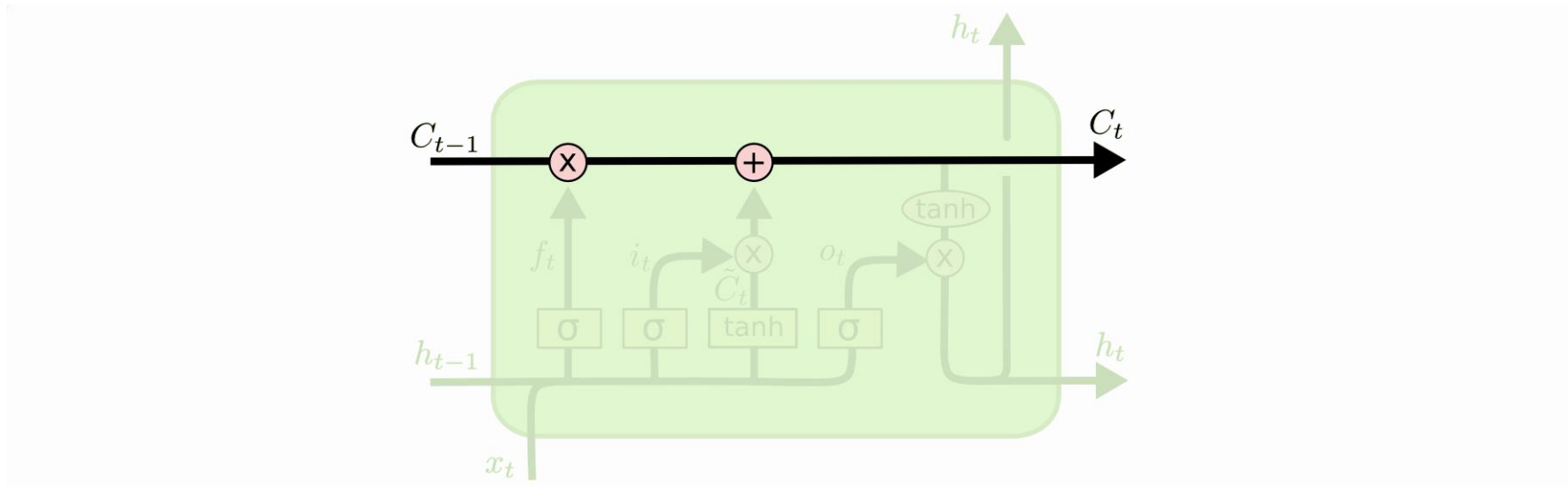


<https://towardsdatascience.com/animated-rnn-lstm-and-gru-ef124d06cf45>

RNN Hidden Unit: LSTM

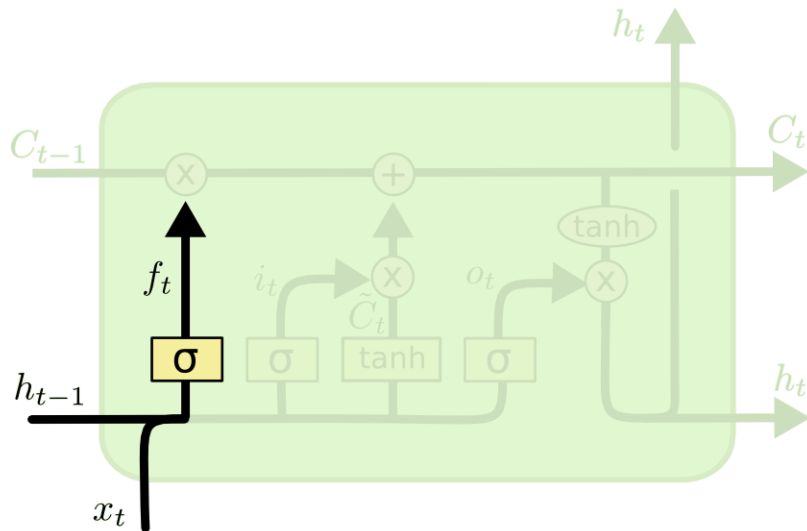
- Cell state

✓ LSTM 핵심 구성 요소, 아래 그림에서는 다이어그램의 상부를 관통하는 선(line)



RNN Hidden Unit: LSTM

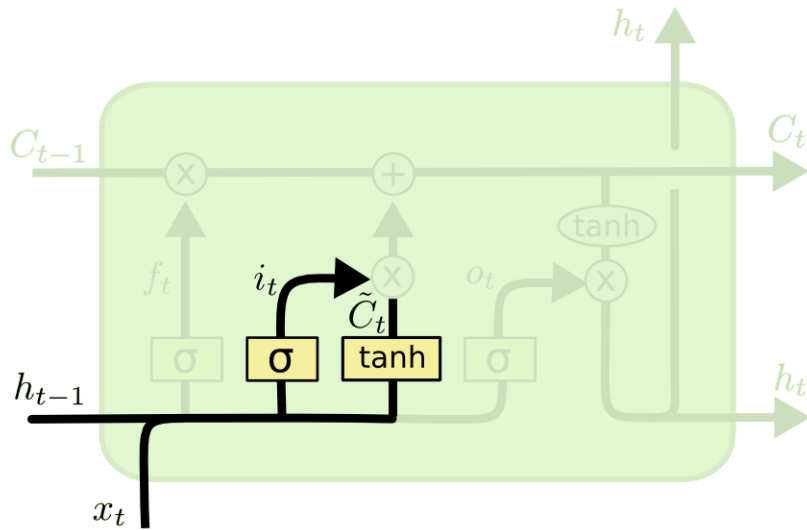
- Step 1: 지금까지의 cell state에 저장된 정보 중에서 얼마만큼을 망각(forget)할 것인지 결정
 - ✓ **Forget gate**: 이전 단계의 hidden state h_{t-1} 와 현 단계의 입력 x_t 으로부터 0과 1사이의 값을 출력 (Sigmoid 함수 사용)
 - 1: 지금까지 cell state에 저장된 모든 정보를 보존
 - 0: 지금까지 cell state에 저장된 모든 정보를 무시



$$f_t = \sigma (W_f \cdot [h_{t-1}, x_t] + b_f)$$

RNN Hidden Unit: LSTM

- Step 2: 새로운 정보를 얼마만큼 cell state에 저장할 것인지를 결정
 - ✓ **Input gate**: 어떤 값을 업데이트 할 것인지 결정
 - ✓ Tanh layer를 사용하여 새로운 cell state의 후보 $\tilde{C}_t$ 을 생성

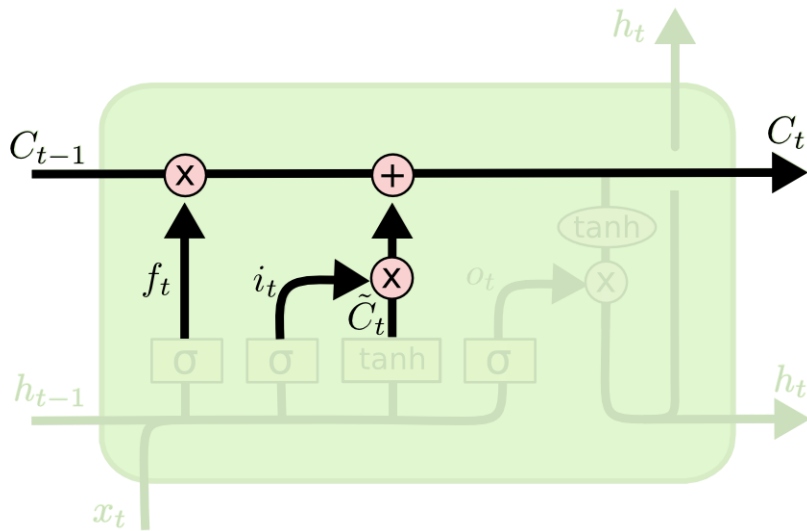


$$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i)$$

$$\tilde{C}_t = \tanh(W_C \cdot [h_{t-1}, x_t] + b_C)$$

RNN Hidden Unit: LSTM

- Step 3: 예전 cell state를 새로운 cell state로 업데이트
 - ✓ 예전 cell state를 얼마만큼 망각할 것인가를 계산한 forget gate 결과값과 곱함
 - ✓ 새로운 cell state 후보와 얼마만큼 보존할 것인가를 계산한 input gate 결과값을 곱함
 - ✓ 두 값을 더하여 새로운 cell state 값으로 결정

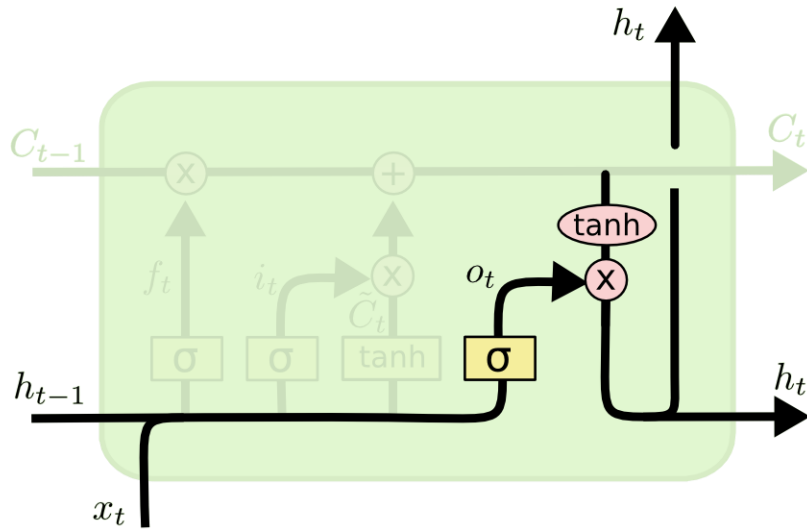


$$C_t = f_t * C_{t-1} + i_t * \tilde{C}_t$$

RNN Hidden Unit: LSTM

- Step 4: 출력 값을 결정

- ✓ 이전 hidden state 값과 현재의 입력 값을 이용하여 output gate 값을 산출
- ✓ Output gate 값과 현재의 cell state 값을 결합하여 현재의 hidden state 값을 계산

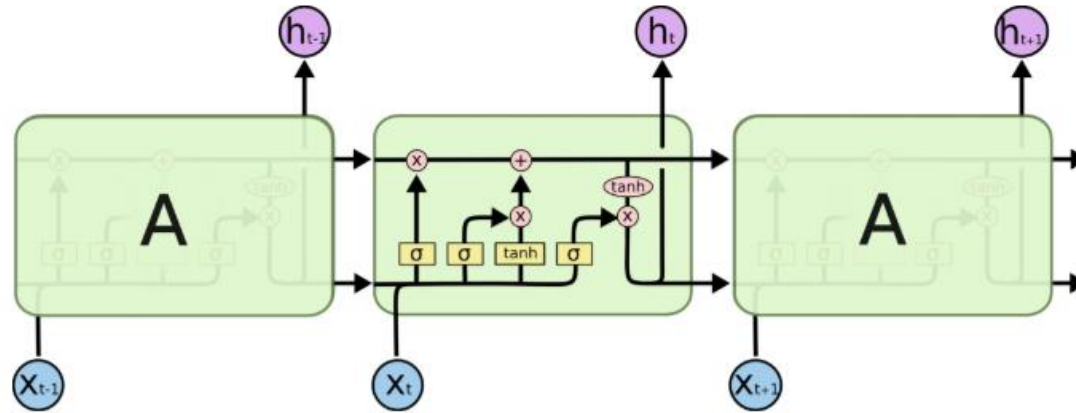


$$o_t = \sigma (W_o [h_{t-1}, x_t] + b_o)$$

$$h_t = o_t * \tanh (C_t)$$

RNN Hidden Unit: LSTM

- LSTM 요약



$$f_t = \sigma (W_f \cdot [h_{t-1}, x_t] + b_f)$$

$$i_t = \sigma (W_i \cdot [h_{t-1}, x_t] + b_i)$$

$$\tilde{C}_t = \tanh(W_C \cdot [h_{t-1}, x_t] + b_C)$$

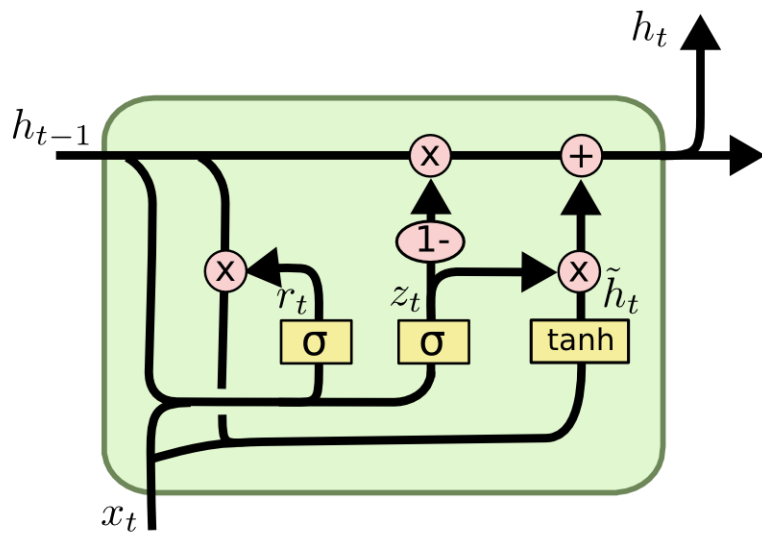
$$C_t = f_t * C_{t-1} + i_t * \tilde{C}_t$$

$$o_t = \sigma (W_o \cdot [h_{t-1}, x_t] + b_o)$$

$$h_t = o_t * \tanh(C_t)$$

RNN Hidden Unit: GRU

- GRU: Gated Recurrent Unit



$$z_t = \sigma(W_z \cdot [h_{t-1}, x_t]) \quad \text{Update gate}$$

$$r_t = \sigma(W_r \cdot [h_{t-1}, x_t]) \quad \text{Reset gate}$$

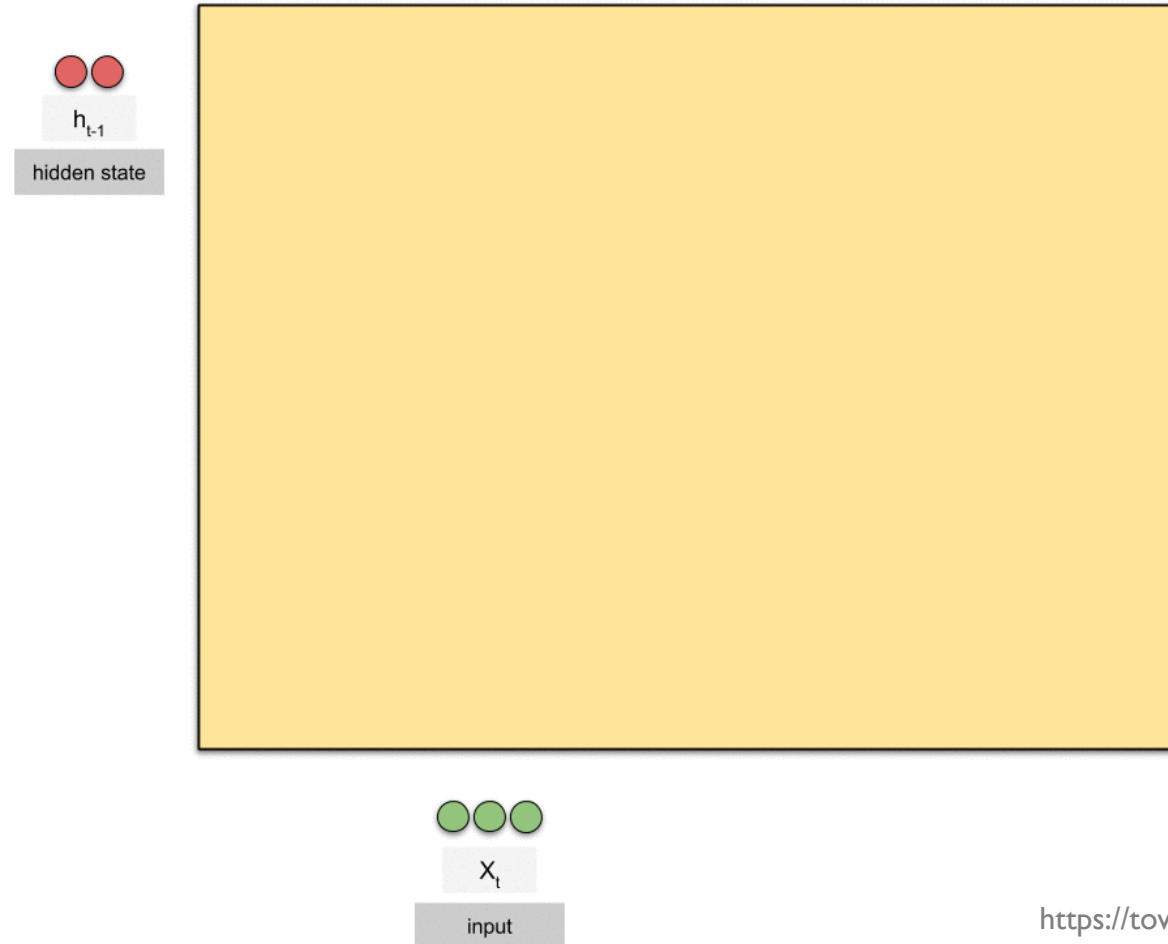
$$\tilde{h}_t = \tanh(W \cdot [r_t * h_{t-1}, x_t])$$

$$h_t = (1 - z_t) * h_{t-1} + z_t * \tilde{h}_t$$

- ✓ LSTM을 단순화한 구조, 실제 활용에서는 LSTM과 GRU의 성능 차이는 미비함
- ✓ 별도의 Cell state가 존재하지 않음
- ✓ LSTM의 forget gate와 input gate를 하나의 update gate로 결합
- ✓ Reset gate를 통해 망각과 새로운 정보 업데이트 정도를 결정

RNN Hidden Unit: GRU

- GRU: Gated Recurrent Unit

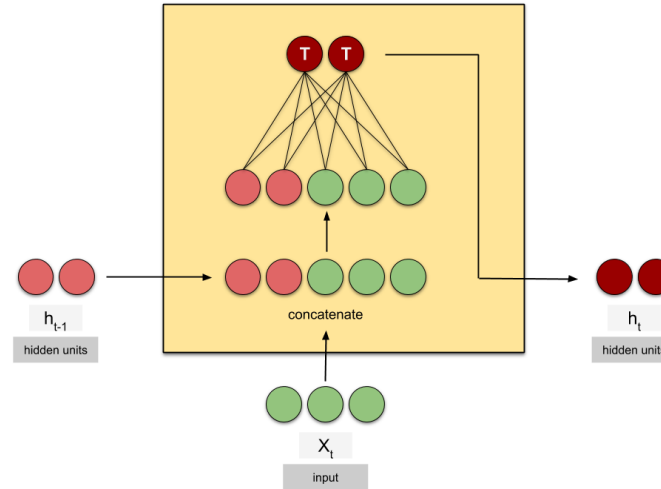


<https://towardsdatascience.com/animated-rnn-lstm-and-gru-eef124d06cf45>

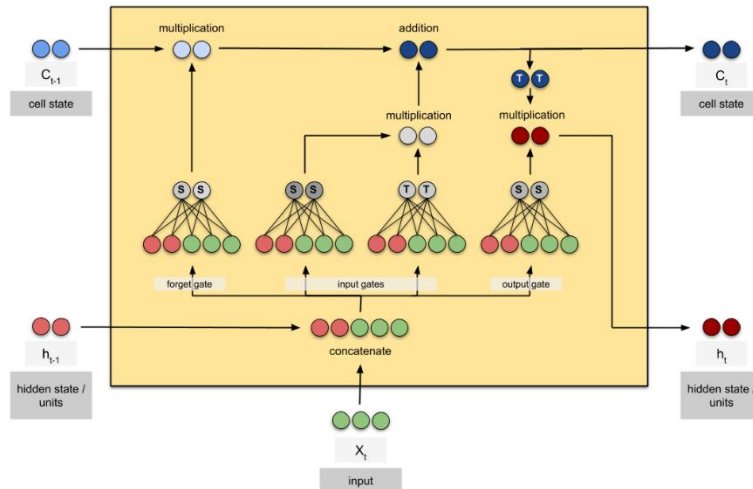
Vanilla RNN vs. LSTM vs. GRU

- 세 가지 RNN 구조 비교

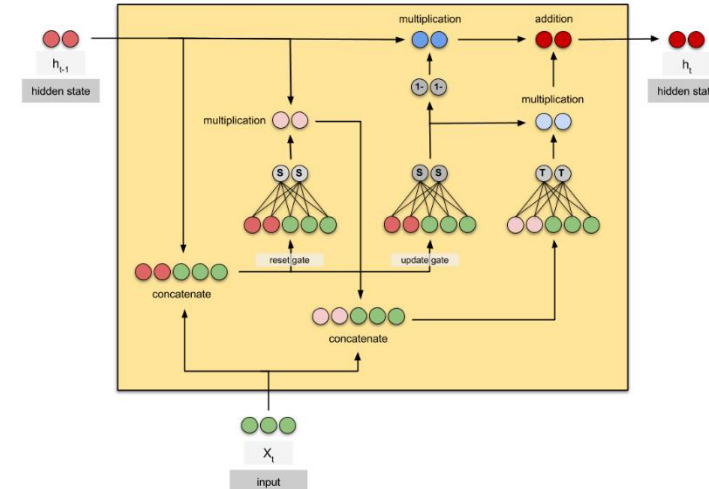
Vanilla RNN



LSTM



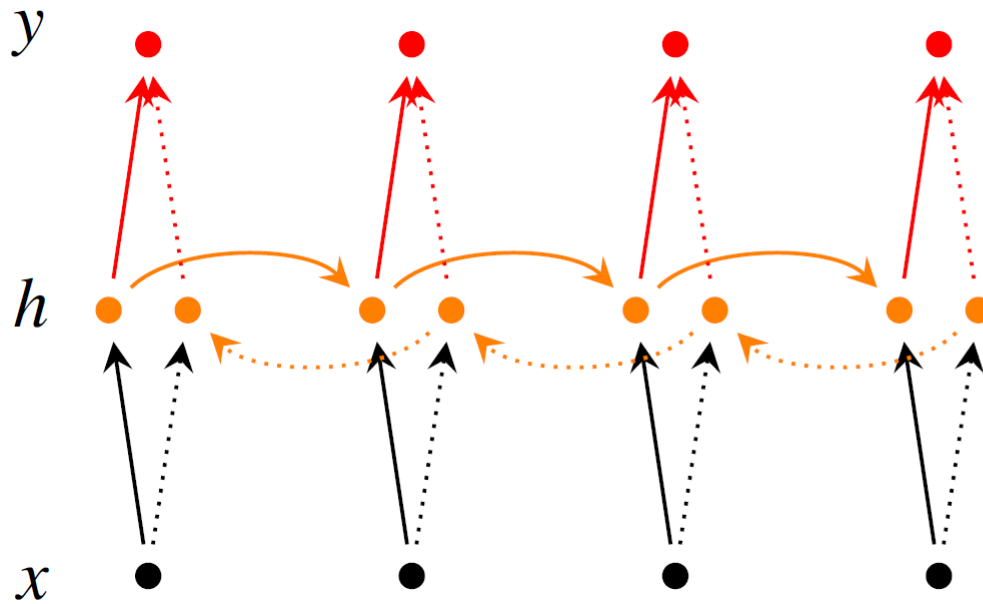
GRU



RNN Variations: Bidirectional RNN

- Bidirectional RNN: 양방향 순환신경망

✓ 정보의 입력을 시간의 순방향과 역방향 관점에서 함께 처리



$$\vec{h}_t = f(\vec{W}x_t + \vec{V}\vec{h}_{t-1} + \vec{b})$$

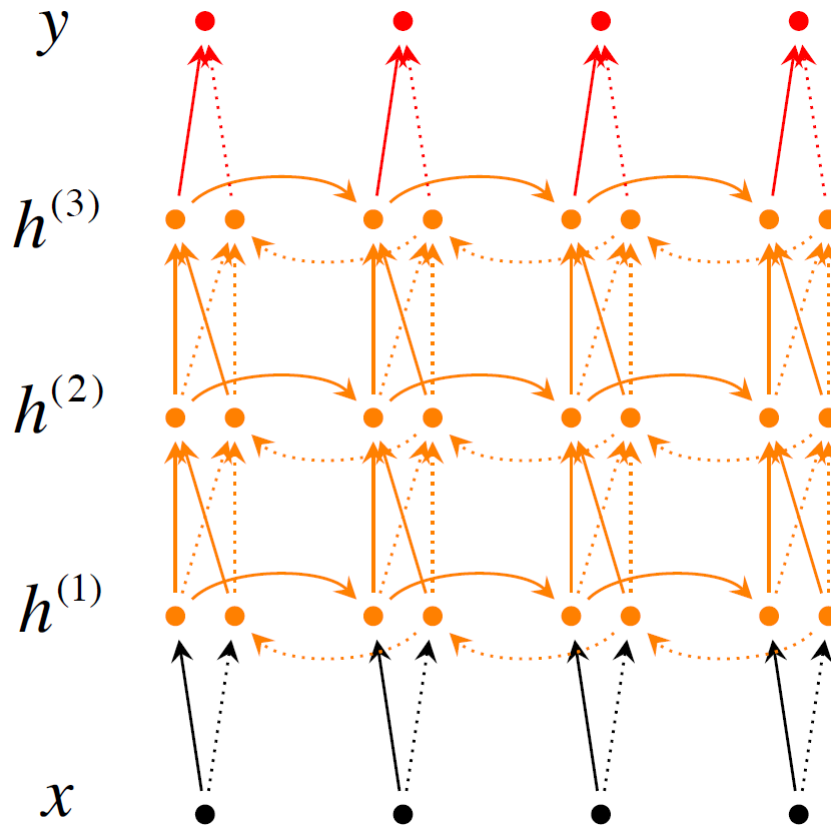
$$\overleftarrow{h}_t = f(\overleftarrow{W}x_t + \overleftarrow{V}\overleftarrow{h}_{t+1} + \overleftarrow{b})$$

$$y_t = g(U[\vec{h}_t; \overleftarrow{h}_t] + c)$$

RNN Variations: Bidirectional RNN

- Deep-Bidirectional RNN

✓ RNN의 hidden layer가 꼭 한 층일 필요가 있나? 더 쌓아보자!



$$\vec{h}_t^{(i)} = f(\vec{W}^{(i)} h_t^{(i-1)} + \vec{V}^{(i)} \vec{h}_{t-1}^{(i)} + \vec{b}^{(i)})$$

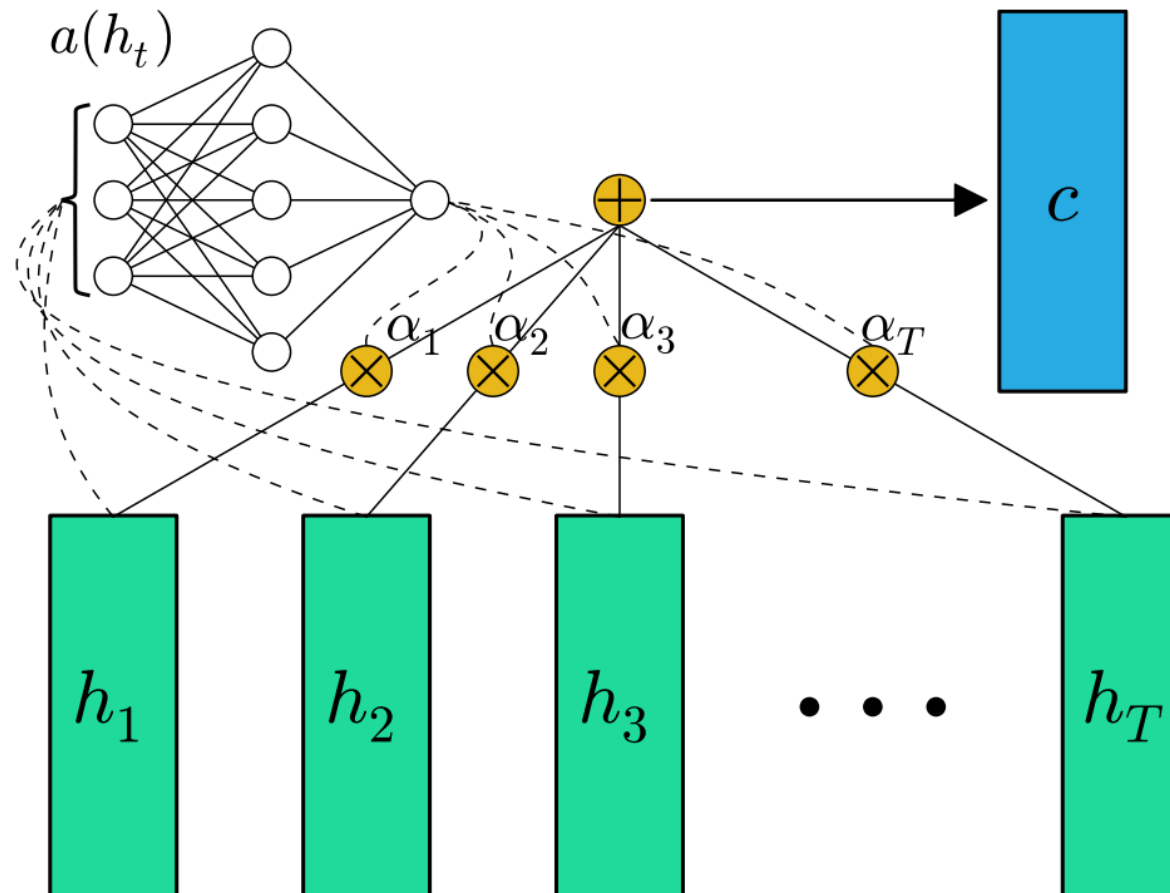
$$\overleftarrow{h}_t^{(i)} = f(\overleftarrow{W}^{(i)} h_t^{(i-1)} + \overleftarrow{V}^{(i)} \overleftarrow{h}_{t+1}^{(i)} + \overleftarrow{b}^{(i)})$$

$$y_t = g(U[\vec{h}_t^{(L)}; \overleftarrow{h}_t^{(L)}] + c)$$

RNN: Attention

- Attention

✓ 어느 시점 정보가 RNN의 최종 출력 값에 영향을 미치는지를 알려줄 수 있는 메커니즘



RNN: Attention

- 두 가지 대표적 Attention 메커니즘

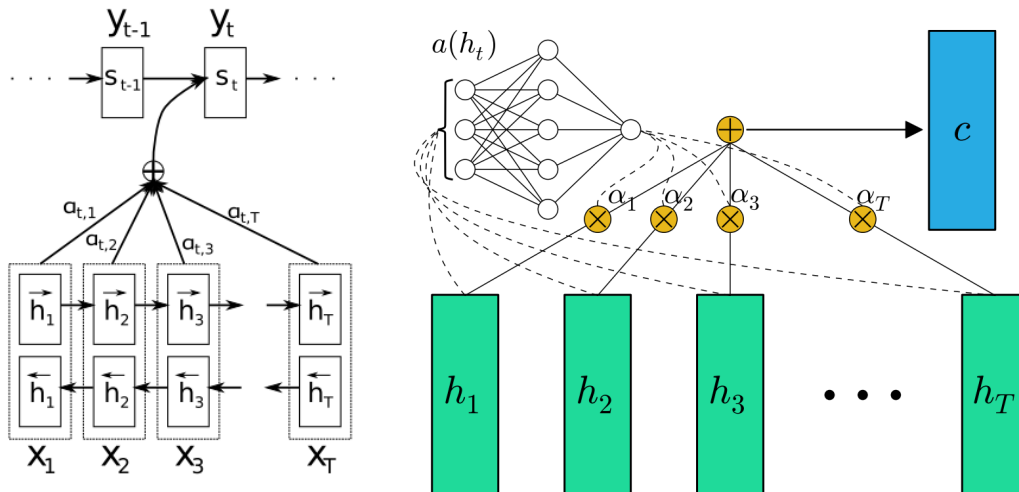
- ✓ Bahdanau attention (Bahdanau et al., 2015)

- Attention scores are separated trained, the current hidden state is a function of the context vector and the previous hidden state

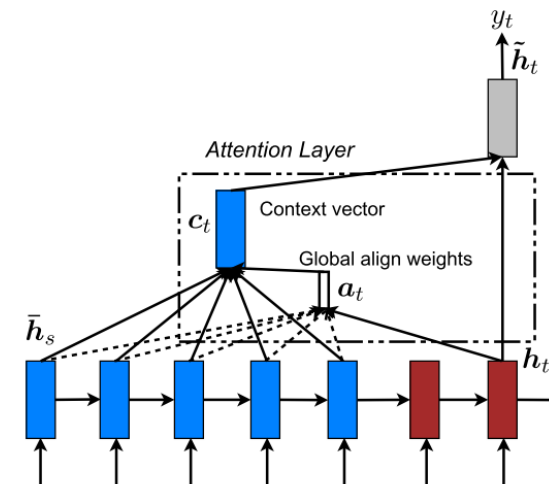
- ✓ Luong attention (Luong et al., 2015)

- Attention scores are not trained, the new current hidden state is the simple tanh of the weighted concatenation of the context vector and the current hidden state of the decoder

Bahdanau attention



Luong attention

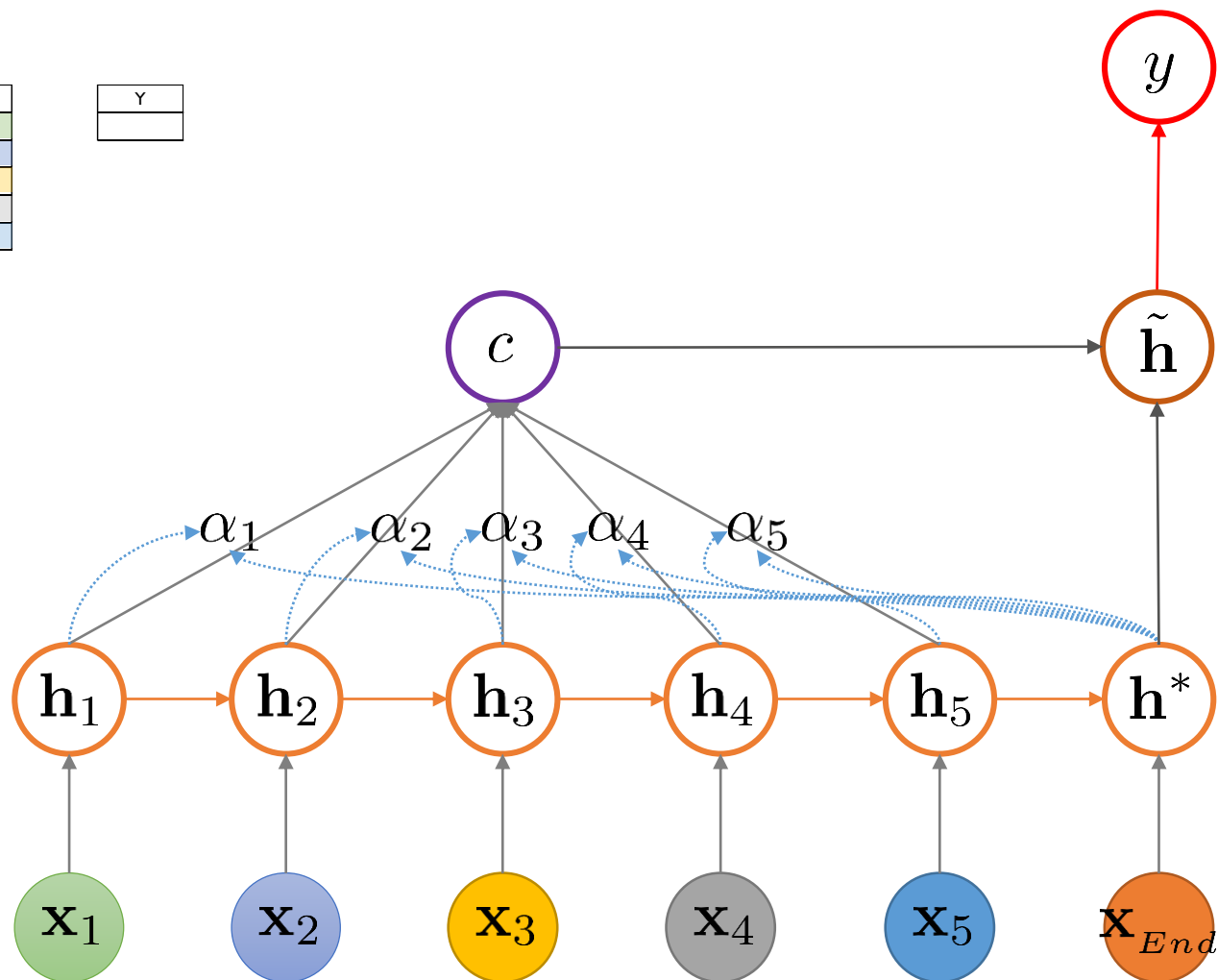


RNN: Attention

- RNN with Attention

Obs I	Var 1	Var 2	Var 3	...	Var D
Time 1					
Time 2					
Time 3					
Time 4					
Time 5					

Y



RNN: Attention

- RNN with Attention

Obs I	Var 1	Var 2	Var 3	...	Var D
Time 1					
Time 2					
Time 3					
Time 4					
Time 5					

Y

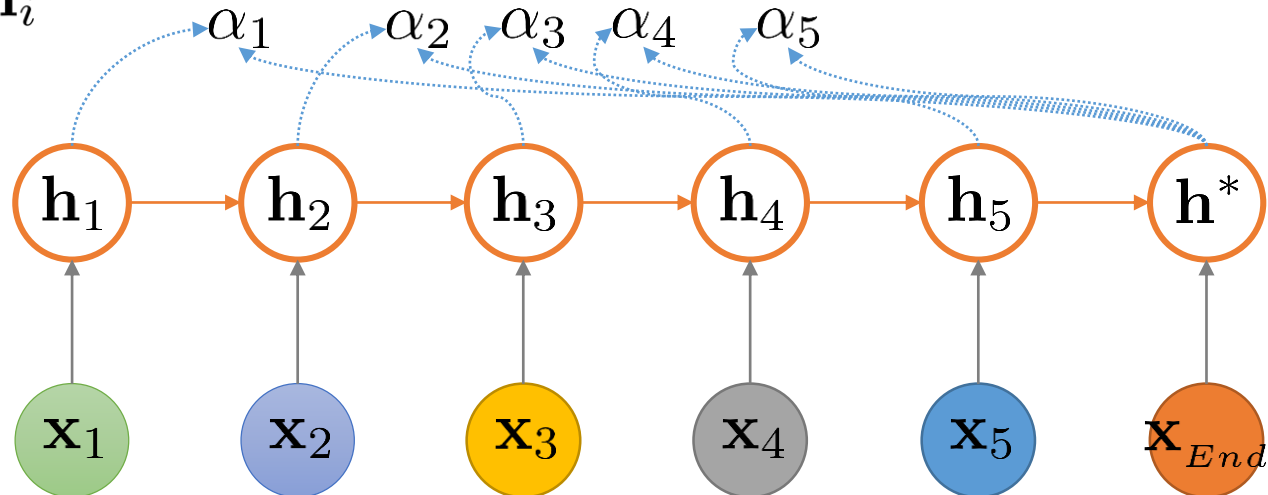
두 hidden state vector 사이의 score 산출
방식 중 가장 간단한 방식은 두 벡터의
내적을 사용하는 것

$$\text{score}(\mathbf{h}^*, \mathbf{h}_i) = \mathbf{h}^{*T} \mathbf{h}_i$$

i번째 hidden state vector가 context vector
생성에 기여하는 비중

$$\alpha_i = \text{align}(\mathbf{h}^*, \mathbf{h}_i)$$

$$= \frac{\exp(\text{score}(\mathbf{h}^*, \mathbf{h}_i))}{\sum_{i'} \exp(\text{score}(\mathbf{h}^*, \mathbf{h}'_i))}$$



RNN: Attention

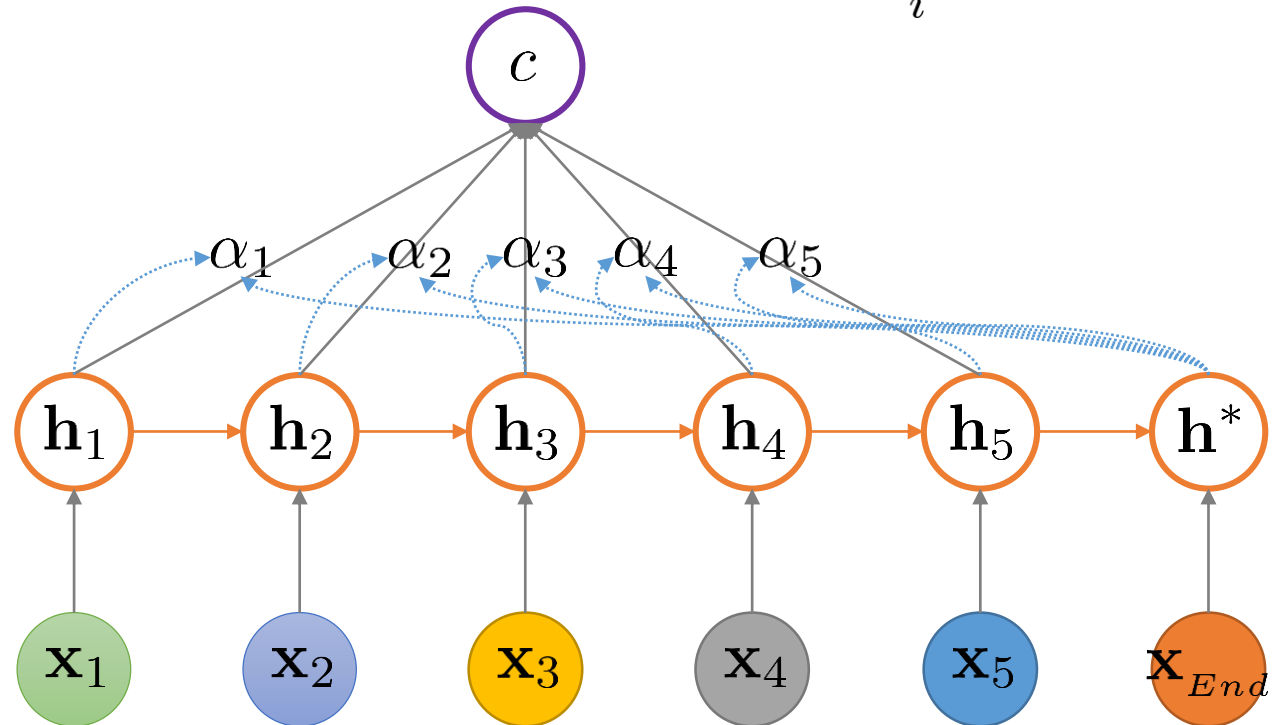
- RNN with Attention

Obs I	Var 1	Var 2	Var 3	...	Var D
Time 1					
Time 2					
Time 3					
Time 4					
Time 5					

Y

Context vector는 각 입력 단계의 hidden state의 선형 결합 (가중치 = alpha)

$$\mathbf{c} = \sum_i \alpha_i \mathbf{h}_i$$



RNN: Attention

- RNN with Attention

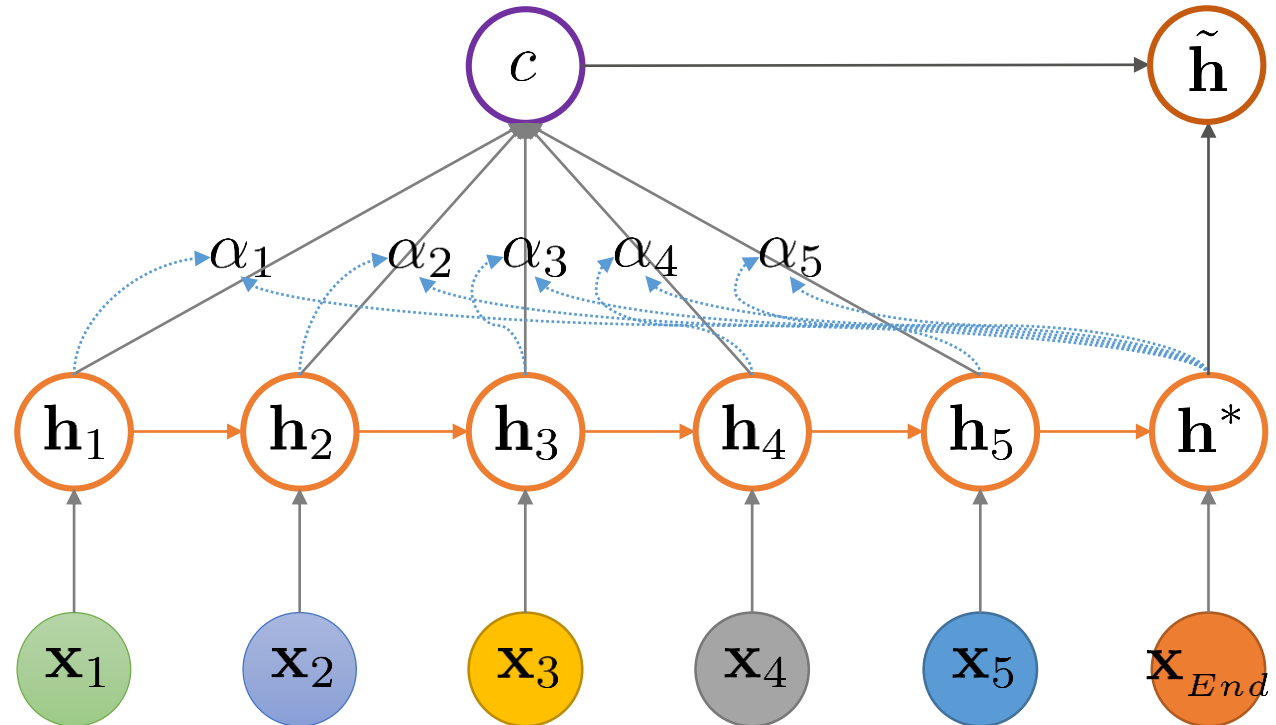
Obs I	Var 1	Var 2	Var 3	...	Var D
Time 1					
Time 2					
Time 3					
Time 4					
Time 5					

Y

Attention이 고려된 hidden state는 context vector와 기존 hidden state를 concatenation한 뒤 선형 및 비선형 변환을

수행

$$\tilde{\mathbf{h}} = \tanh(\mathbf{W}_c[\mathbf{c}; \mathbf{h}^*])$$



RNN: Attention

- RNN with Attention

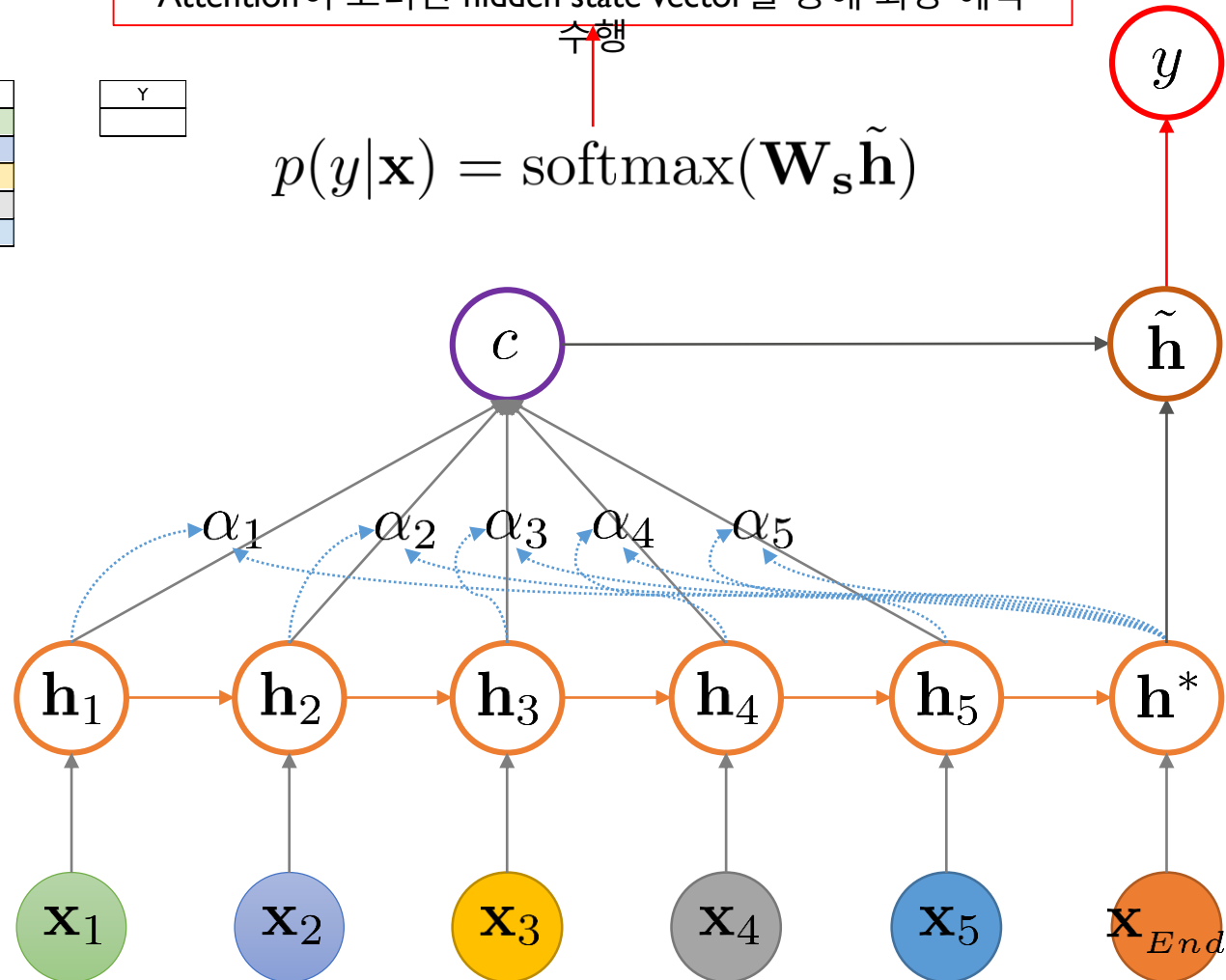
Obs I	Var 1	Var 2	Var 3	...	Var D
Time 1					
Time 2					
Time 3					
Time 4					
Time 5					

Y

Attention이 고려된 hidden state vector를 통해 최종 예측

수행

$$p(y|\mathbf{x}) = \text{softmax}(\mathbf{W}_s \tilde{\mathbf{h}})$$



RNN: Attention

- Luong Attention

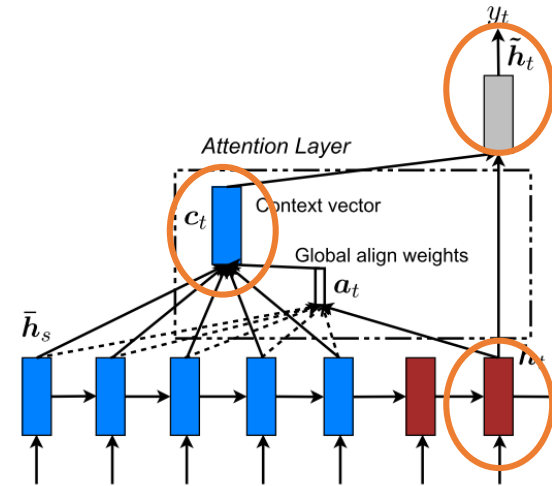
✓ Decoder의 새로운 hidden state는

- Weighted concatenation of the context vector와
- Current hidden state of the decoder를
- Concatenation한 뒤 Tanh함수를 적용한 것

$$\tilde{\mathbf{h}}_t = \tanh(\mathbf{W}_c[\mathbf{c}_t; \mathbf{h}_t])$$

- 이 hidden state가 RNN의 출력을 결정하는 softmax에 입력으로 투입

$$p(y_t | y_{y < t}, x) = \text{softmax}(\mathbf{W}_s \tilde{\mathbf{h}}_t)$$



RNN: Attention

- Luong attention

- ✓ A variable-length alignment vector:

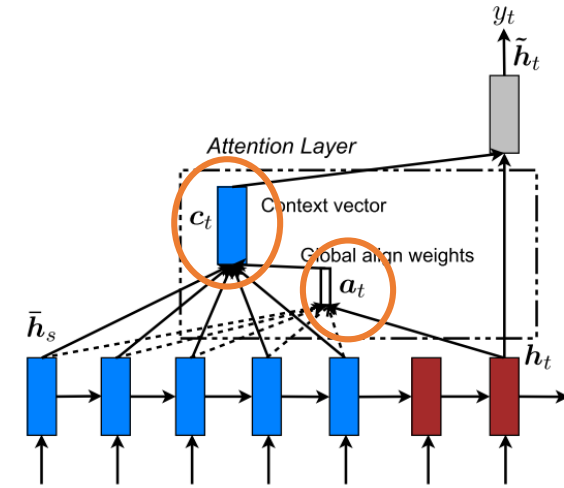
$$\mathbf{a}_t(s) = \text{align}(\mathbf{h}_t, \bar{\mathbf{h}}_s) = \frac{\exp(\text{score}(\mathbf{h}_t, \bar{\mathbf{h}}_s))}{\sum_{s'} \exp(\text{score}(\mathbf{h}_t, \bar{\mathbf{h}}_{s'}))}$$

- ✓ **score** is referred as a **context-based function**:

$$\text{score}(\mathbf{h}_t, \bar{\mathbf{h}}_s) = \begin{cases} \mathbf{h}_t^T \bar{\mathbf{h}}_s, & \text{dot} \\ \mathbf{h}_t^T \mathbf{W}_a \bar{\mathbf{h}}_s, & \text{general} \\ \mathbf{v}_a^T \tanh(\mathbf{W}_c [\mathbf{c}_t; \mathbf{h}_t]), & \text{concat} \end{cases}$$

- ✓ Context vector

$$\mathbf{c}_t = \bar{\mathbf{h}}_s \mathbf{a}_t$$

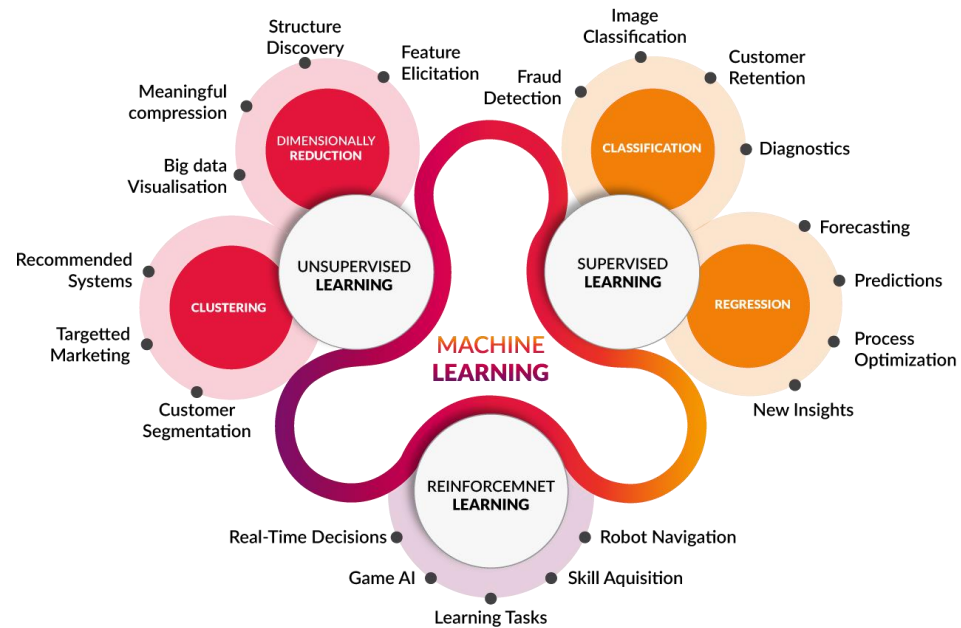


Summary

- 순환 신경망을 활용한 시계열 회귀

- ✓ 순환신경망은 순차 데이터를 처리하는데 특화된 인공신경망 구조이다.
- ✓ 현 시점에서의 Hidden State(H_t)는 이전 시점에서의 Hidden State(H_{t-1})와 현 시점에서의 입력(x_t)을 이용한 연산을 통해 계산된다.
- ✓ Vanilla RNN은 Long-term relationship을 파악하는데 어려움을 겪기 때문에 이를 해결하기 위해 LSTM/GRU 구조가 제안되었다.
 - Long Short-Term Memory (LSTM) 구조는 Input/Forget/Output gate와 Cell State를 활용하여 정보를 선택적으로 활용한다.
 - Gated Recurrent Unit (GRU)는 Reset/Update 두 가지 gates만 사용하여 LSTM에 비해 보다 단순화된 구조이다.
- ✓ Attention은 시계열 데이터의 어느 시점이 최종 예측에 영향을 주는지를 판별할 수 있는 기법이며, Attention을 추가한 구조가 그렇지 않은 구조보다 일반적으로 예측 성능이 우수한 경향이 있다.





합성곱 신경망 기반의 시계열 회귀

강필성

고려대학교 산업경영공학부

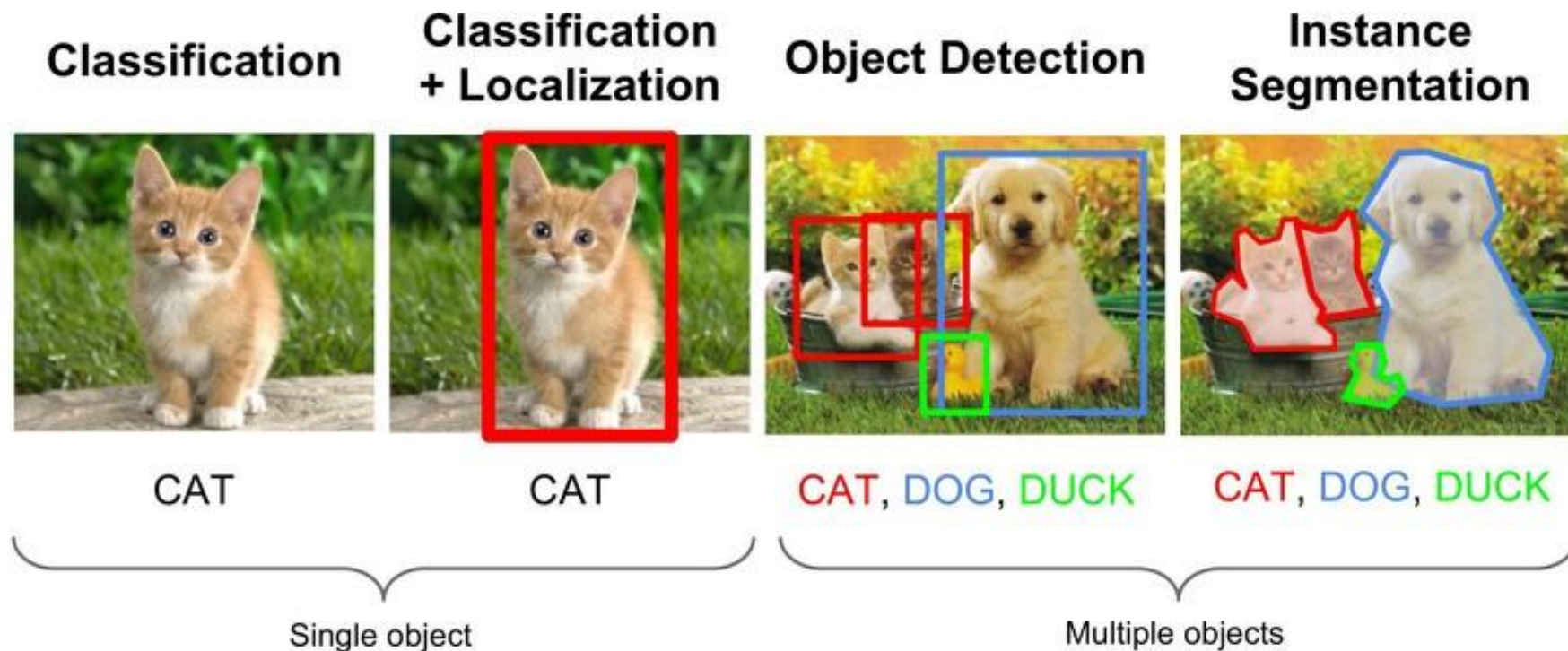
pilsung_kang@korea.ac.kr

Convolutional Neural Network (CNN)

- 합성곱 신경망

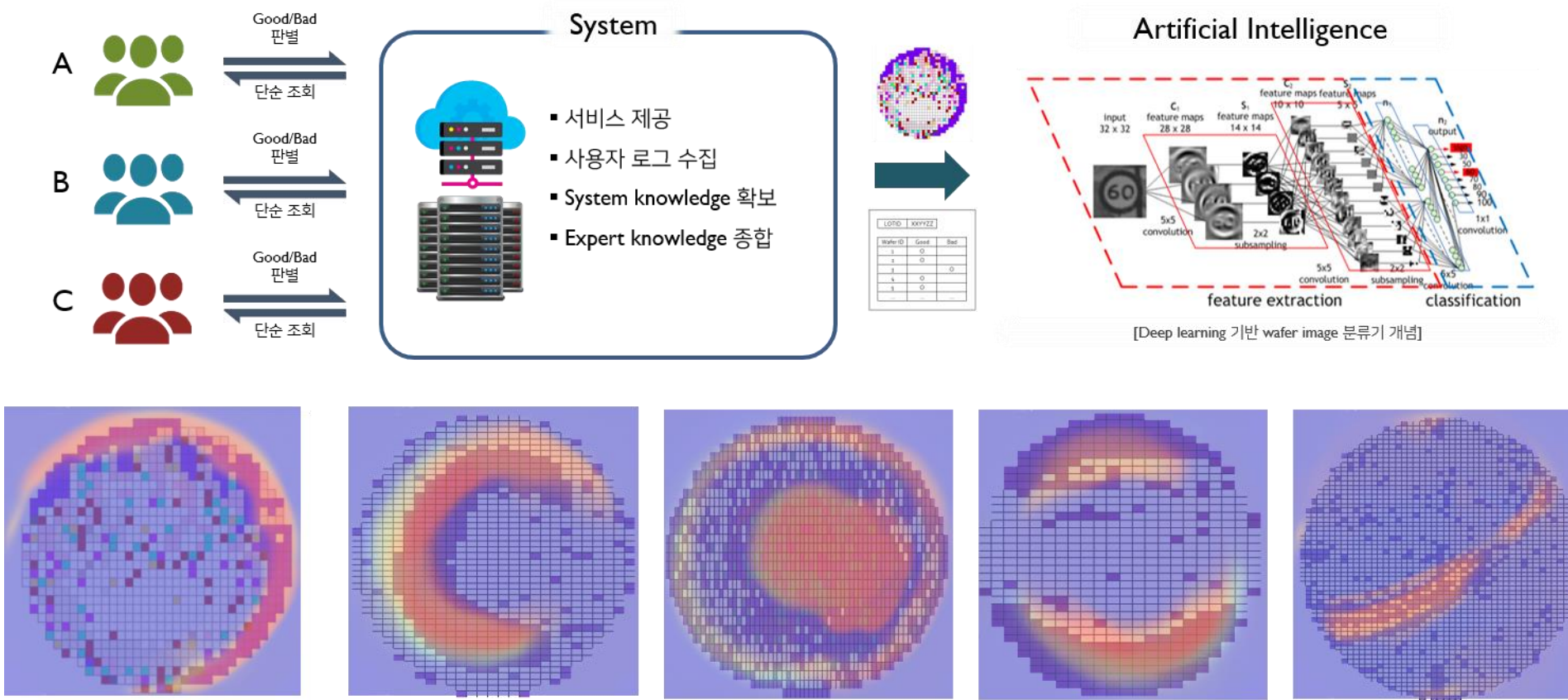
- ✓ Convolution 연산을 통해 이미지로부터 필요한 특징^{feature}을 스스로 학습할 수 있는 능력을 갖춘 심층 신경망 구조

- 이미지 인식 종류



Convolutional Neural Network (CNN)

- 제조업 활용 사례 2: 불량 제품 및 원인 영역 탐지 (by DSBA@KU with Samsung)



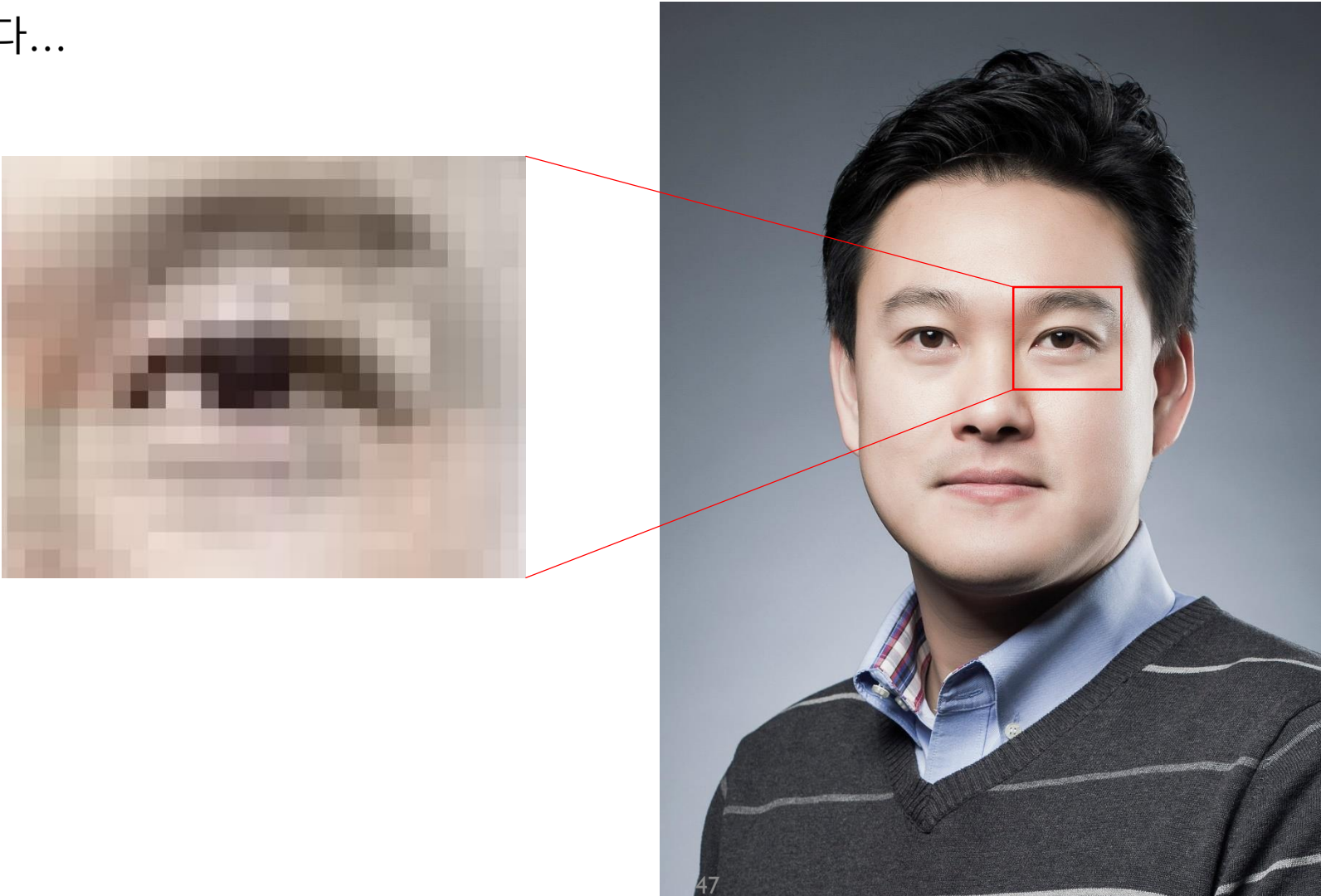
CNN Basics: Image Representation

- Image Representation: 이미지를 어떻게 컴퓨터한테 숫자로 인식시킬 것인가?
 - ✓ 누구의 눈일까요?



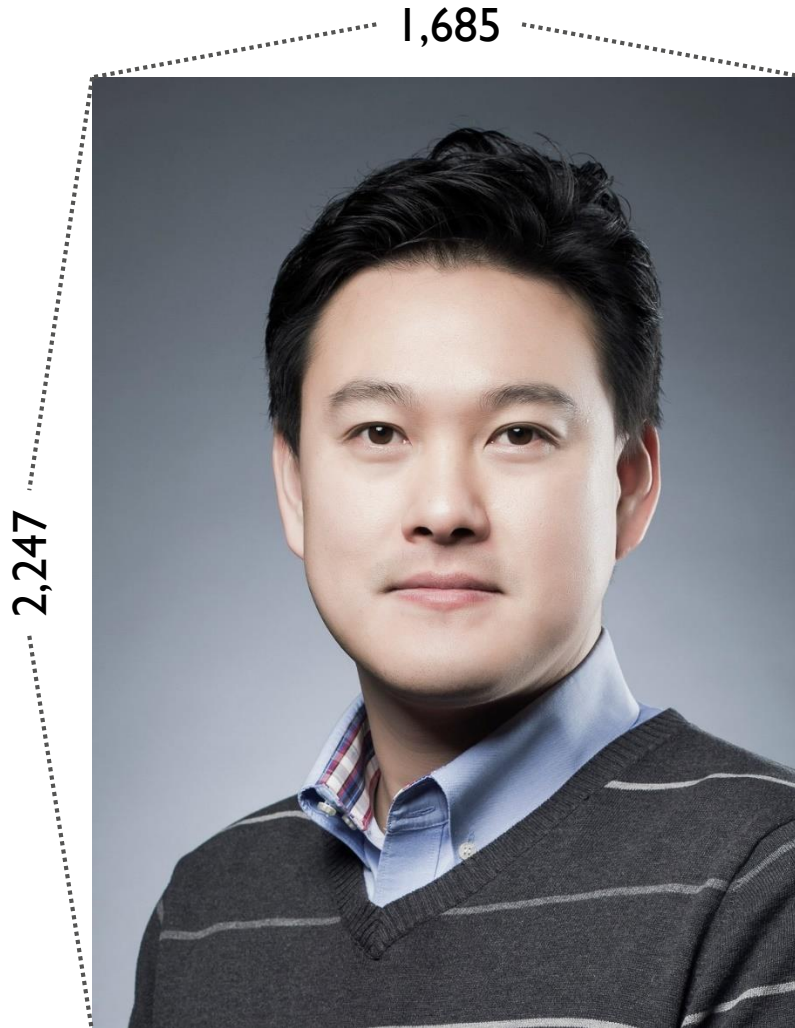
CNN Basics: Image Representation

- Image Representation: 이미지를 어떻게 컴퓨터한테 숫자로 인식시킬 것인가?
✓ 제 눈입니다...



CNN Basics: Image Representation

- 컬러 이미지는 3차원의 Tensor로 표현됨: Width X Height X 3 (RGB)



```
> ps kang[1:10, 1:10, 1]
      [,1]      [,2]      [,3]      [,4]      [,5]      [,6]      [,7]      [,8]      [,9]     [,10]
[1,] 0.2784314 0.2784314 0.2745098 0.2745098 0.2745098 0.2745098 0.2705882 0.2705882 0.2862745 0.2901961
[2,] 0.2745098 0.2745098 0.2745098 0.2745098 0.2745098 0.2745098 0.2745098 0.2745098 0.2666667 0.2705882
[3,] 0.2745098 0.2745098 0.2745098 0.2745098 0.2745098 0.2745098 0.2745098 0.2745098 0.2705882 0.2745098
[4,] 0.2705882 0.2705882 0.2745098 0.2745098 0.2745098 0.2745098 0.2745098 0.2784314 0.2784314 0.2823529
[5,] 0.2705882 0.2705882 0.2705882 0.2745098 0.2745098 0.2784314 0.2784314 0.2784314 0.2784314 0.2784314
[6,] 0.2705882 0.2705882 0.2745098 0.2745098 0.2745098 0.2745098 0.2784314 0.2784314 0.2823529 0.2784314
[7,] 0.2705882 0.2745098 0.2745098 0.2745098 0.2745098 0.2745098 0.2745098 0.2784314 0.2941176 0.2862745
[8,] 0.2745098 0.2745098 0.2745098 0.2745098 0.2745098 0.2745098 0.2745098 0.2745098 0.2901961 0.2823529
[9,] 0.2745098 0.2705882 0.2745098 0.2784314 0.2823529 0.2823529 0.2745098 0.2666667 0.2784314 0.2784314
[10,] 0.2784314 0.2784314 0.2784314 0.2823529 0.2862745 0.2862745 0.2784314 0.2745098 0.2745098 0.2745098
```

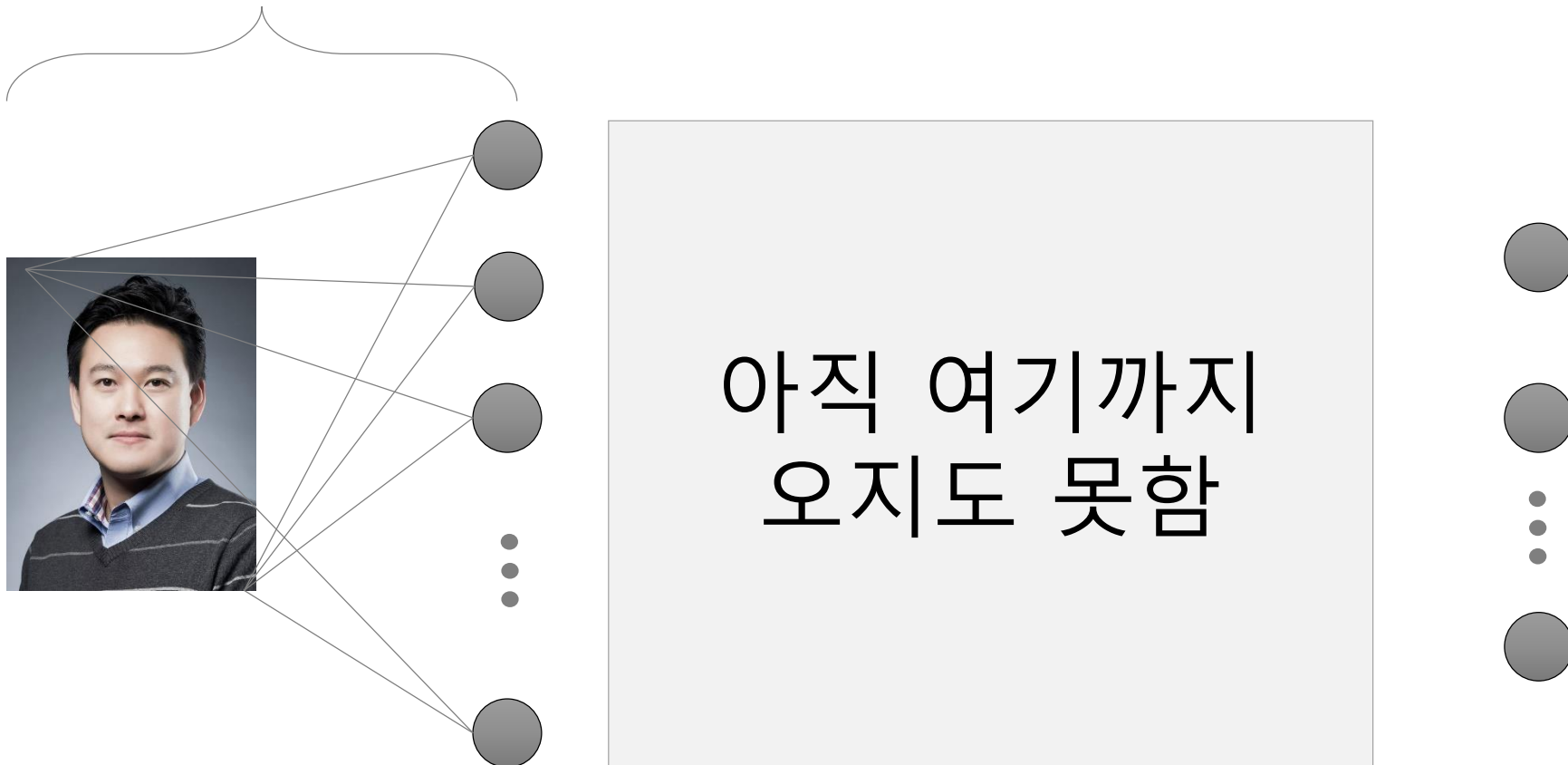
```
> ps kang[1:10, 1:10, 2]
      [,1]      [,2]      [,3]      [,4]      [,5]      [,6]      [,7]      [,8]      [,9]     [,10]
[1,] 0.2980392 0.2980392 0.2941176 0.2941176 0.2941176 0.2941176 0.2901961 0.2901961 0.2980392 0.3019608
[2,] 0.2941176 0.2941176 0.2941176 0.2941176 0.2941176 0.2941176 0.2941176 0.2941176 0.2784314 0.2823529
[3,] 0.2941176 0.2941176 0.2941176 0.2941176 0.2941176 0.2941176 0.2941176 0.2941176 0.2823529 0.2862745
[4,] 0.2901961 0.2901961 0.2941176 0.2941176 0.2941176 0.2941176 0.2980392 0.2980392 0.2941176 0.2941176
[5,] 0.2901961 0.2901961 0.2901961 0.2941176 0.2941176 0.2980392 0.2980392 0.2980392 0.2901961 0.2901961
[6,] 0.2901961 0.2901961 0.2941176 0.2941176 0.2941176 0.2941176 0.2980392 0.2980392 0.2941176 0.2901961
[7,] 0.2901961 0.2941176 0.2941176 0.2941176 0.2941176 0.2941176 0.2941176 0.2980392 0.3058824 0.2980392
[8,] 0.2941176 0.2941176 0.2941176 0.2941176 0.2941176 0.2941176 0.2941176 0.2941176 0.3019608 0.2941176
[9,] 0.2862745 0.2823529 0.2862745 0.2901961 0.2941176 0.2941176 0.2862745 0.2784314 0.2901961 0.2901961
[10,] 0.2901961 0.2901961 0.2901961 0.2941176 0.2980392 0.2980392 0.2901961 0.2862745 0.2862745 0.2862745
```

```
> ps kang[1:10, 1:10, 3]
      [,1]      [,2]      [,3]      [,4]      [,5]      [,6]      [,7]      [,8]      [,9]     [,10]
[1,] 0.3176471 0.3176471 0.3176471 0.3176471 0.3176471 0.3176471 0.3137255 0.3137255 0.3254902 0.3294118
[2,] 0.3176471 0.3176471 0.3176471 0.3176471 0.3176471 0.3176471 0.3176471 0.3176471 0.3058824 0.3098039
[3,] 0.3176471 0.3176471 0.3176471 0.3176471 0.3176471 0.3176471 0.3176471 0.3176471 0.3098039 0.3137255
[4,] 0.3137255 0.3137255 0.3176471 0.3176471 0.3176471 0.3176471 0.3215686 0.3215686 0.3215686 0.3215686
[5,] 0.3137255 0.3137255 0.3137255 0.3176471 0.3176471 0.3215686 0.3215686 0.3215686 0.3176471 0.3176471
[6,] 0.3137255 0.3137255 0.3176471 0.3176471 0.3176471 0.3176471 0.3215686 0.3215686 0.3215686 0.3176471
[7,] 0.3137255 0.3176471 0.3176471 0.3176471 0.3176471 0.3176471 0.3176471 0.3215686 0.3333333 0.3254902
[8,] 0.3176471 0.3176471 0.3176471 0.3176471 0.3176471 0.3176471 0.3176471 0.3176471 0.3294118 0.3215686
[9,] 0.3058824 0.3019608 0.3058824 0.3098039 0.3137255 0.3137255 0.3058824 0.2980392 0.3098039 0.3098039
[10,] 0.3098039 0.3098039 0.3098039 0.3137255 0.3176471 0.3176471 0.3098039 0.3058824 0.3058824 0.3058824
```


CNN Basics: Image Representation

- 문제점

- ✓ 모든 픽셀 하나의 입력 노드로 간주하고 서로 다른 가중치로 연결하게 되면 Input layer와 First hidden layer 사이에 너무 많은 weights가 생김
 - 이 단계에서 필요한 가중치의 수: $1,685 \times 2,247 \times 3 \times$ 첫 번째 Hidden Layer의 노드 수



CNN Basics: Convolution

- 문제점

- ✓ 모든 픽셀을 서로 다른 가중치로 연결하게 되면 Input layer와 First hidden layer 사이에 너무 많은 weights가 생김

- Image Convolution (Filter, Kernel)

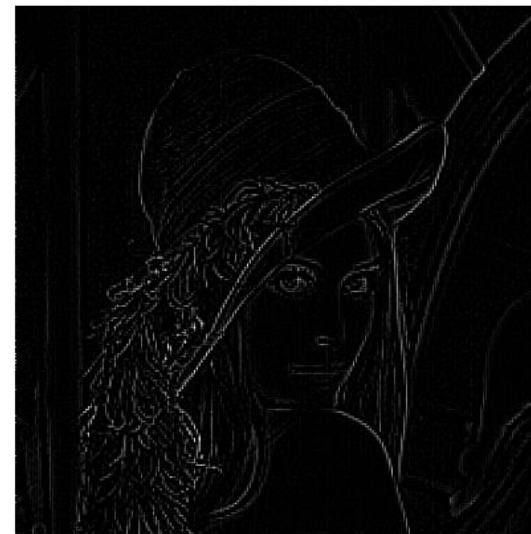
- ✓ 특정 속성을 탐지하는데 사용하는 matrix (예: edge detection)



파일 선택 220px-Lenna.png Live video

-1	-1	-1
-1	8	-1
-1	-1	-1

outline ▼



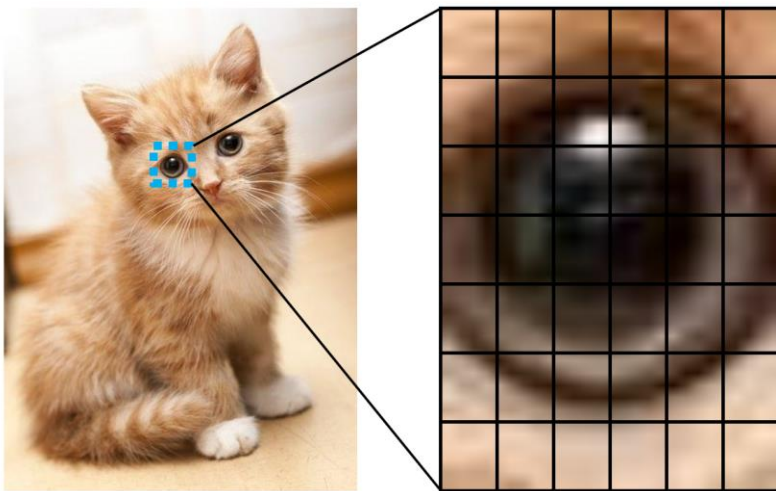
She appeared on the cover of the journal *Optical Engineering* in 1991, and the model herself was invited to a conference of the Image Science and Technology society in 1997. (덕중의 덕은 양덕이니라)

Convolutional Neural Network (CNN)

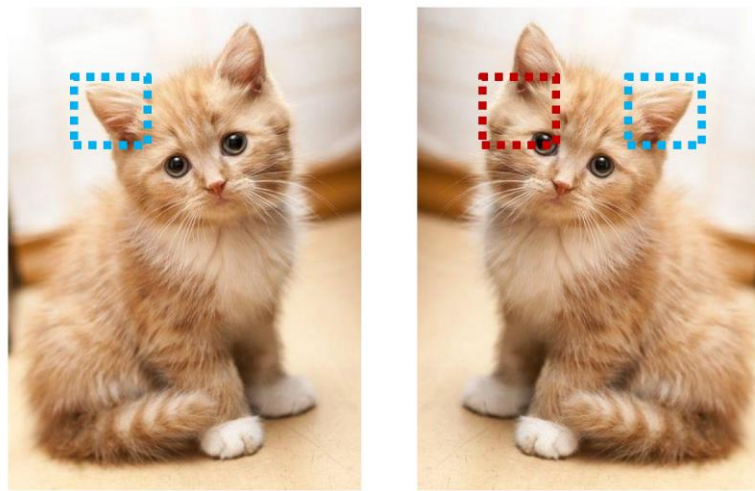
- 합성곱 신경망 Convolutional Neural Network (CNN)

- ✓ 합성곱 Convolution 연산을 통해 이미지로부터 필요한 특징 feature 을 스스로 학습할 수 있는 능력을 갖춘 심층 신경망 구조
- ✓ 이미지 데이터가 갖는 특징
 - 인접 픽셀간 높은 상관관계 (spatially-local correlation)
 - 이미지의 부분적 특성(e.g., 눈, 귀)은 고정된 위치에 등장하지 않음(feature invariance)

Spatially-Local Correlation



Invariant Feature



Convolutional Neural Network (CNN)

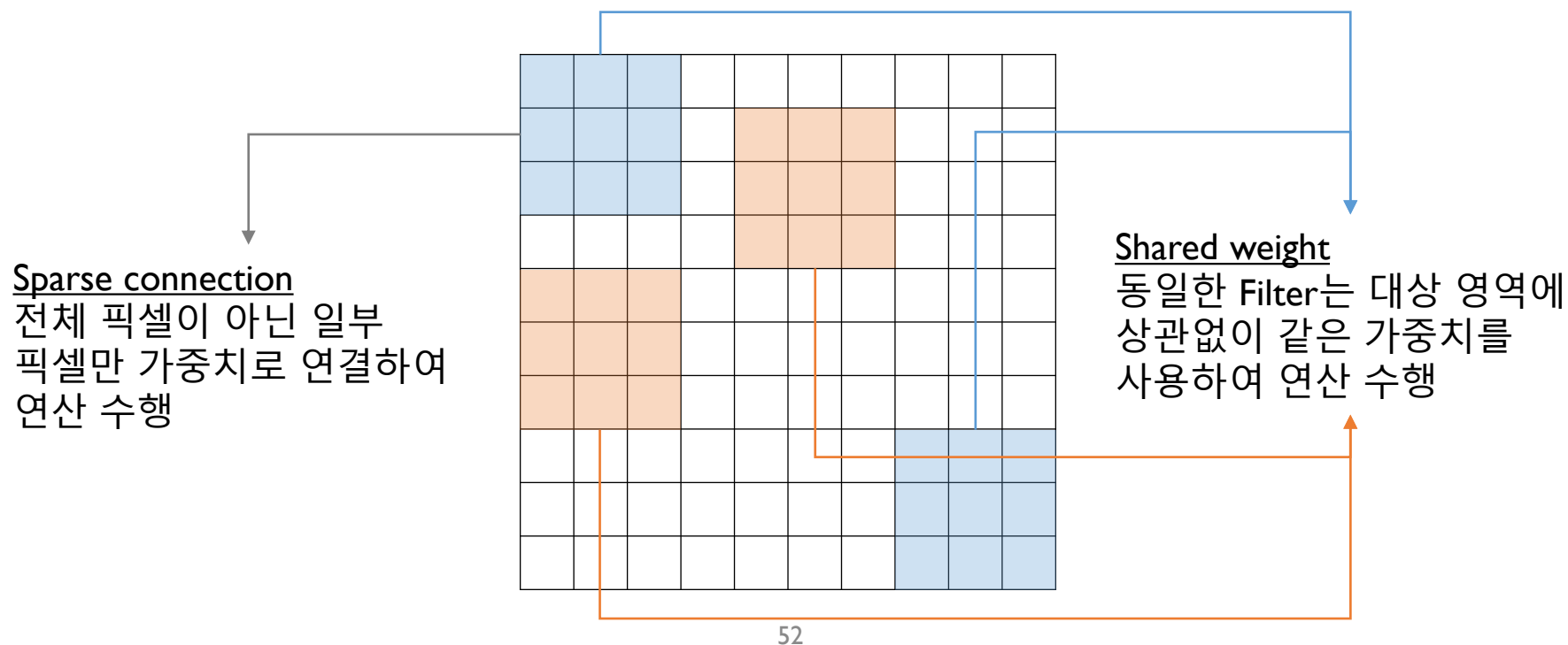
- 합성곱 연산 Convolution

- ✓ Spatially-local correlation을 고려하기 위한 Sparse connection 구성

- 인접한 변수만을 이용하여 새로운 Feature 생성

- ✓ Invariant feature를 추출하기 위해 Shared weight 개념 이용

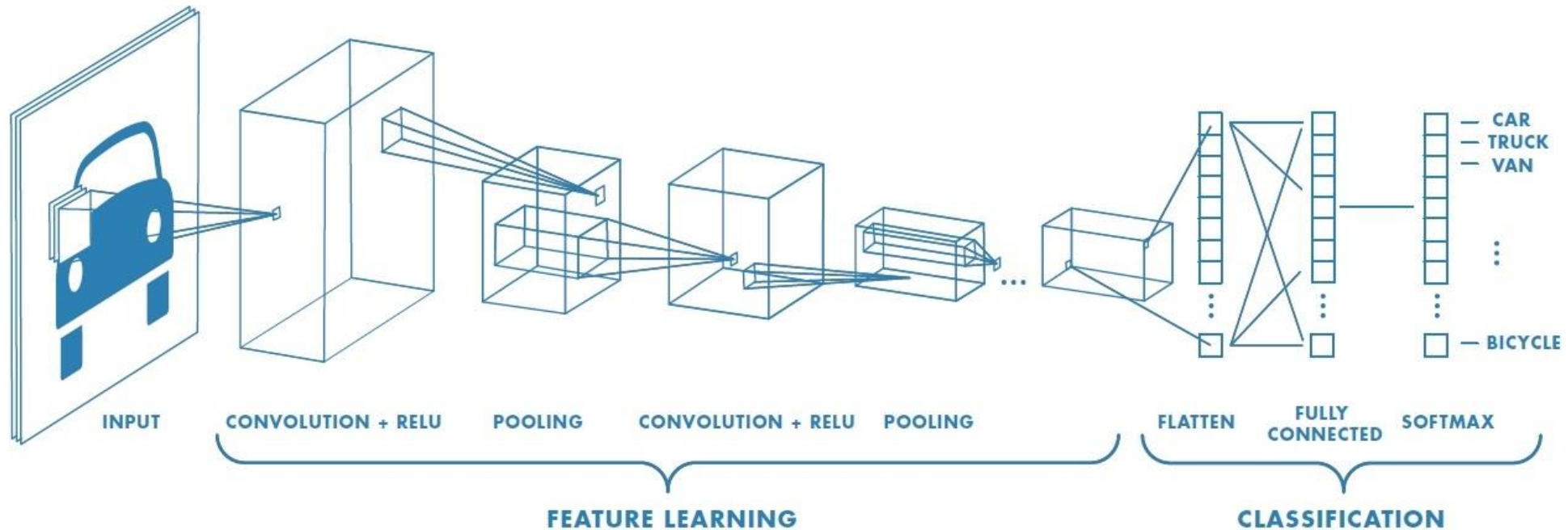
- 특정 특질을 추출하기 위한 도구로서 같은 대상 크기에는 위치가 다르더라도 동일한 Weight 적용



Convolutional Neural Network (CNN)

- CNN 작동 과정

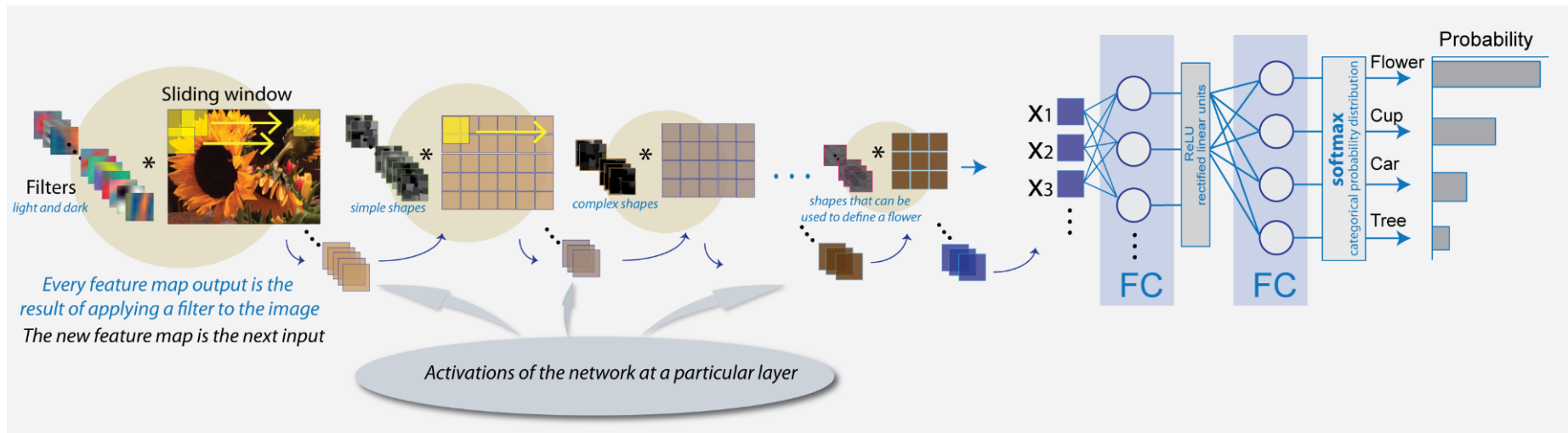
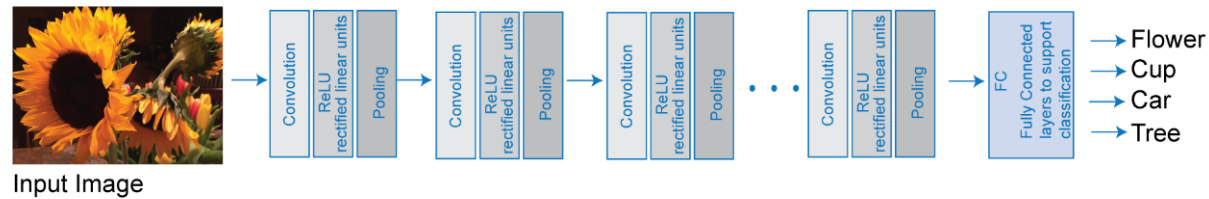
- ✓ 일반적인 CNN은 Convolution 연산, Activation 연산(대부분 ReLU), Pooling 연산의 반복으로 구성됨 (Feature Learning)
- ✓ 일정 횟수 이상의 Feature Learning 과정 이후에는 Flatten 과정을 통해 이미지가 1차원의 벡터로 변환됨



Convolutional Neural Network (CNN)

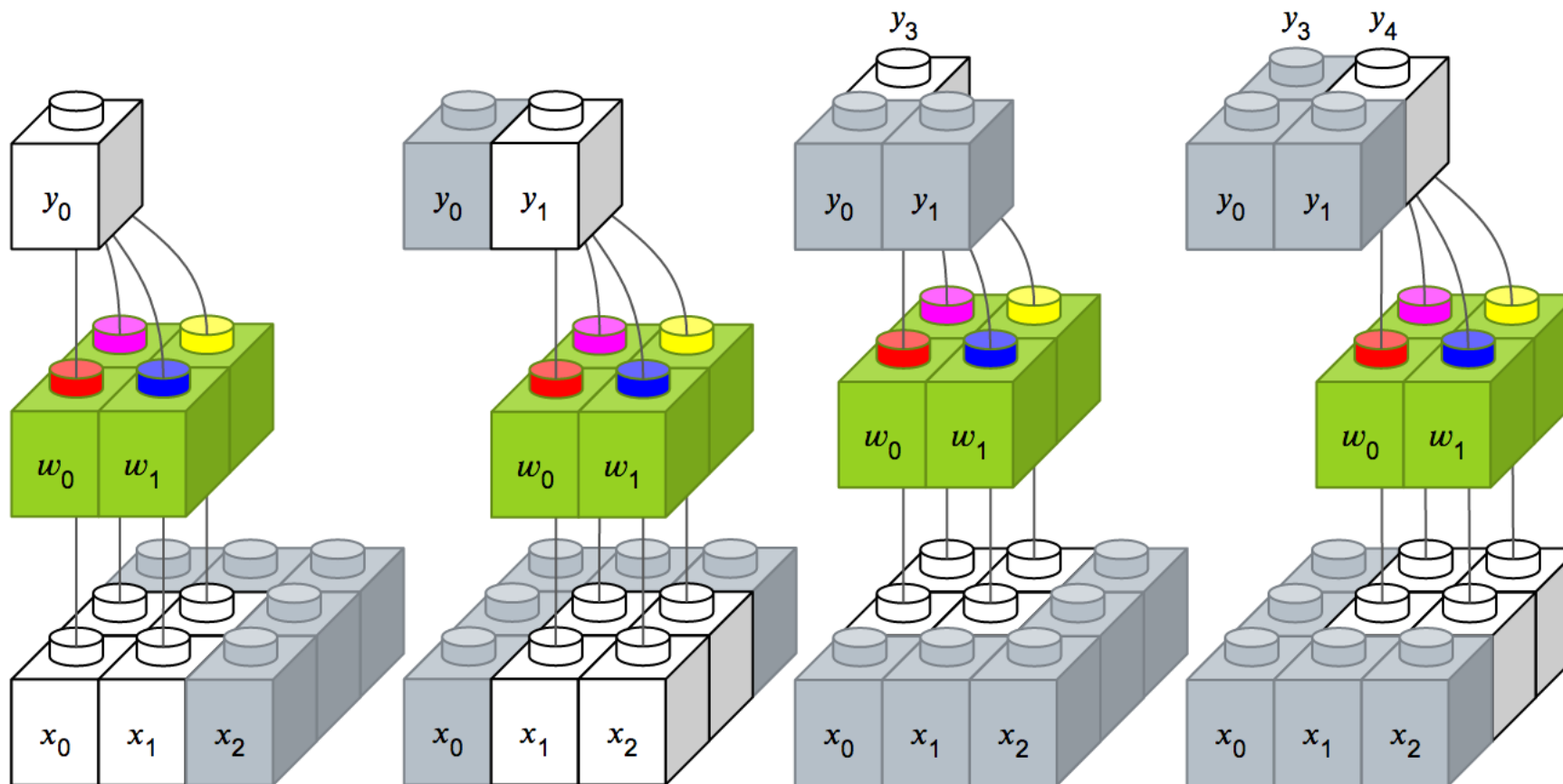
- CNN 작동 과정

- ✓ 일반적인 CNN은 Convolution 연산, Activation 연산(대부분 ReLU), Pooling 연산의 반복으로 구성됨 (Feature Learning)
- ✓ 일정 횟수 이상의 Feature Learning 과정 이후에는 Flatten 과정을 통해 이미지가 1차원의 벡터로 변환됨



Convolutional Neural Network (CNN)

- CNN 연산의 직관적 설명



https://tykimos.github.io/2017/01/27/CNN_Layer_Talk/

CNN Basics: Convolution

- Image Convolution (Filter, Kernel)

✓ 특정 속성을 탐지하는데 사용하는 matrix

1	1	1	0	0
0	1	1	1	0
0	0	1	1	1
0	0	1	1	0
0	1	1	0	0

Input

1	0	1
0	1	0
1	0	1

Filter / Kernel

CNN Basics: Convolution

- Image Convolution (Filter, Kernel)

✓ 특정 속성을 탐지하는데 사용하는 matrix

1x1	1x0	1x1	0	0
0x0	1x1	1x0	1	0
0x1	0x0	1x1	1	1
0	0	1	1	0
0	1	1	0	0

4		

<https://towardsdatascience.com/applied-deep-learning-part-4-convolutional-neural-networks-584bc134c1e2>

CNN: Wrap-Up

- CNN 기본 유닛: 합성곱 연산Convolution

1	1	1	0	0
0	1	1	1	0
0	0	1	1	1
0	0	1	1	0
0	1	1	0	0

Input

1	0	1
0	1	0
1	0	1

Filter / Kernel

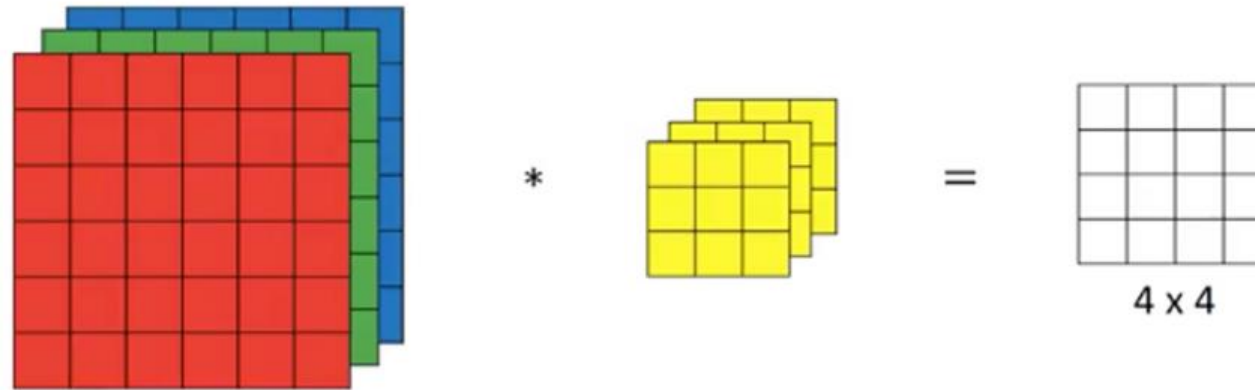
1x1	1x0	1x1	0	0
0x0	1x1	1x0	1	0
0x1	0x0	1x1	1	1
0	0	1	1	0
0	1	1	0	0

4		

<https://towardsdatascience.com/applied-deep-learning-part-4-convolutional-neural-networks-584bc134c1e2>

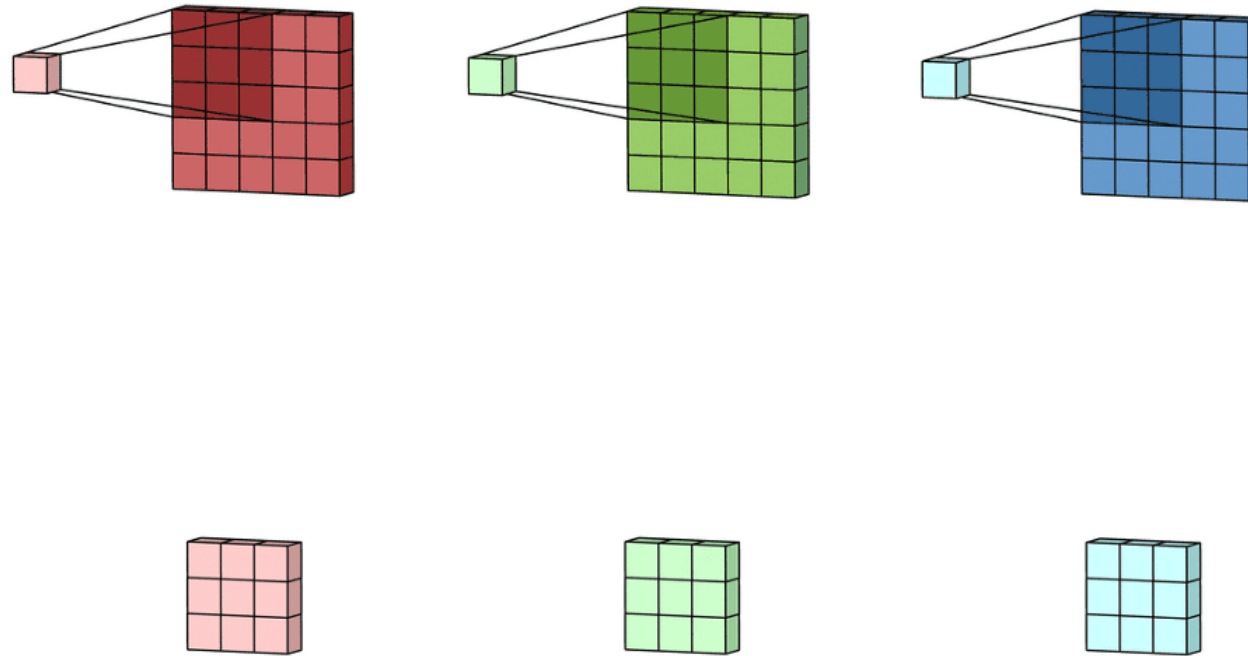
CNN Basics: Convolution

- 이미지는 3차원 Tensor(가로 픽셀, 세로 픽셀, RGB)이므로 필터 역시 3차원임



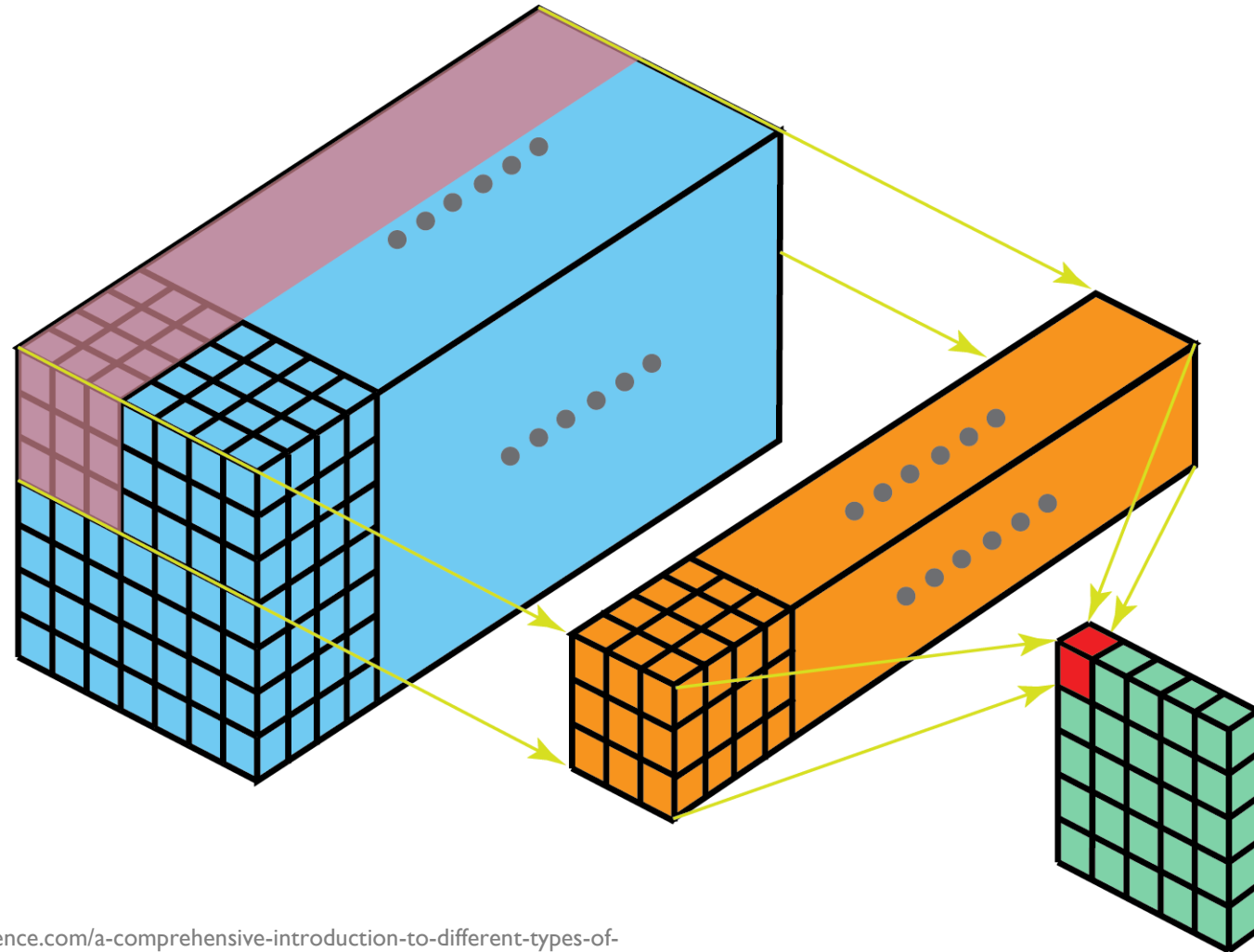
CNN Basics: Convolution

- 이미지는 3차원 Tensor(가로 픽셀, 세로 픽셀, RGB)이므로 필터 역시 3차원임



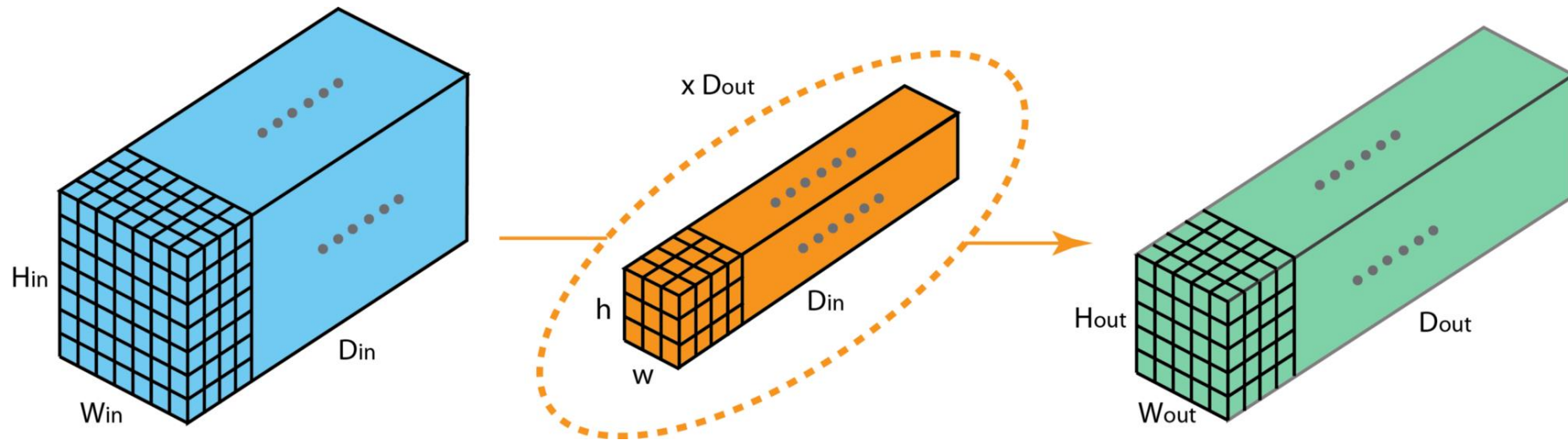
CNN Basics: Convolution

- 이를 일반화하면...



CNN Basics: Convolution

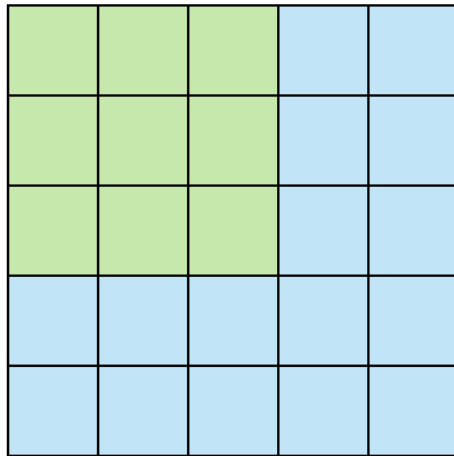
- 이를 일반화하면...



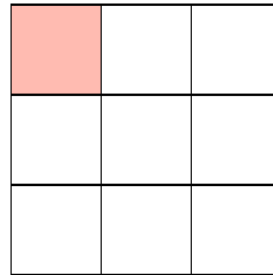
CNN Basics: Convolution

- Image Convolution (Filter, Kernel)

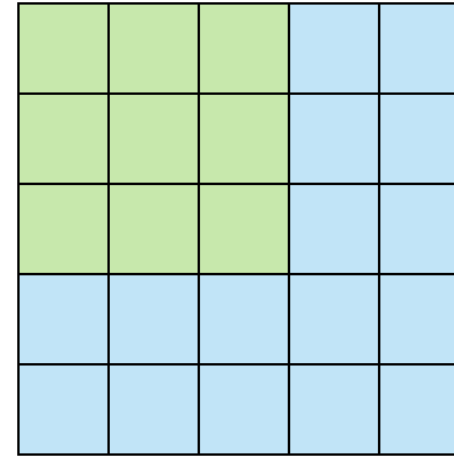
- ✓ 이슈: 한번에 한 칸씩 이동하면 너무 오래 걸리지 않을까?
- ✓ 해결책: Filter가 한번에 여러 칸을 이동하도록 허용하자 (Stride)



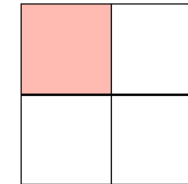
Stride 1



Feature Map



Stride 2

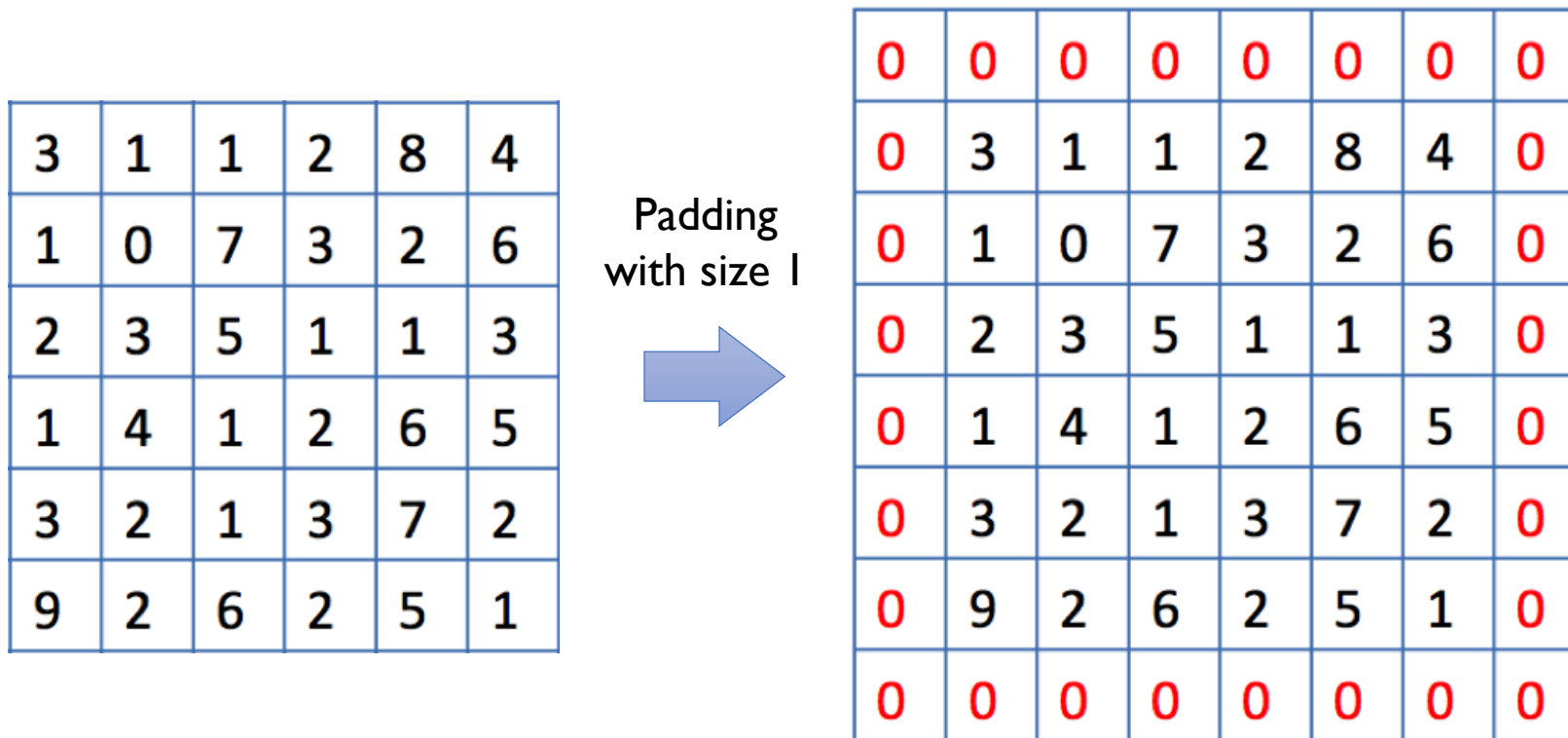


Feature Map

CNN Basics: Convolution

- Image Convolution (Filter, Kernel)

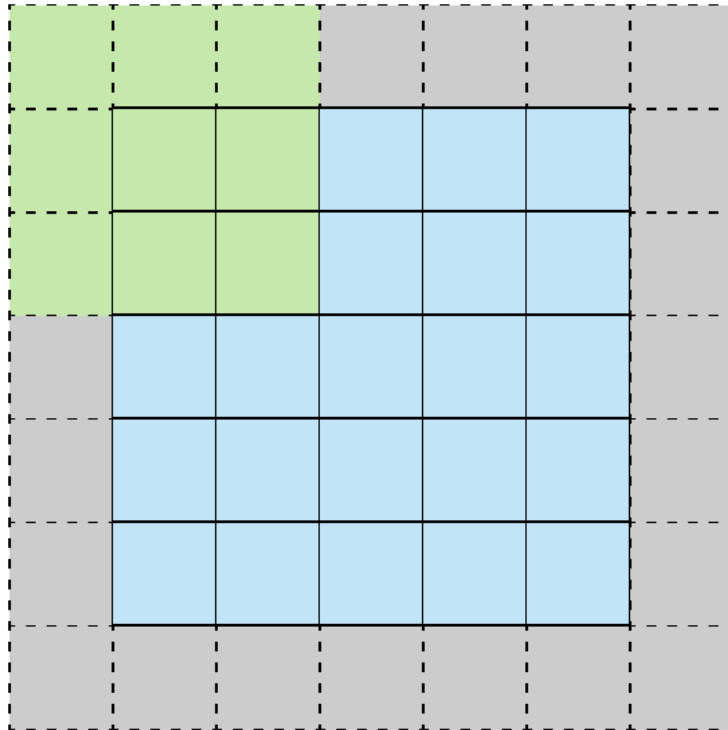
- ✓ 문제점: 가장자리에 있는 픽셀들은 중앙에 위치한 픽셀들에 비해 Convolution 연산이 적게 수행됨
- ✓ 해결책: Padding (원래 이미지 테두리에 0의 값을 갖는 pad를 추가)



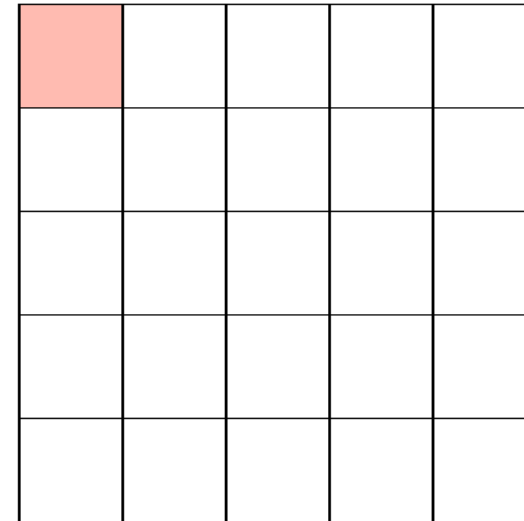
CNN Basics: Convolution

- Image Convolution (Filter, Kernel)

✓ 해결책: Padding (원래 이미지 테두리에 0의 값을 갖는 pad를 추가)



Stride 1 with Padding



Feature Map

CNN Basics: Convolution

- Image Convolution (Filter, Kernel)

✓ Convolution with padding

0	0	0	0	0	0
0	105	102	100	97	96
0	103	99	103	101	102
0	101	98	104	102	100
0	99	101	106	104	99
0	104	104	104	100	98

Image Matrix

Kernel Matrix

0	-1	0
-1	5	-1
0	-1	0

320				

Output Matrix

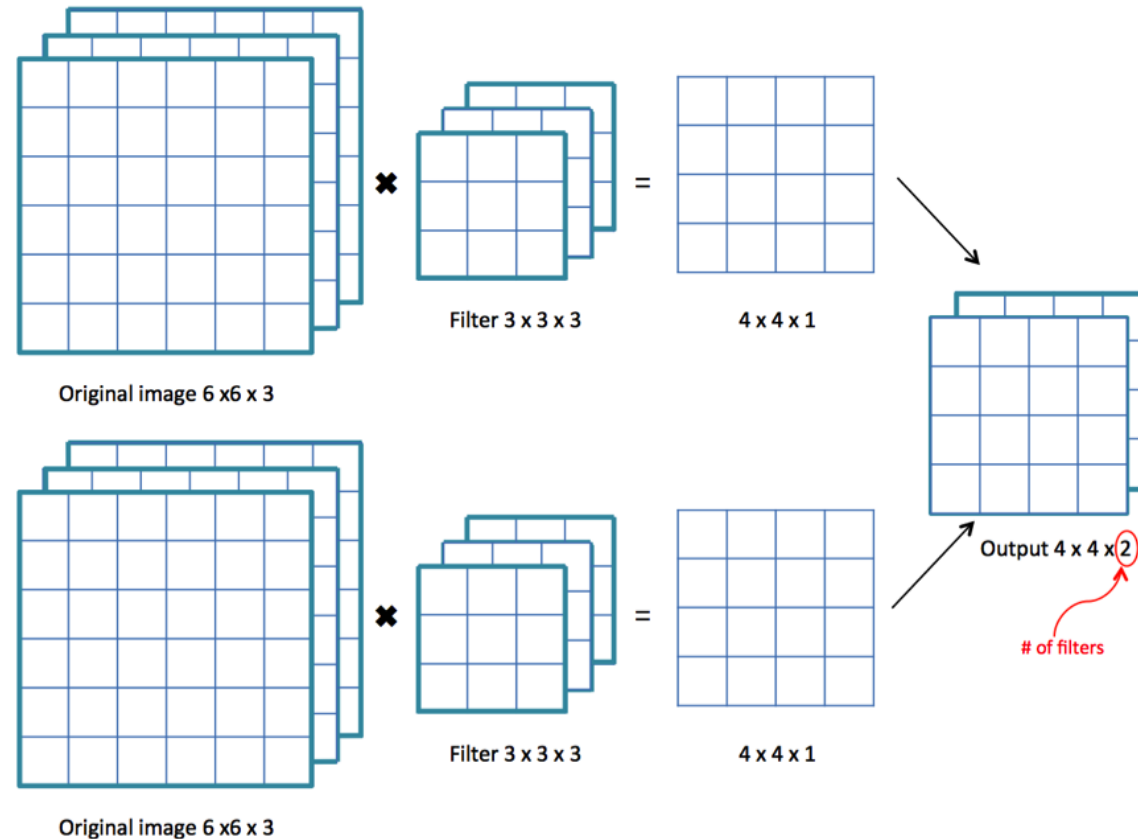
$$\begin{aligned} &0 * 0 + 0 * -1 + 0 * 0 \\ &+ 0 * -1 + 105 * 5 + 102 * -1 \\ &+ 0 * 0 + 103 * -1 + 99 * 0 = 320 \end{aligned}$$

Convolution with horizontal and
vertical strides = 1

CNN Basics: Convolution

- Padding과 Stride 크기에 따른 Output Size

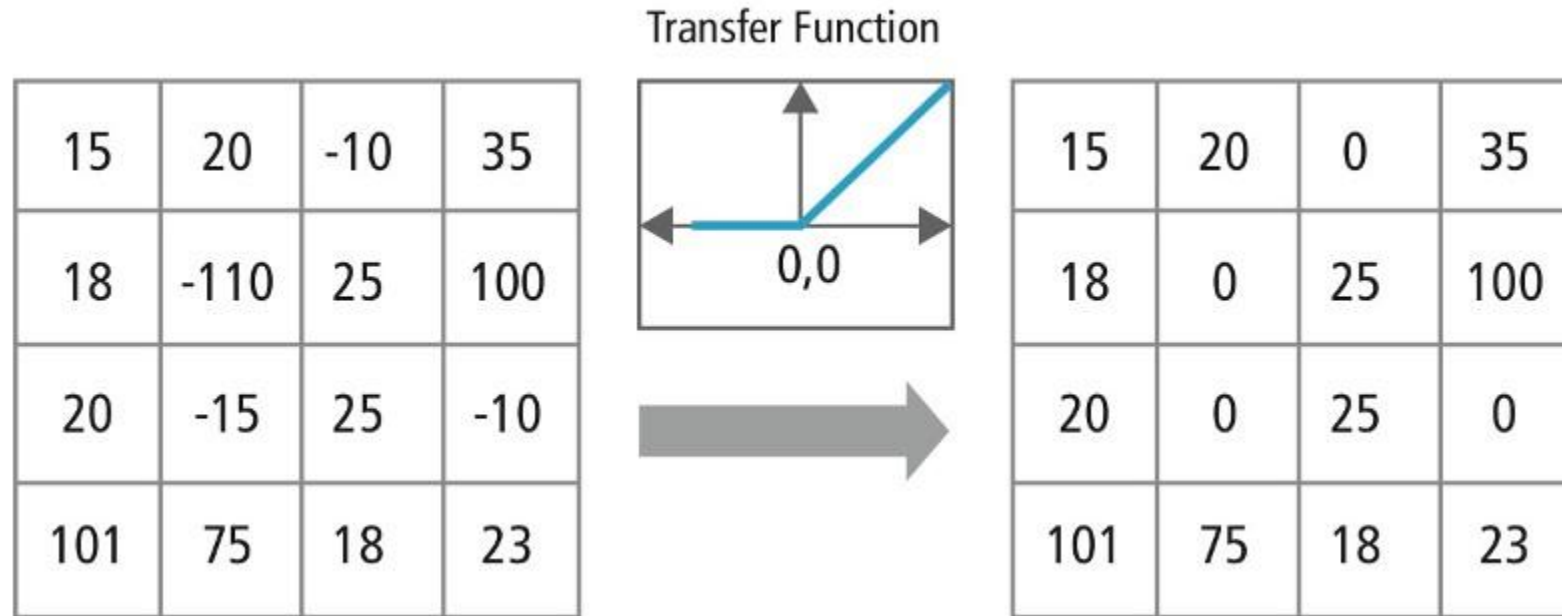
$$\text{Output size} = \left(\frac{H + 2P - F}{S} + 1 \right) \times \left(\frac{W + 2P - F}{S} + 1 \right)$$



CNN Basics: Activation

- Activation

- ✓ Convolution을 통해 학습된 값들의 비선형 변환을 수행
- ✓ 대부분 Rectified Linear Unit (ReLU)을 사용

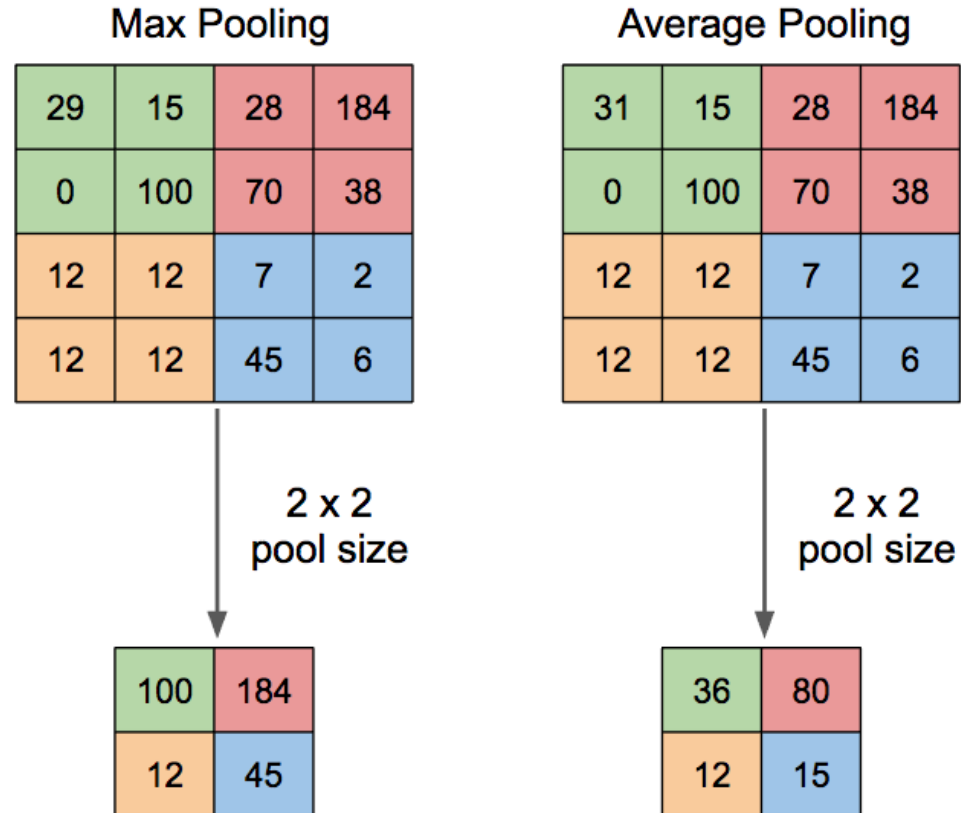


<https://medium.com/data-science-group-iitr/building-a-convolutional-neural-network-in-python-with-tensorflow-d251c3ca8117>

CNN Basics: Pooling

- Pooling

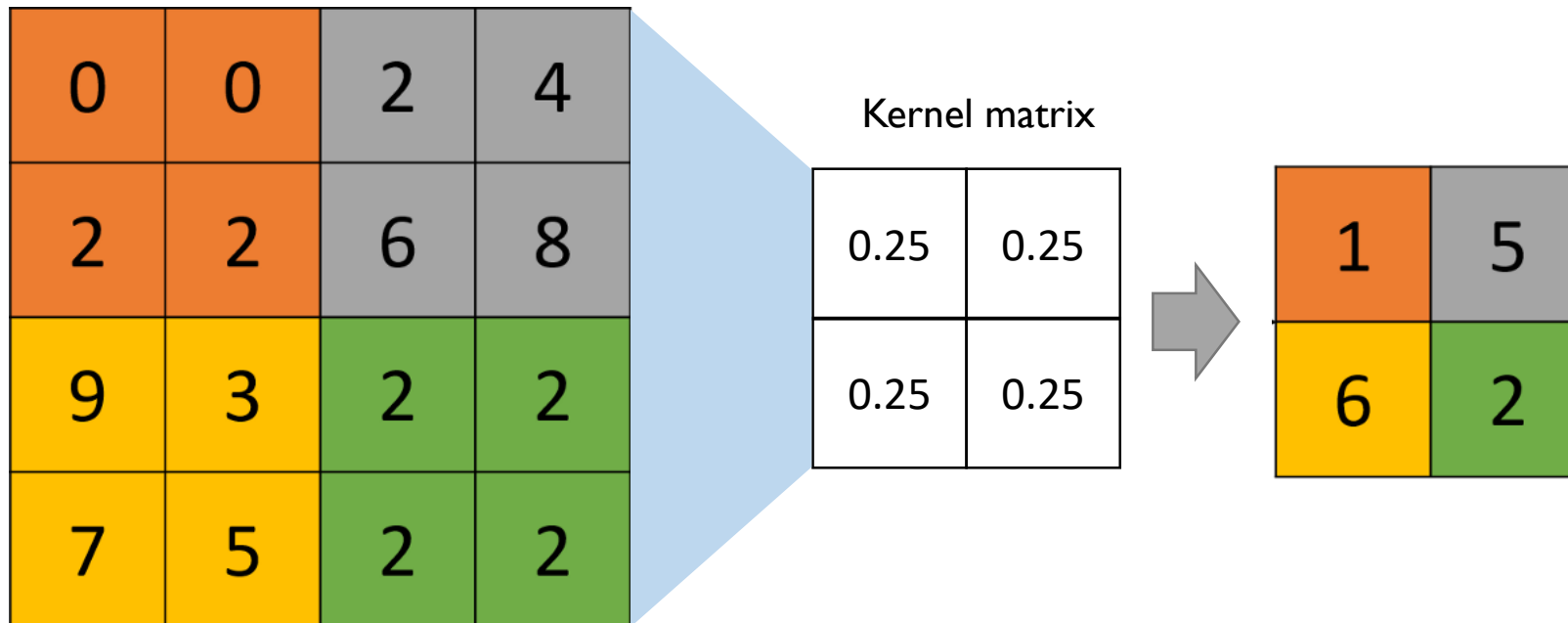
- ✓ 문제점: 고차원의 Tensor를 보다 Compact하게 축약해야 하지 않을까?
- ✓ Pooling: 일정 영역의 정보를 축약하는 역할



CNN Basics: Strided Convolution

- Strided Convolution

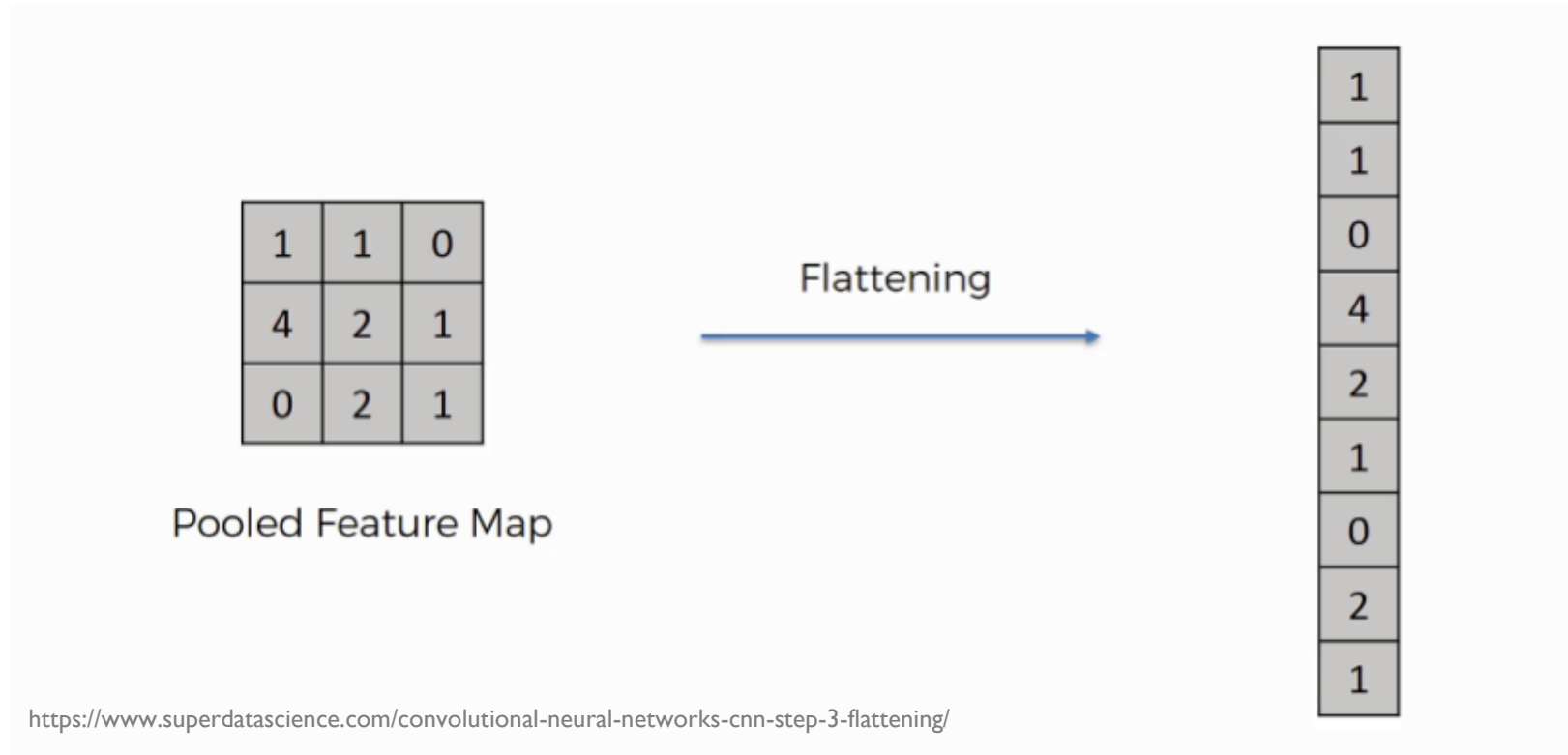
✓ Average Pooling은 strided convolution의 특수한 케이스: 모든 weight가 $1/n$



CNN Basics: Flatten

- 평탄화: Flatten

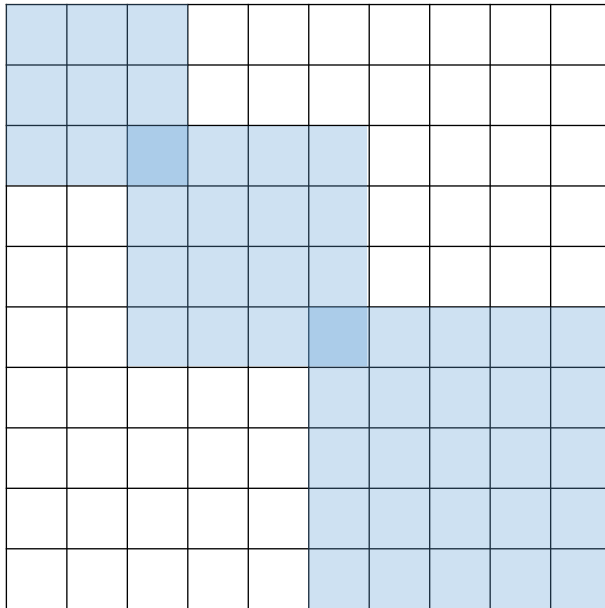
✓ 2차원/3차원의 Matrix/Tensor 구조를 1차원의 Vector로 변환하는 과정



CNN: 하이퍼파라미터

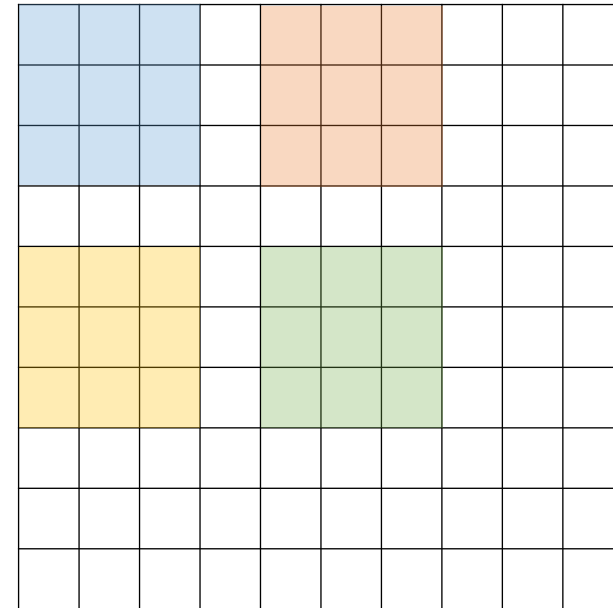
- CNN의 하이퍼 파라미터

[Convolution Filter의 크기]



- Convolution Filter의 가로/세로 크기
- 이미지 데이터에는 주로 2-d Conv를 사용 (가로와 세로의 크기가 같은 Conv 2*2, 3*3 등)

[Convolution Filter의 수]

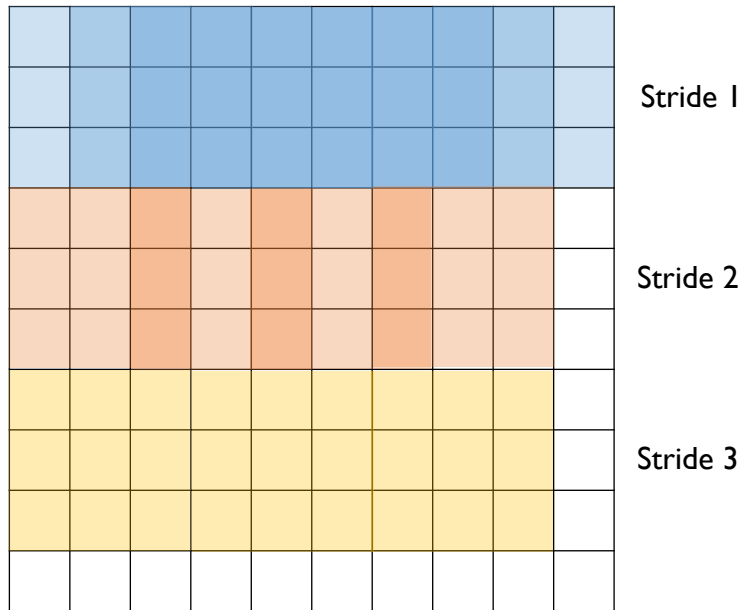


- Convolution Filter의 개수
- 이 수가 많을 수록 동일 영역에서 다양한 형태의 특질을 추출할 수 있음

CNN: 하이퍼파라미터

- CNN의 하이퍼 파라미터

[Stride의 크기]



- Conv Filter가 건너뛰는 픽셀의 수
- Stride가 작을수록 촘촘하게 특질을 추출하나, 연산 시간은 오래 걸림

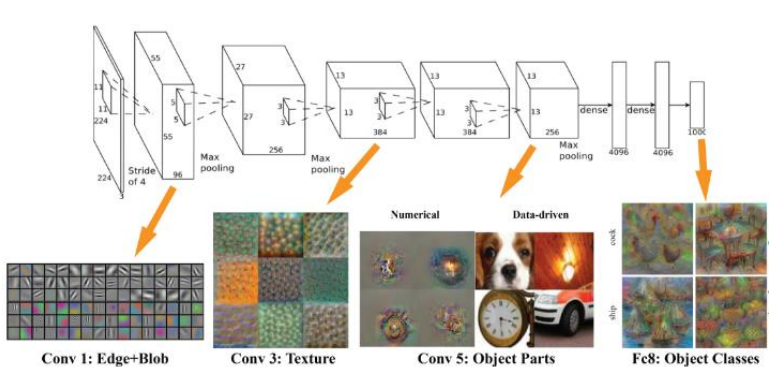
[Zero Padding 크기]

0	0	0	0	0	0	0	0	0	0	0	0
0											0
0											0
0											0
0											0
0											0
0											0
0											0
0											0
0											0
0											0
0	0	0	0	0	0	0	0	0	0	0	0

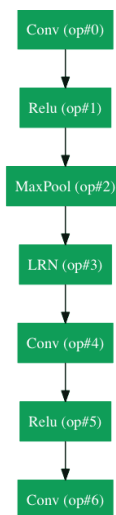
- Conv 연산의 편의성을 위해 사용
- Conv 연산 전-후의 크기를 일치시키기 위한 목적
- Filter와 Stride의 크기에 따라 필요한 Padding 크기가 결정됨

Convolutional Neural Network (CNN)

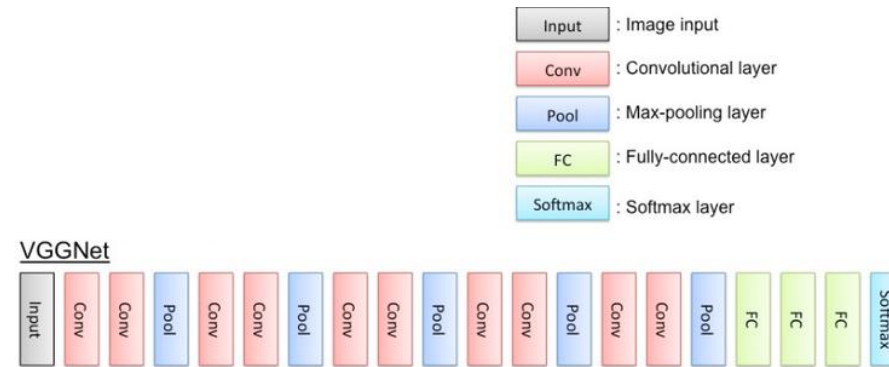
• 대표적인 CNN 구조



Alexnet

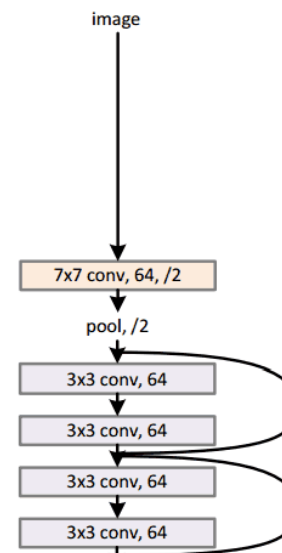


GoogLeNet



VGGNet

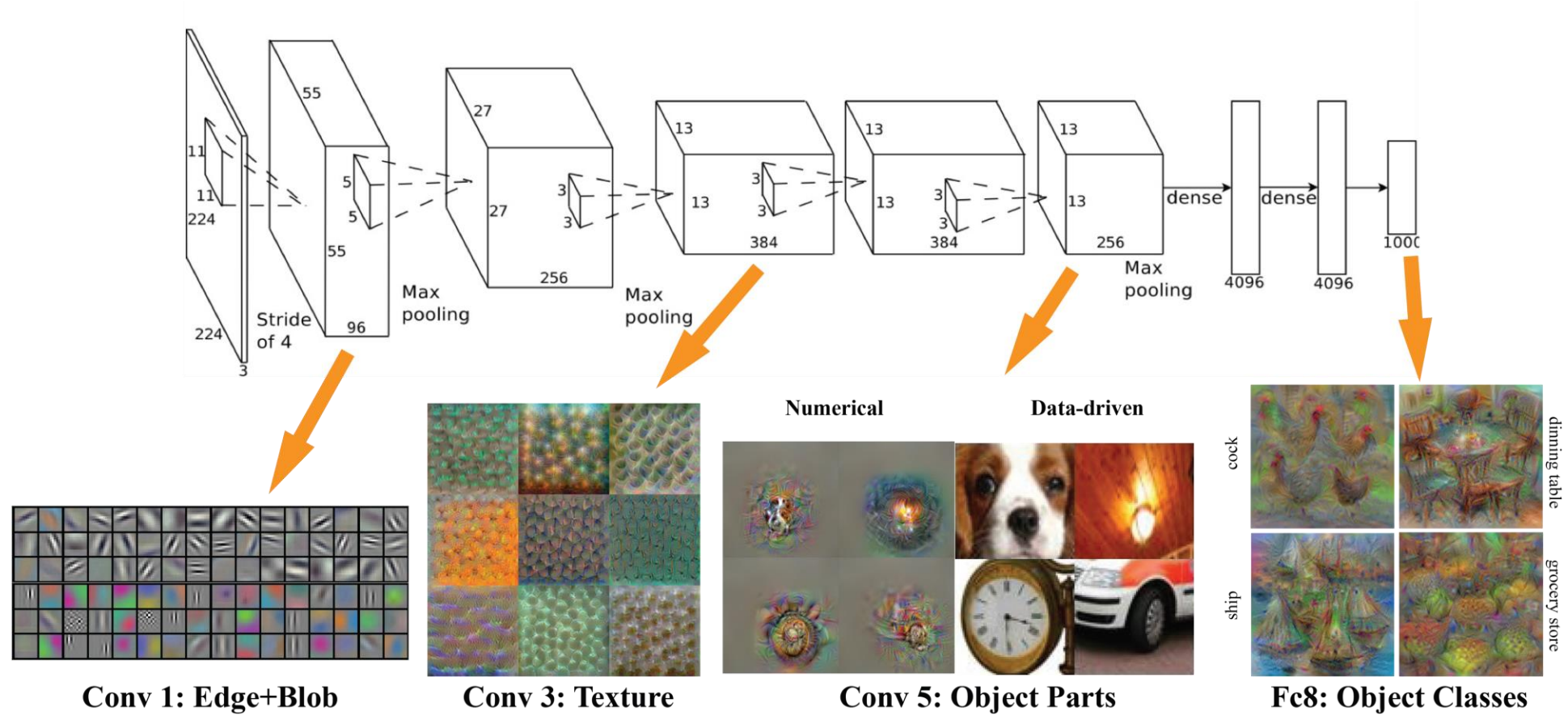
34-layer residual



ResNet

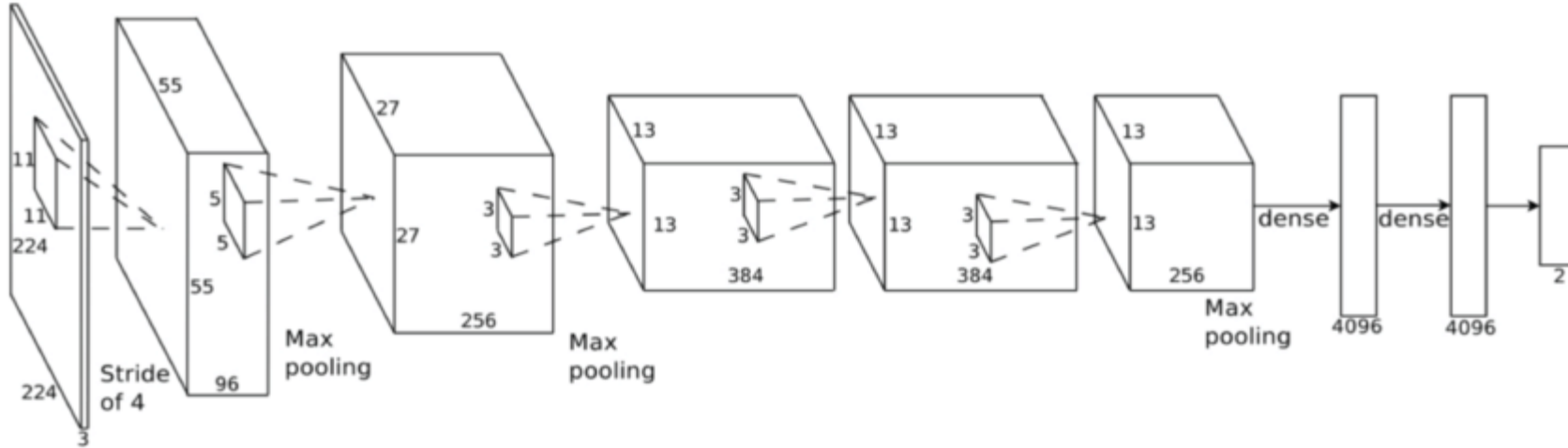
CNN Architecture I: AlexNet

- AlexNet



CNN Architecture I: AlexNet

- AlexNet

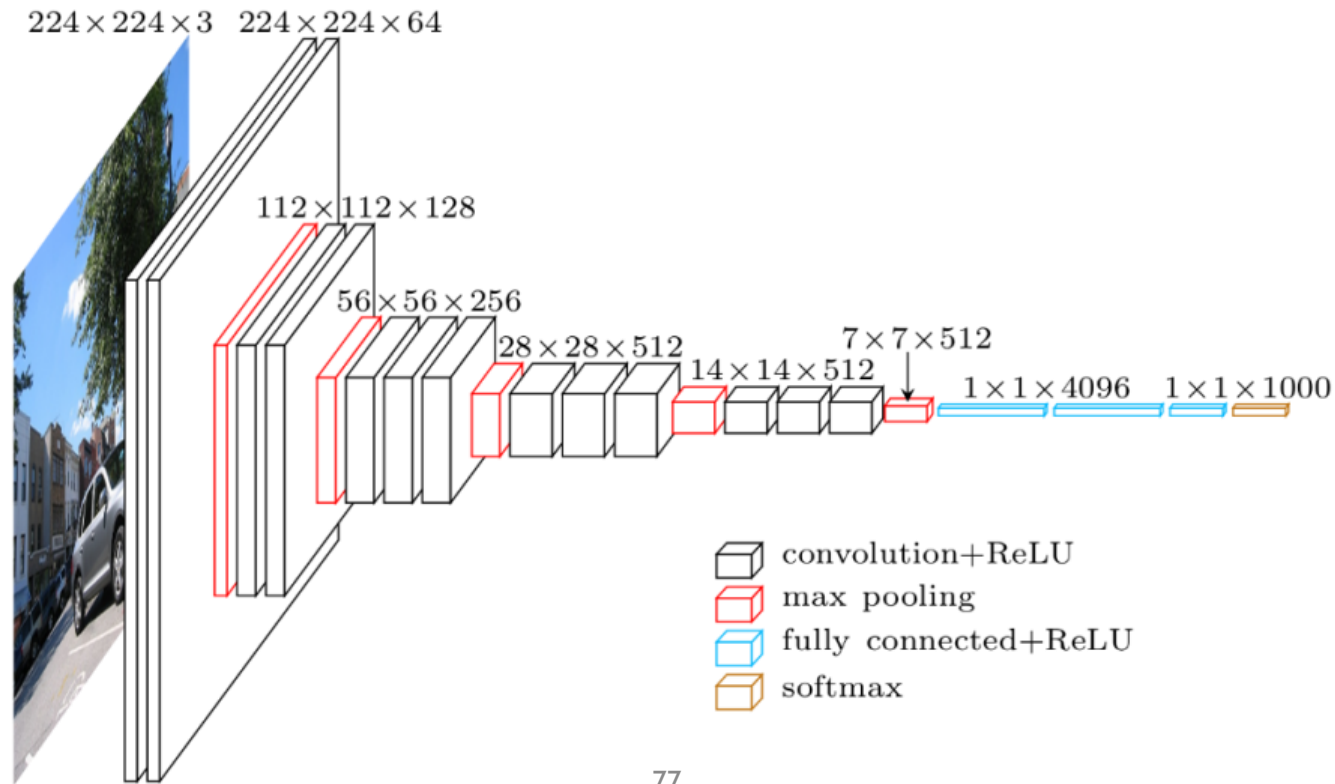


- ✓ 224*224 크기의 이미지를 Input으로 사용
- ✓ 초기 단계에서는 큰 필터 사이즈와 Stride를 사용
- ✓ 상위 Layer로 갈수록 작은 필터 사이즈와 Stride를 사용
- ✓ 2개의 Fully connected layer 존재
- ✓ 파라미터 총 수: 60 million

CNN Architecture 2:VGGNet

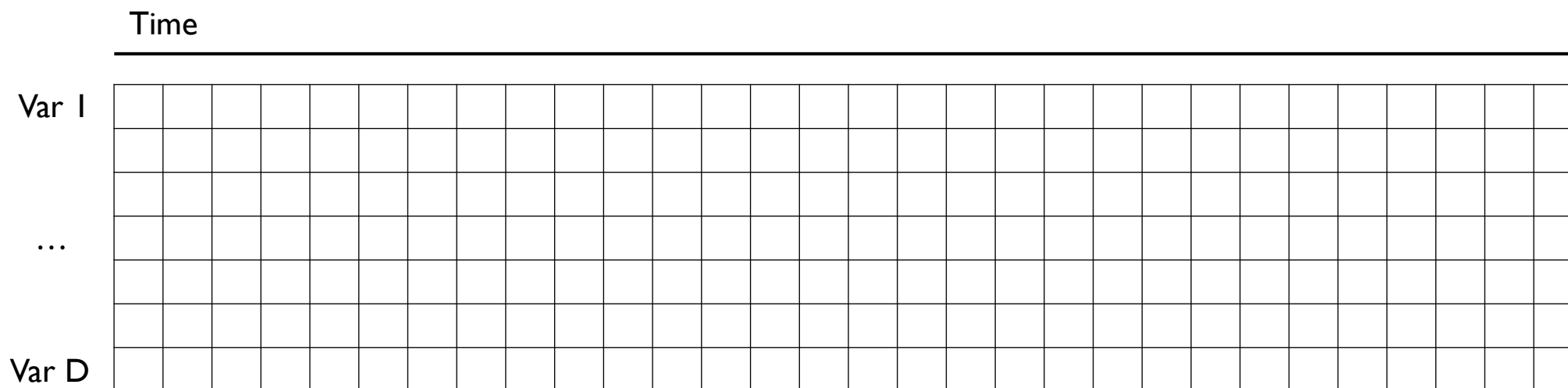
- VGGNet

- ✓ AlexNet에 비해 단순하지만 깊은 구조
- ✓ 3 by 3 convolution with stride 1을 기본 연산으로 하며 중간 중간에 2 by 2 max pooling을 수행
- ✓ 파라미터 총 수: 138 million



CNN for Time-Series Data

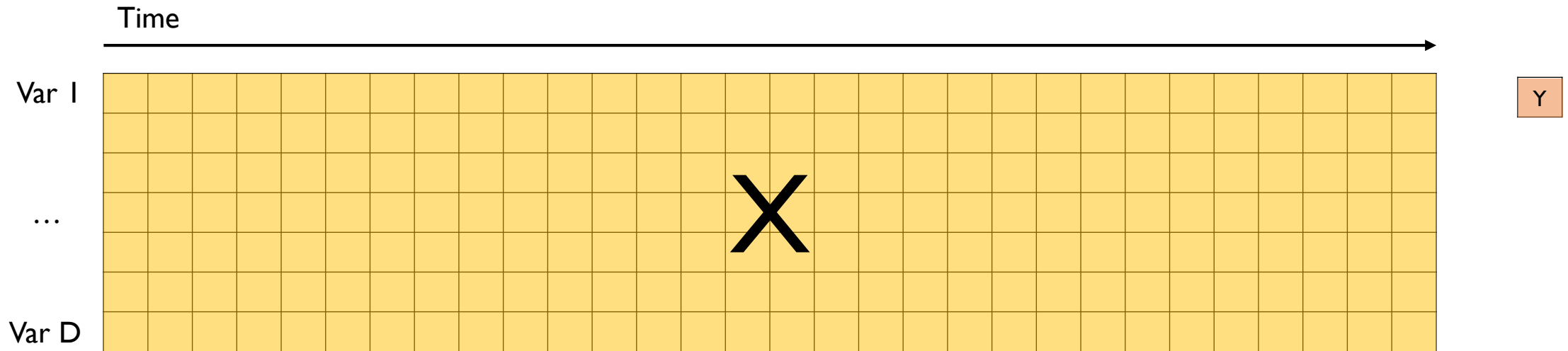
- Time-Series Data 가정
 - ✓ 모든 변수는 동일한 주기(초, 분, 시간 등)로 수집되고 있음



CNN for Time-Series Data

- Time-Series Data

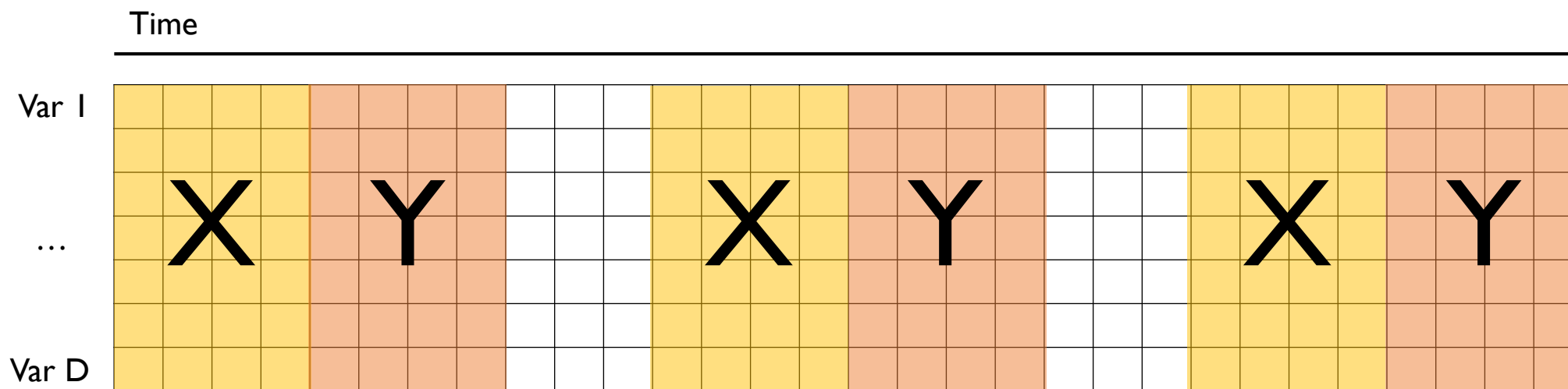
- ✓ Task 1-Classification: 특정 기간(제품 가공 기간) 데이터를 입력으로 하고 특정 범주를 출력으로 하는 분류 모델 (제품의 양/불 판정 등)
- ✓ Task 2-Regression: 특정 기간 데이터를 입력으로 하고 특정 수치(품질 지표)를 출력으로 하는 회귀 모델



CNN for Time-Series Data

- Time-Series Data

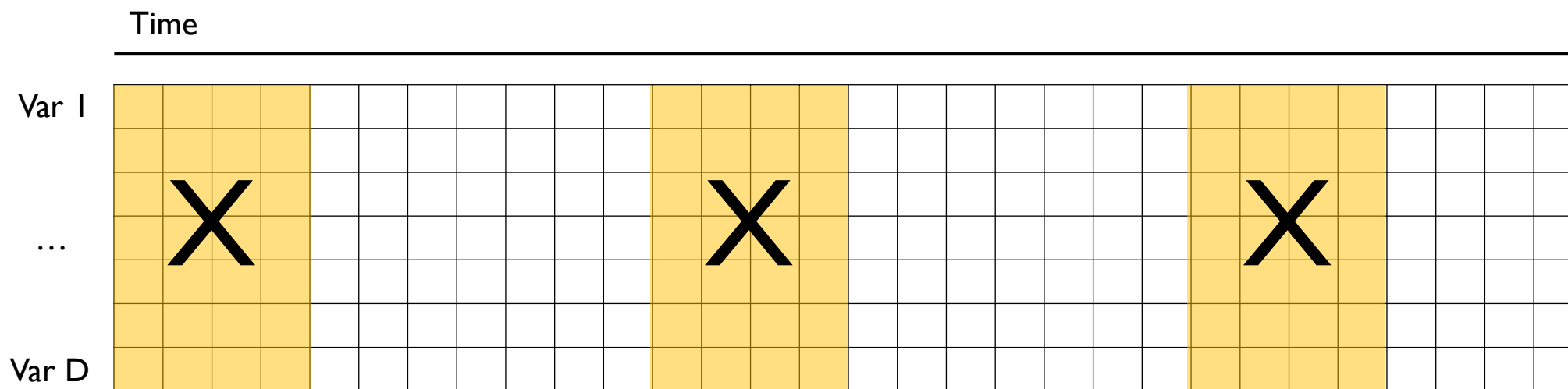
✓ Task 3-Regression: 일부 기간 데이터를 입력으로 하여 이후 기간 데이터를 예측



CNN for Time-Series Data

- Time-Series Data

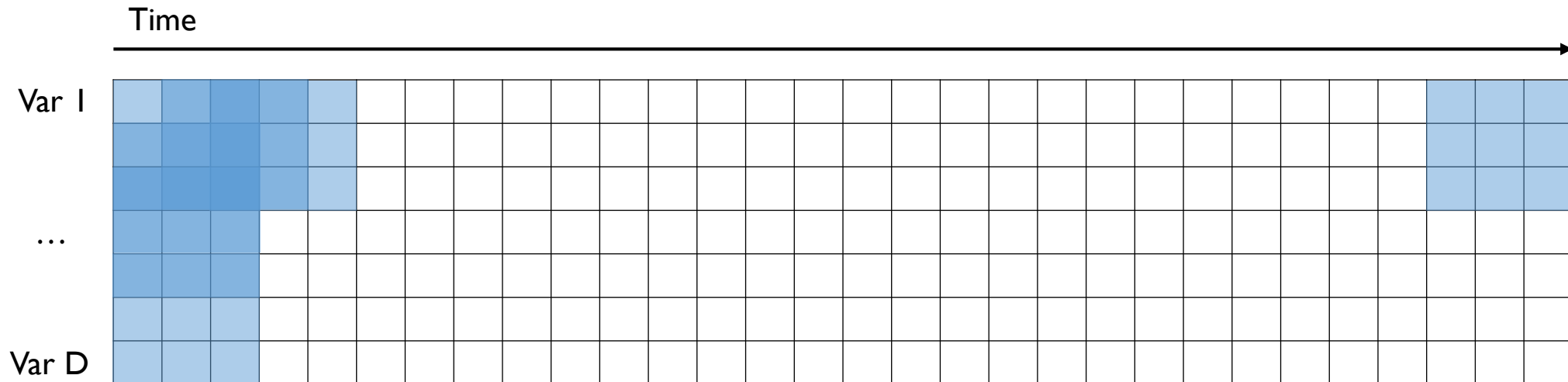
✓ Task 4-Anomaly Detection: 일부 기간 데이터를 입력으로 하여 해당 상황의 정상/비정상 여부를 탐지



CNN for Time-Series Data

- 1-D Convolution vs. 2-D Convolution

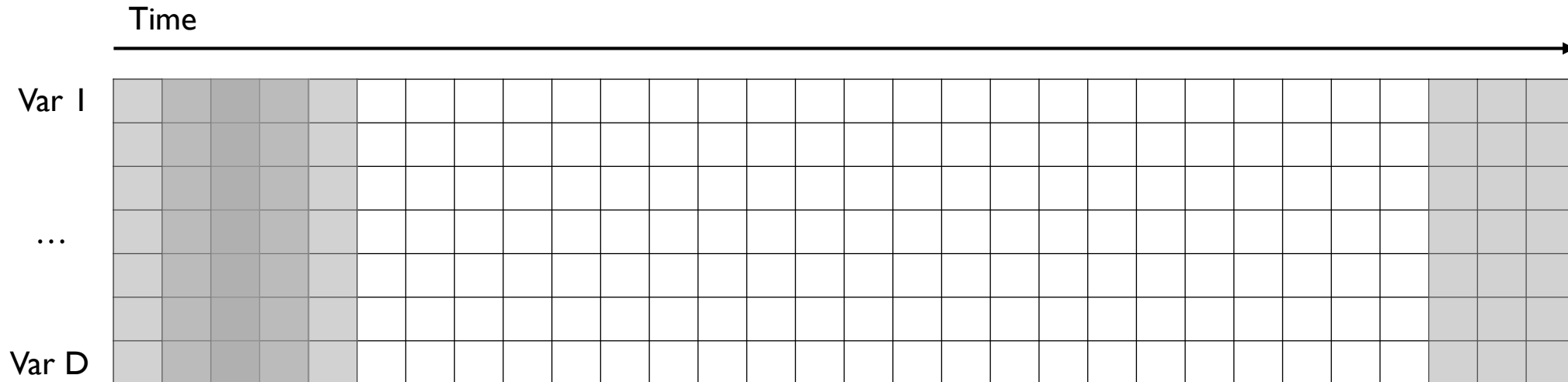
- ✓ 시계열 데이터는 이미지 데이터와는 달리 변수들 사이의 Spatial Correlation이 존재하지 않음
- ✓ 시간축으로 움직이는 Convolution은 의미가 있으나, 변수축으로 움직이는 Convolution은 의미가 없음
- ✓ 2-D Convolution은 시간과 변수 두 축을 모두 Convolution 연산을 통해 탐색하는 방식



CNN for Time-Series Data

- 1-D Convolution vs. 2-D Convolution

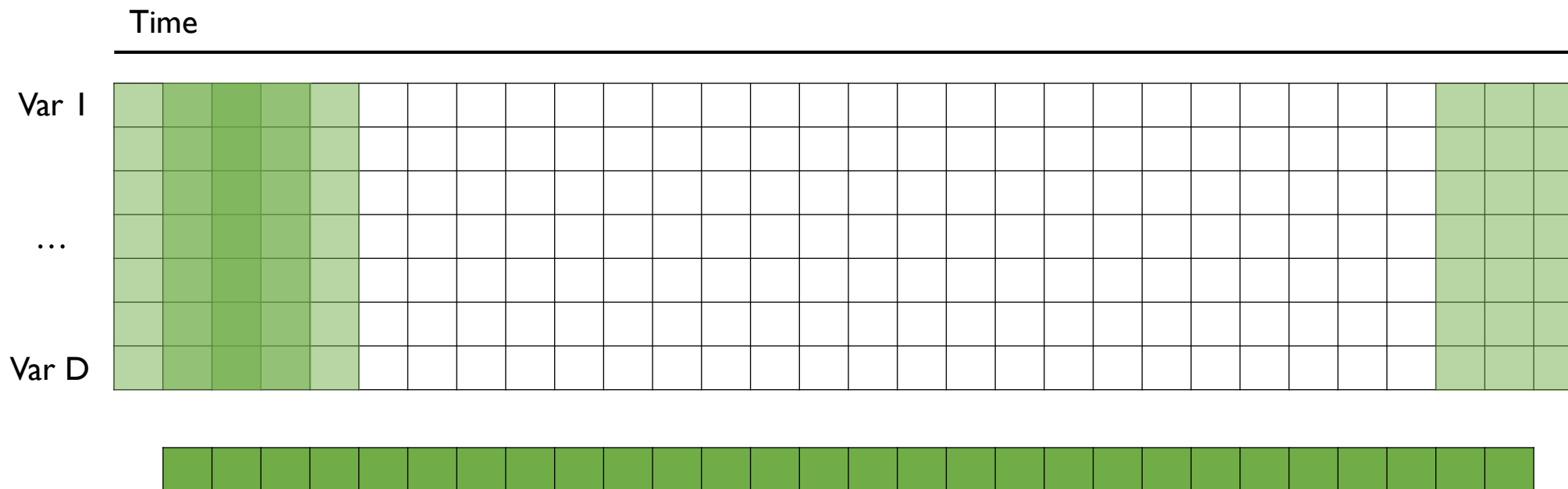
- ✓ 시계열 데이터는 이미지 데이터와는 달리 변수들 사이의 Spatial Correlation이 존재하지 않음
- ✓ 시간축으로 움직이는 Convolution은 의미가 있으나, 변수축으로 움직이는 Convolution은 의미가 없음
- ✓ 1-D Convolution은 모든 변수를 한번에 고려하여 시간 축에 대해서만 Convolution 연산을 수행 (필터 크기가 정사각형이 아님)



CNN for Time-Series Data

- 1-D Convolution

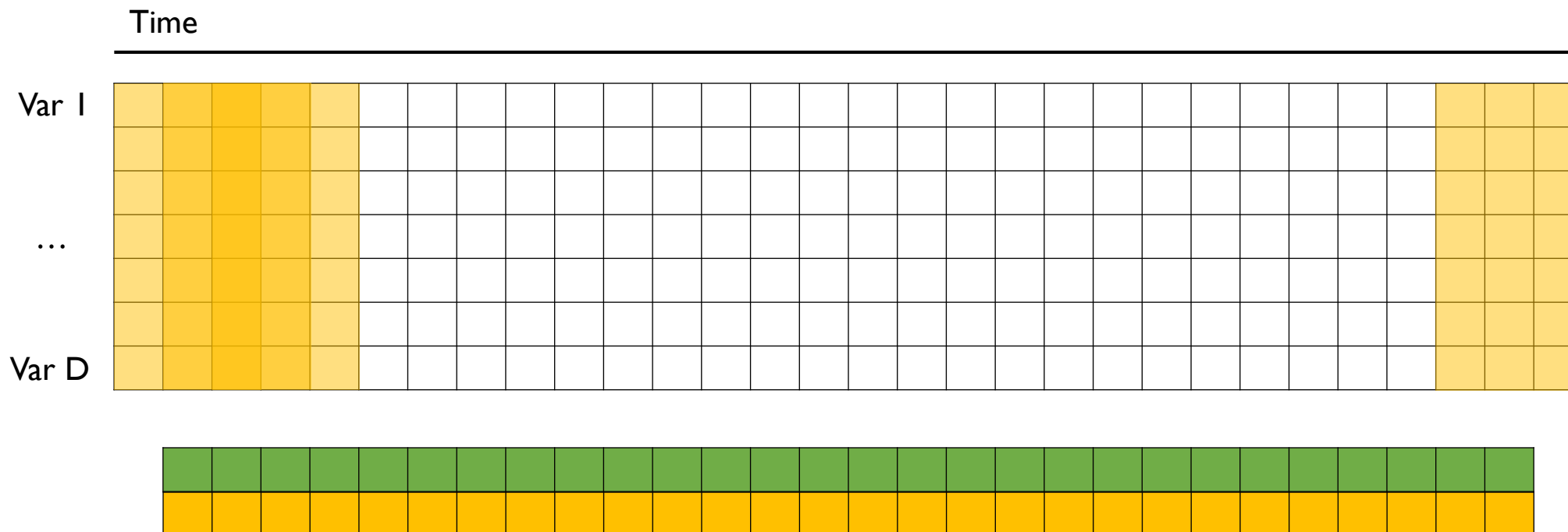
✓ 1개의 Filter를 사용하면 1개의 vector가 결과물로 산출됨



CNN for Time-Series Data

- 1-D Convolution

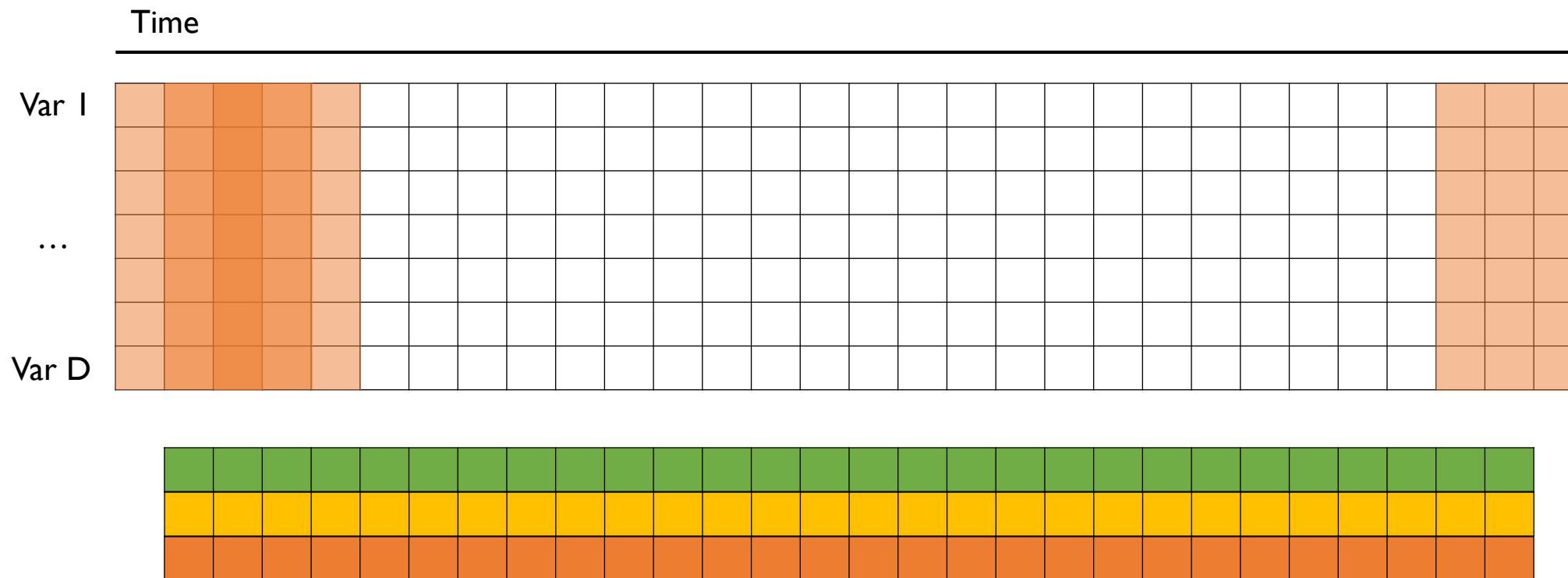
✓ 1개의 Filter를 사용하면 1개의 vector가 결과물로 산출됨



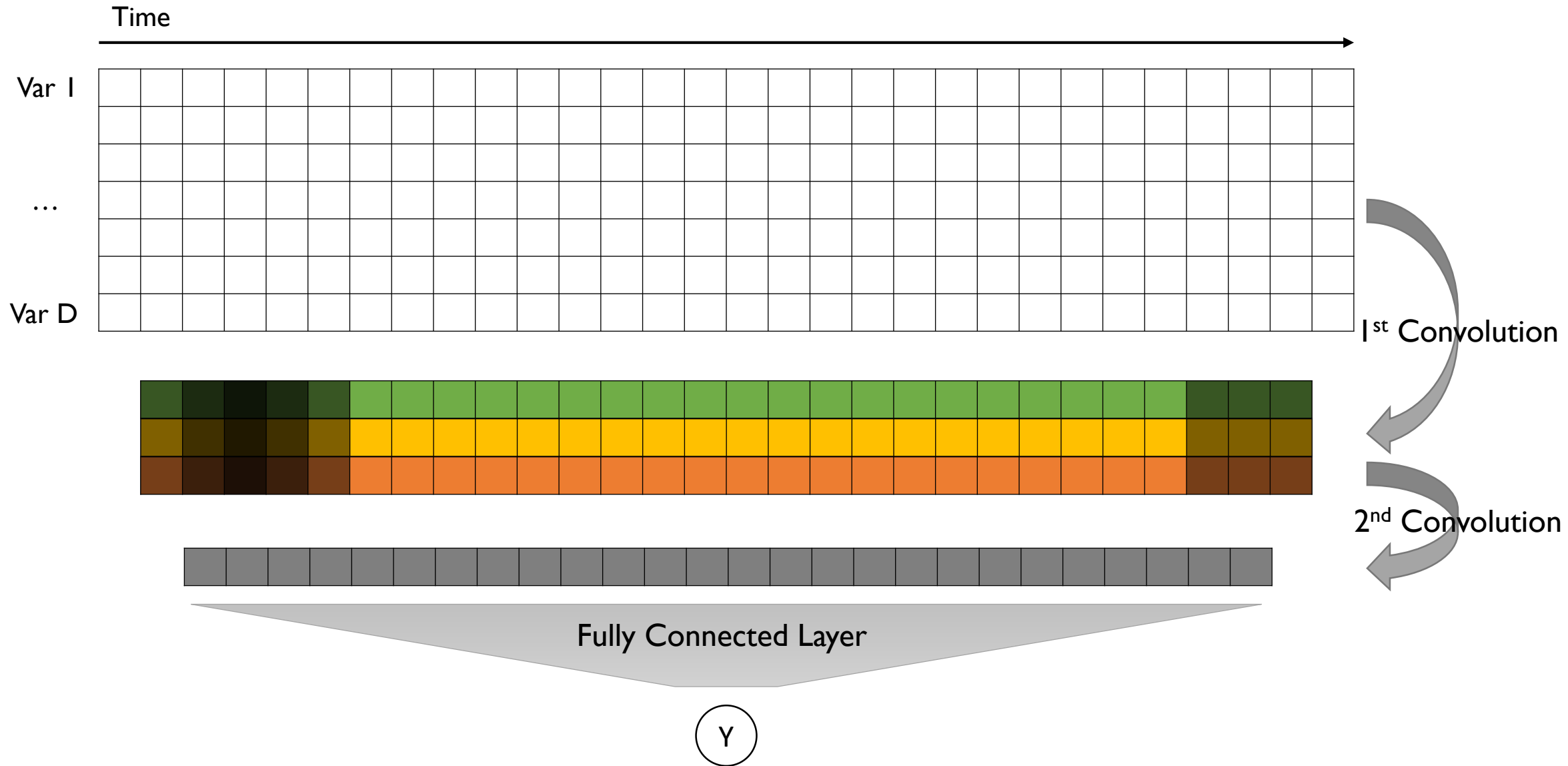
CNN for Time-Series Data

- 1-D Convolution

✓ 1개의 Filter를 사용하면 1개의 vector가 결과물로 산출됨

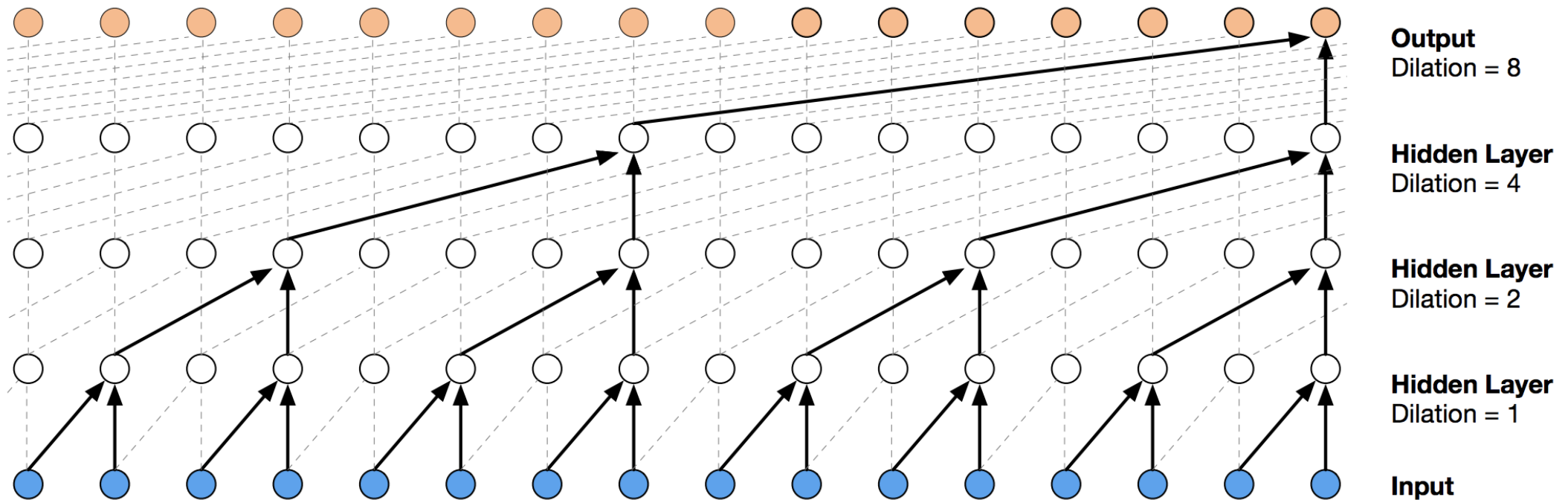


CNN for Time-Series Data



Dilated Convolution

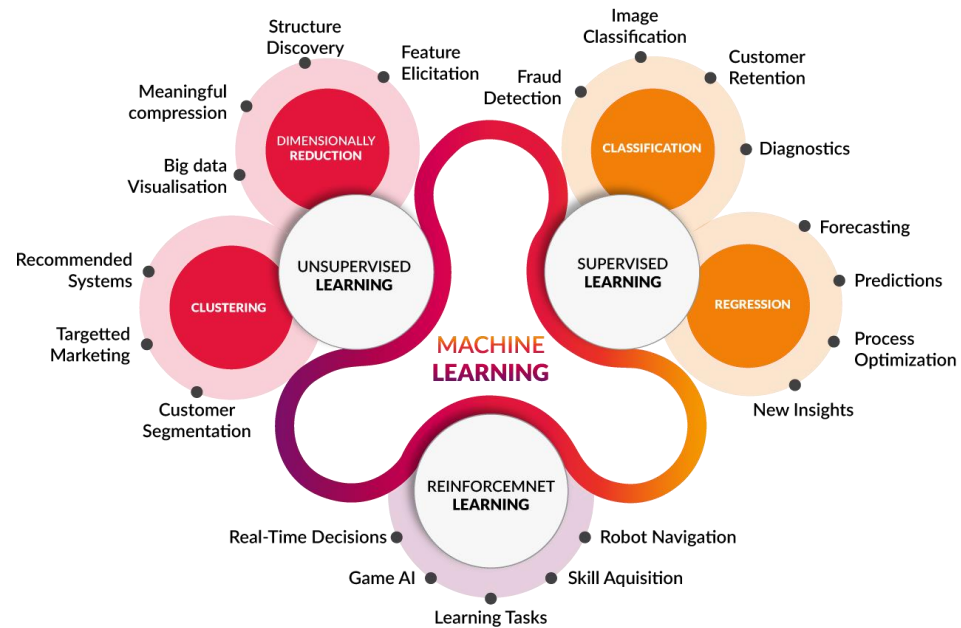
- Q) 보다 긴 길이의 시계열 데이터를 효율적으로 처리할 수는 없을까?
 - ✓ Standard Convolution은 항상 인접한 연속된 시점의 데이터에 대한 합성곱 연산을 수행
 - ✓ 합성곱 연산을 조금 더 띄엄띄엄 해보면 어떨까?



Summary

- 합성곱 신경망을 활용한 시계열 회귀
 - ✓ 합성곱 연산은 Feature Invariance와 Spatial Correlation을 반영하기 위해 사용되는 연산이다.
 - ✓ 이미지를 처리할 때는 정방형의 필터를 사용하여 가로-세로 두 방향으로 움직이는 2d 합성곱 연산을 사용한다.
 - ✓ 반면, 시계열 데이터를 처리하는 경우에는 변수축으로 필터가 이동하는 것은 의미가 없으므로 필터의 한쪽 크기는 변수의 개수와 같게 정의한 뒤 시간 축으로만 움직이는 1d 합성곱 연산을 사용한다.
 - ✓ 합성곱 연산은 한 레이어에서만 적용하는 것이 아니라 반복적으로 쌓아올려서 사용할 수 있다.
 - ✓ 보다 긴 길이의 시계열 데이터를 한번에 처리하기 위해서는 Standard Convolution보다는 중간 지점을 생략하는 Dilated Convolution을 사용하면 보다 효율적인 정보의 처리가 가능하다.





트랜스포머 기반의 시계열 회귀

강필성

고려대학교 산업경영공학부

pilsung_kang@korea.ac.kr

Transformer란?

- 언어 모델(Language Model)
 - ✓ 특정 문장(=단어의 나열)이 등장할 확률을 계산해주는 모델



$g(\text{준다}|\text{너에게, 나의 입술을, 처음으로}) = ?$

$g(\text{말린다}|\text{너에게, 나의 입술을, 처음으로})$

$= ?$

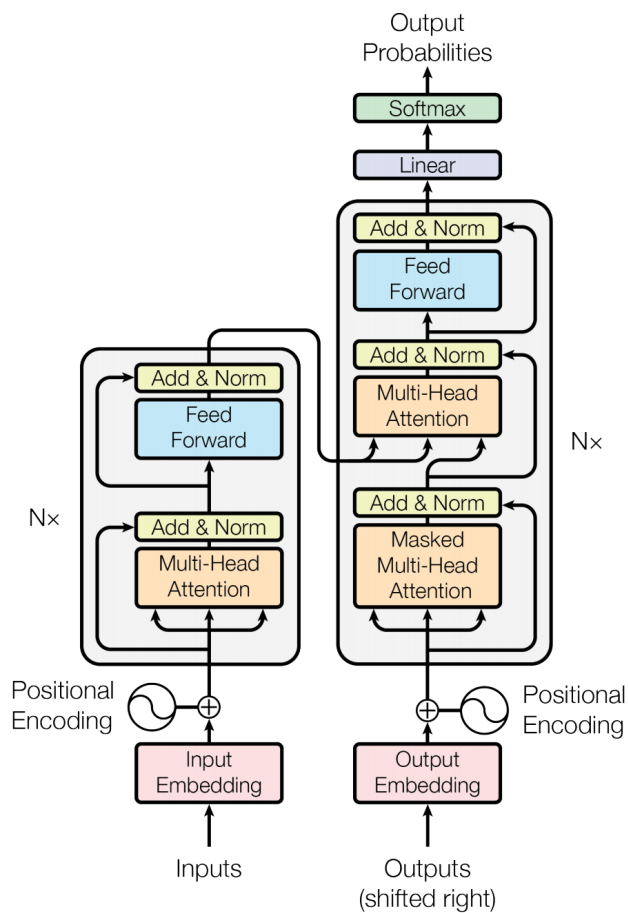
$g(\text{치운다}|\text{너에게, 나의 입술을, 처음으로})$

$= ?$

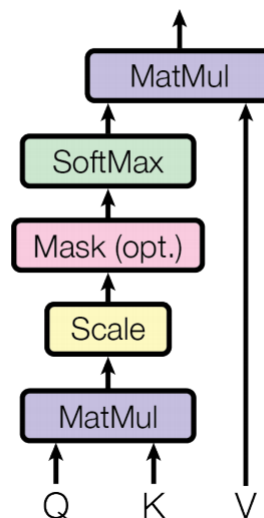
Transformer란?

- Transformer

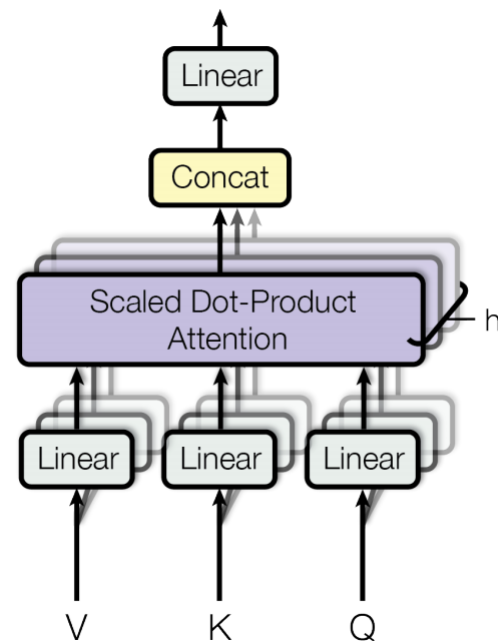
✓ Attention의 병렬적 사용을 통해 효율적인 학습이 가능한 구조의 언어 모델



Scaled Dot-Product Attention



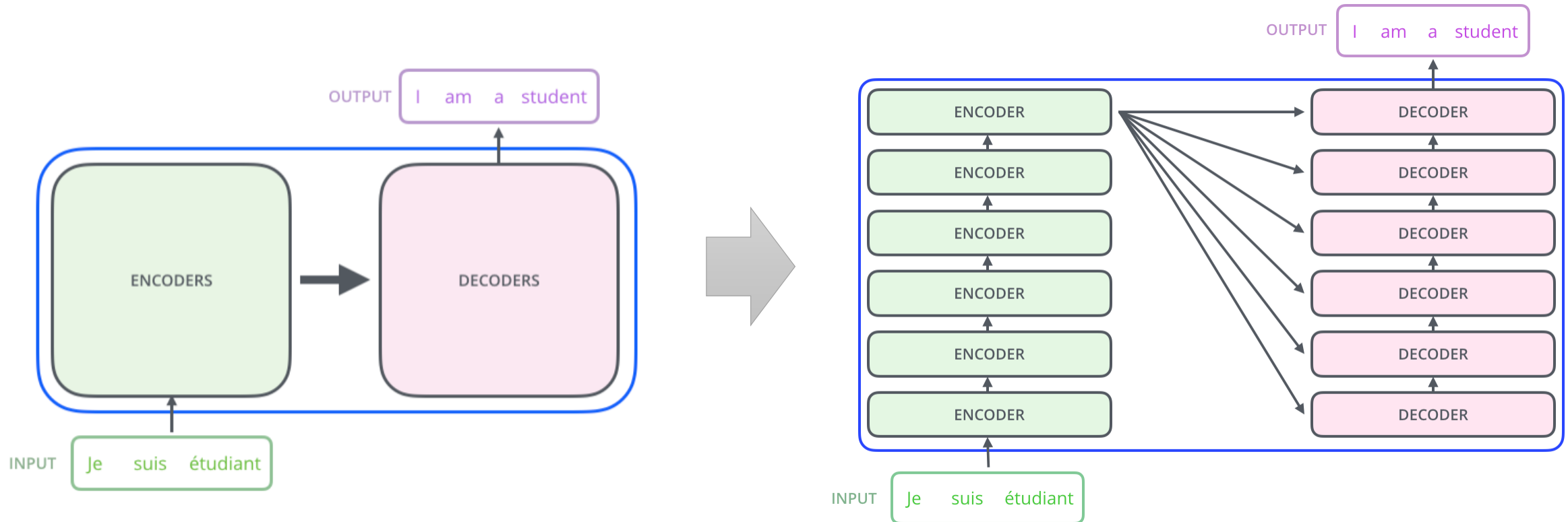
Multi-Head Attention



Transformer란?

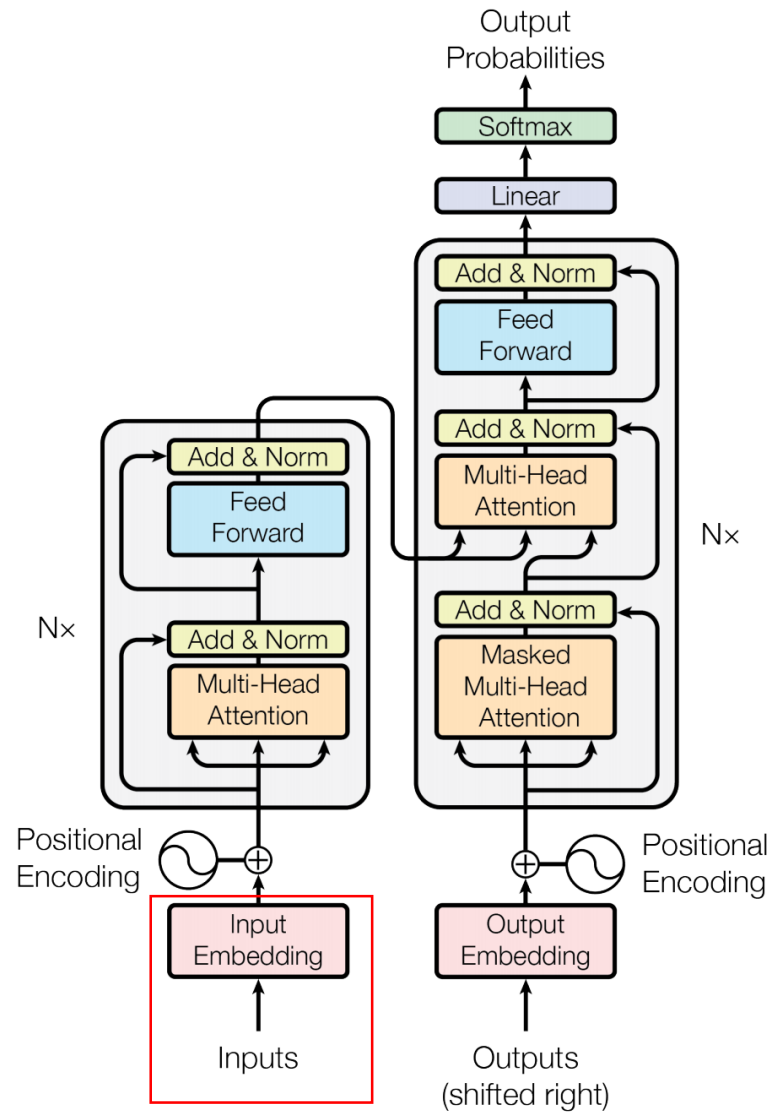
- Transformer의 개괄적 구조

✓ 내부에 인코더 파트와 디코더 파트가 존재하며 이 둘 사이를 이어주는 연결고리가 존재



Transformer의 작동 원리

- 입력 임베딩



Transformer의 작동 원리

✓ 개별 단어에 대한 임베딩 벡터를 최초 입력으로 사용

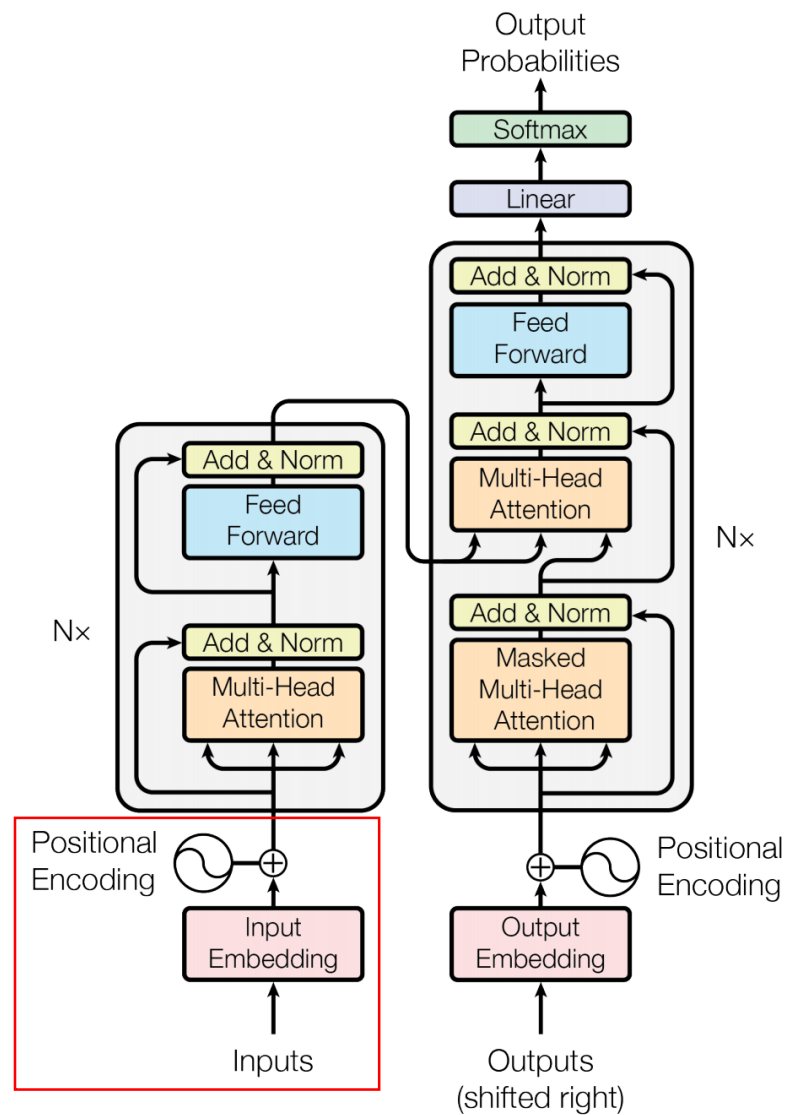


Each word is embedded into a vector of size 512. We'll represent those vectors with these simple boxes.

- 이 단어 임베딩은 가장 아래에 위치한 인코더에서만 입력으로 1회 사용됨
- 나머지 인코더들을 하위 인코더에서 출력된 결과물을 입력으로 사용
- 최초 논문에서는 512차원의 임베딩을 사용
- 입력으로 사용되는 리스트는 사용자가 지정하는 하이퍼파라미터 – 컴퓨팅 자원이 충분하다면 큰 값 지정 가능

Transformer의 작동 원리

- 포지션 인코딩 Positional Encoding

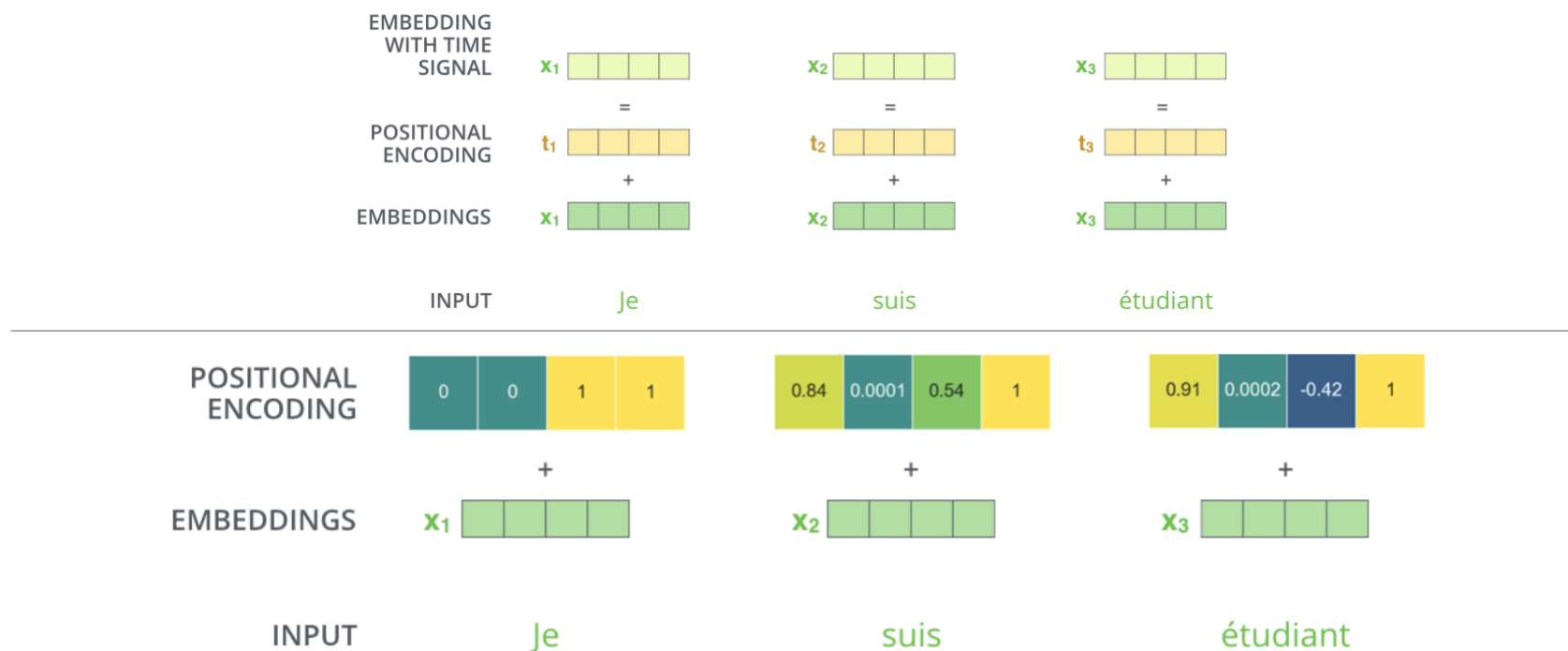


Transformer의 작동 원리

Alamar (Transformer)

- 포지션 인코딩 Positional Encoding

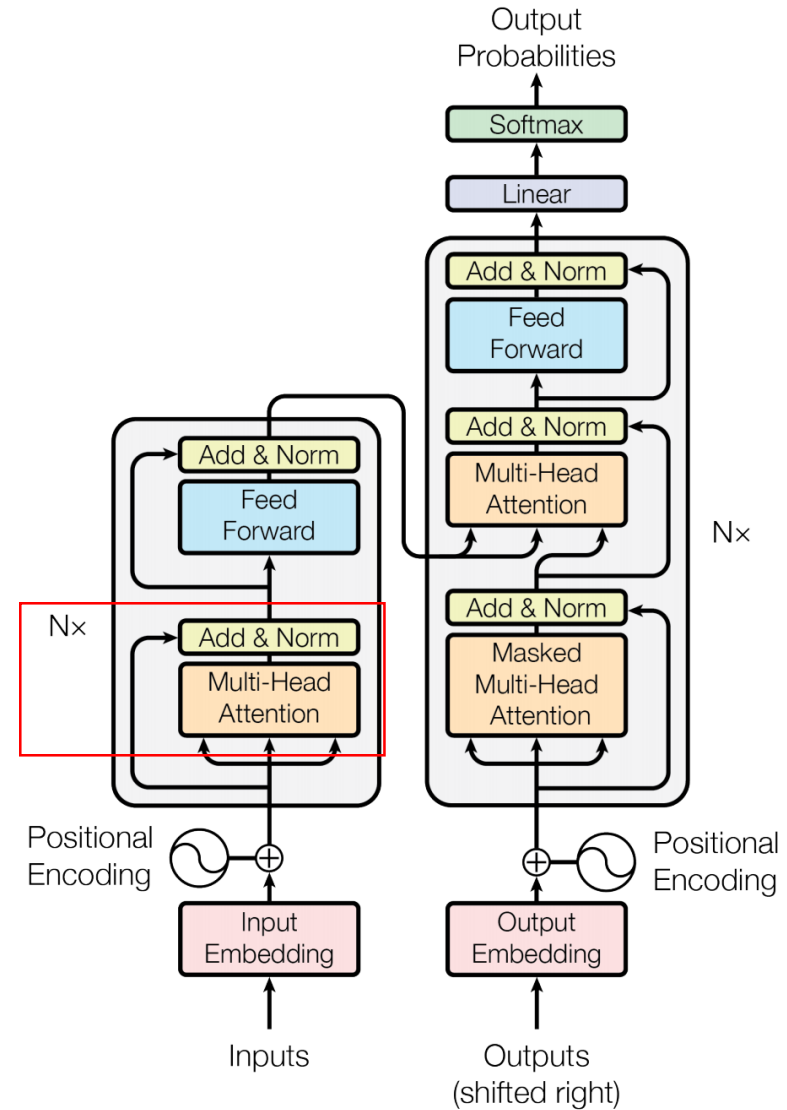
- ✓ Transformer는 단어를 순차적으로 받지 않고 한번에 받는 구조이므로 입력 시퀀스에서 단어들의 위치 관계를 표현해줄 필요가 있음
- ✓ 이러한 역할을 수행하도록 설계된 벡터를 포지션 인코딩이라고 하며, 모든 단어 임베딩에 포지션 인코딩을 더해서 입력 벡터를 구성



A real example of positional encoding with a toy embedding size of 4

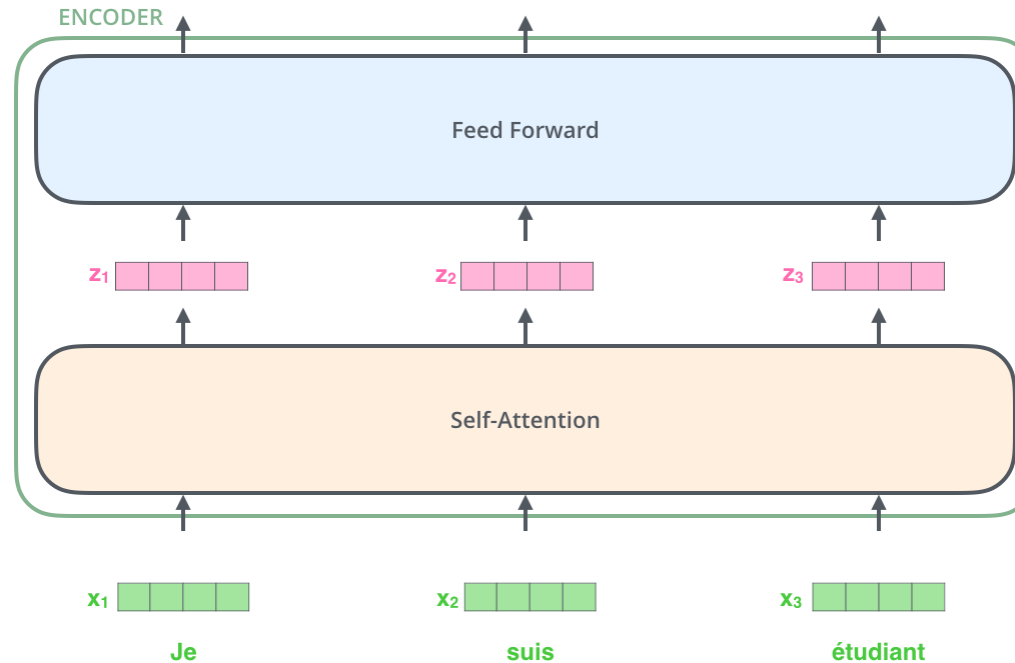
Transformer의 작동 원리

- 멀티 헤드 어텐션 Multi-Head Attention
- Residual connection & Normalization



Transformer의 작동 원리

- 포지션 인코딩이 더해진 단어 임베딩은 첫 번째 인코더 블록에서 셀프 어텐션과 FFNN을 거치게 됨



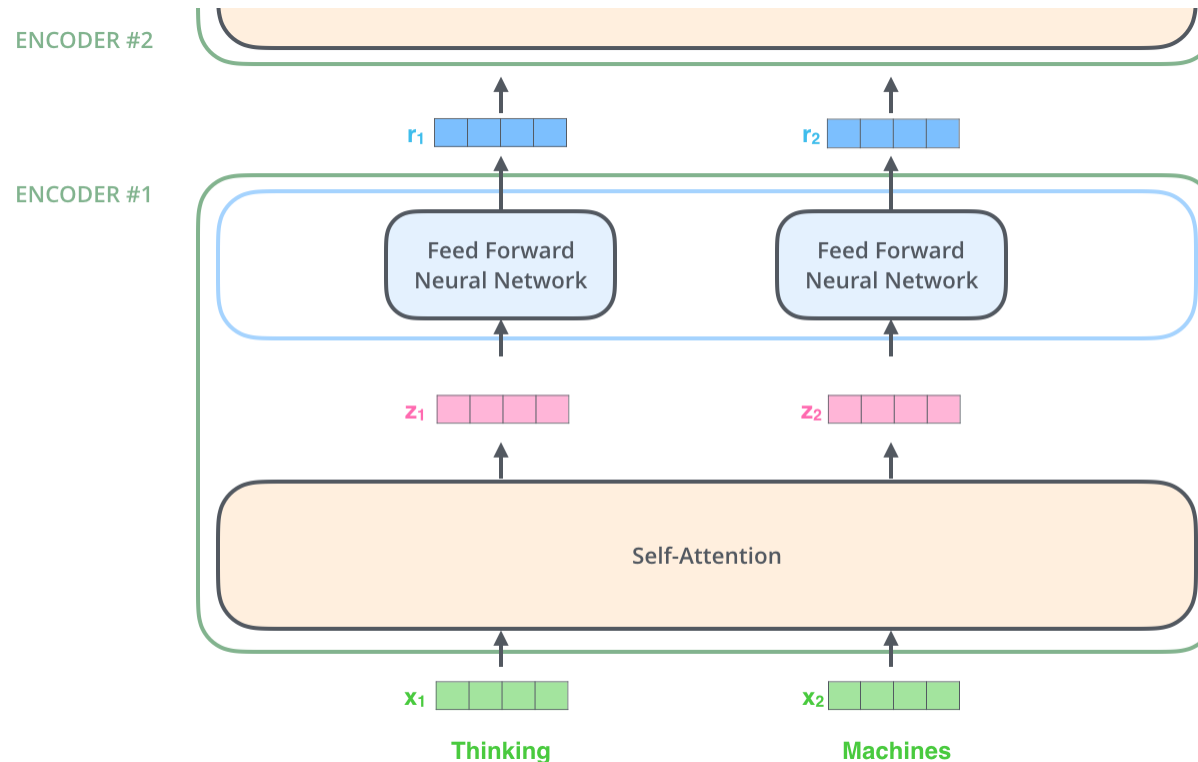
- ✓ 특정 위치의 단어는 해당 위치를 유지하면서 연산이 수행됨
 - 셀프 어텐션 과정에서는 이 경로간 의존성이 존재함
 - FFNN 과정에서는 의존성이 존재하지 않음 (병렬화parallelization 가능)

Transformer의 작동 원리

Alamar (Transformer)

- 인코딩 과정 Encoding procedure

- ✓ 인코더는 일련의 벡터들을 입력으로 받음
- ✓ 입력된 벡터들은 셀프 어텐션과 FFNN을 거쳐 상위 인코더의 입력으로 투입됨



Transformer의 작동 원리

Alamar (Transformer)

- 셀프 어텐션의 이해

✓ 다음 문장을 번역한다고 해보자:

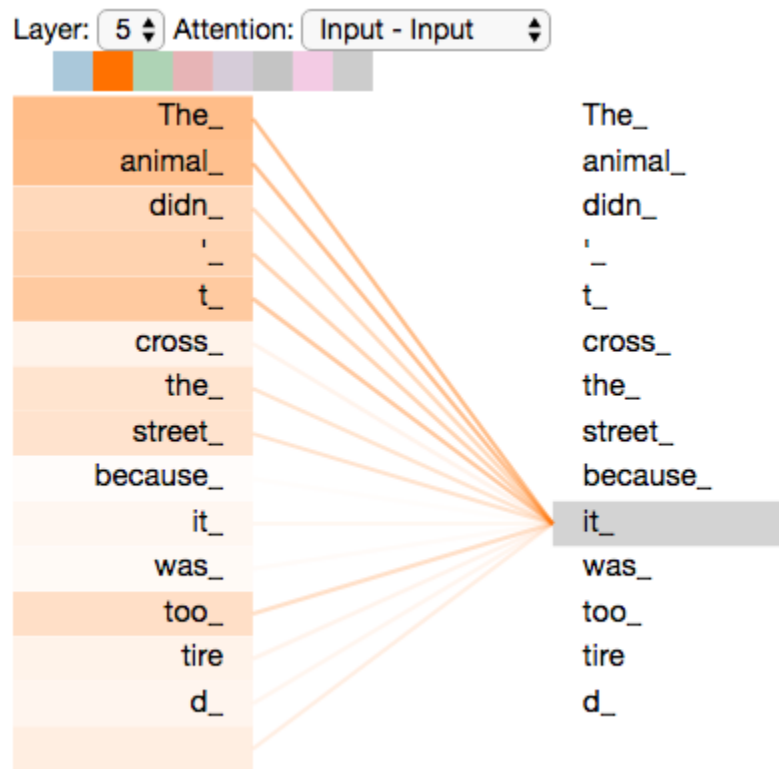
- The animal didn't cross the street because it was too tired

✓ 여기서 “it” 은 어떤 단어를 의미하는가?

Street or animal?

- 사람에게는 매우 쉬운 질문이지만 알고리즘에게는 그렇지 않음

✓ 셀프 어텐션은 입력 시퀀스의 다른 위치에 있는 단어들을 둘러보면서 특정 위치의 단어를 잘 설명/표현할 수 있게 함



https://colab.research.google.com/github/tensorflow/tensor2tensor/blob/master/tensor2tensor/notebooks/hello_t2t.ipynb#scrollTo=OJKU36QAfqOC

Transformer의 작동 원리

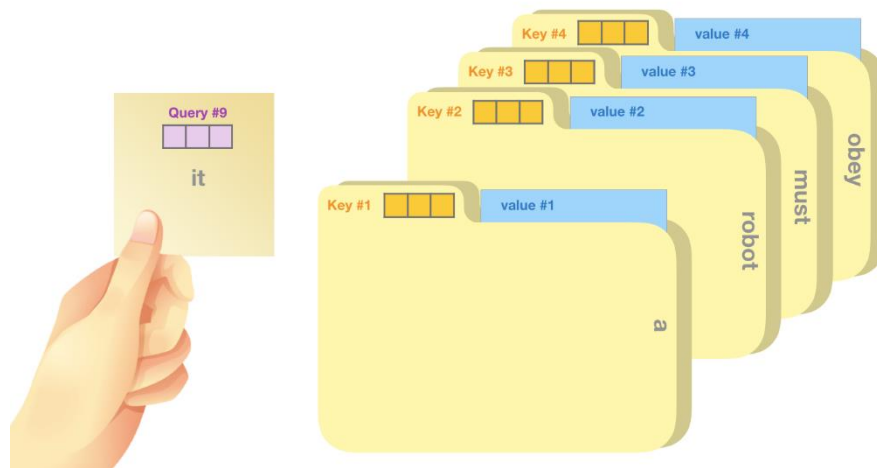
Alamar (Transformer)

- 셀프 어텐션 절차

- ✓ Step 1: 입력 벡터에 대해서 세 가지의 벡터를 생성

- **Query**: 다른 단어들을 고려하여 표현하고자 하는 대상이 되는 현재 단어에 대한 임베딩 벡터
 - **Key**: Query가 들어왔을 때 다른 단어들과 매칭을 하기 위해 사용되는 레이블로 사용되는 임베딩 벡터
 - **Value**: Key와 연결된 실제 단어를 나타내는 임베딩 벡터

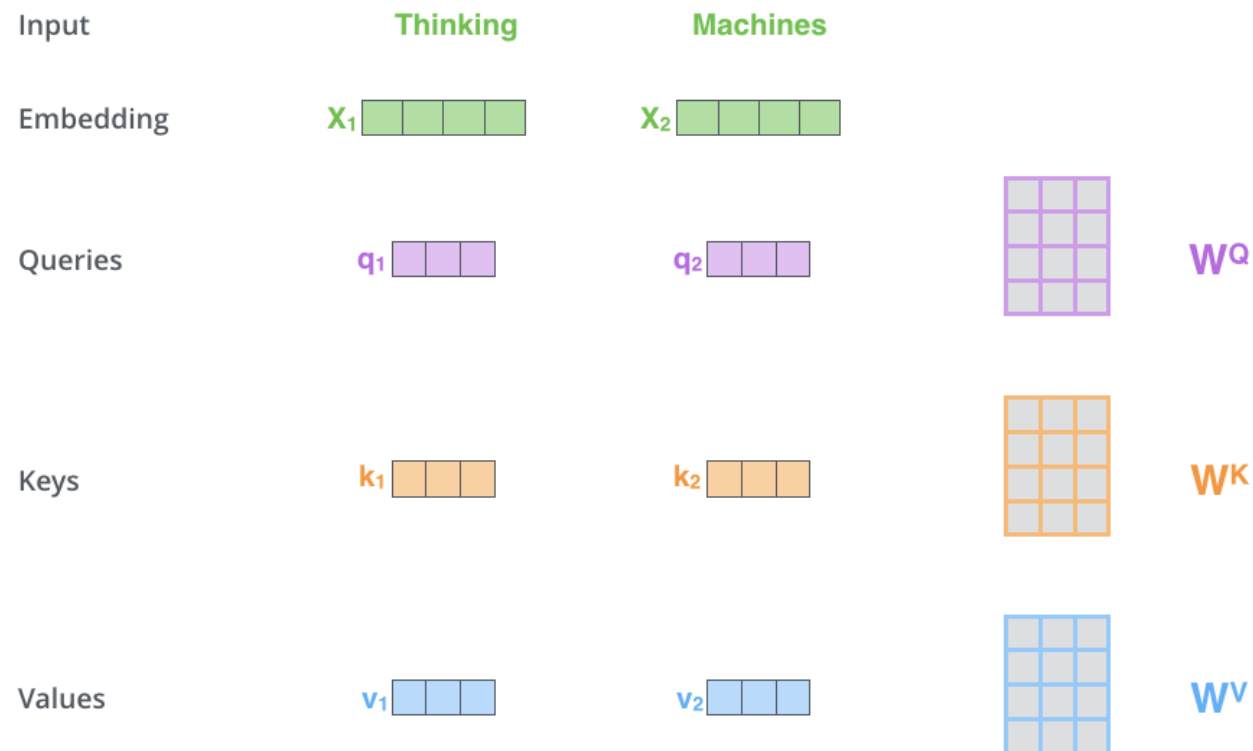
“A robot must obey the orders given it”



Transformer의 작동 원리

Alamar (Transformer)

- 셀프 어텐션 절차
 - ✓ Step 1: 입력 벡터에 대해서 세 가지의 벡터를 생성
 - 실제로는 Query, Key, Value에 대응하는 행렬을 곱해서 생성

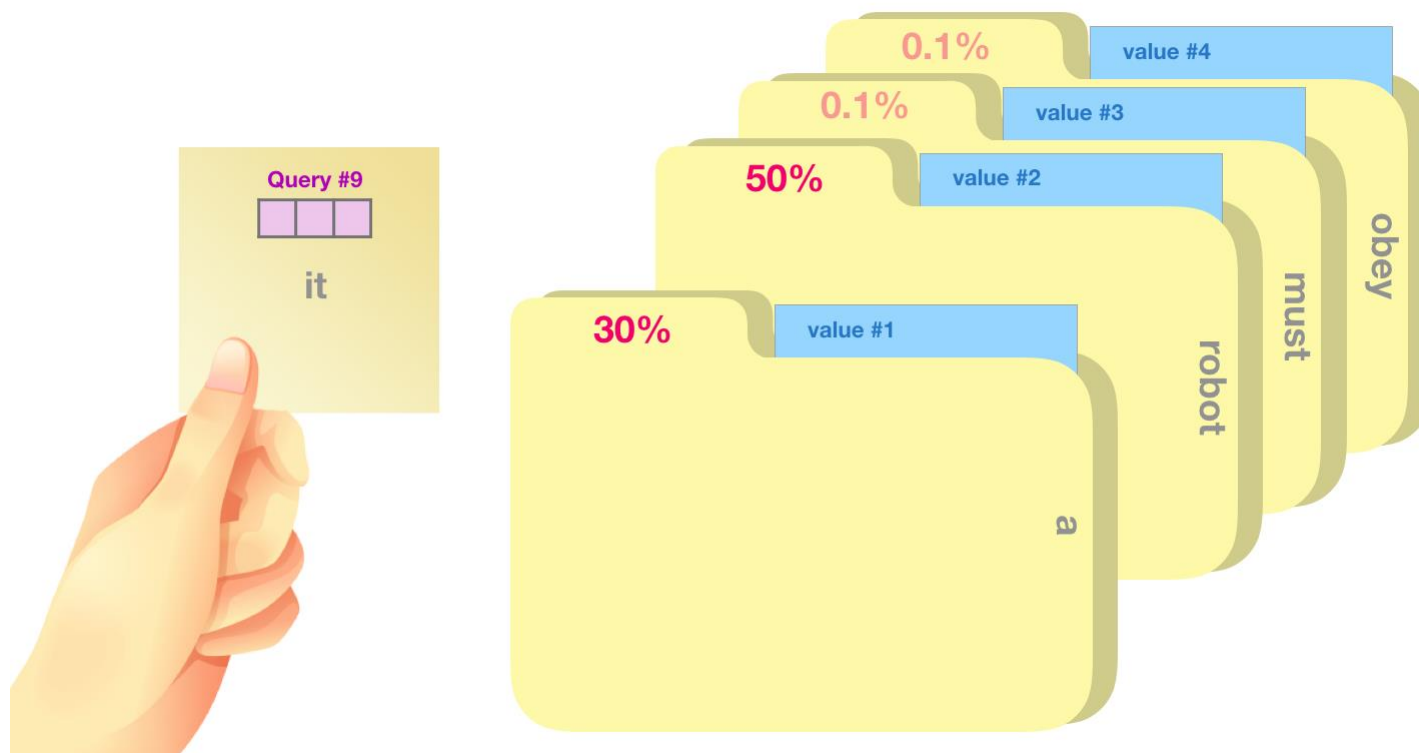


Transformer의 작동 원리

Alamar (Transformer)

- 셀프 어텐션 절차

- ✓ Step 2: 지금 표현하고자 하는 단어(Q)에 대해 어떤 단어들을 고려해야 하는지(K)를 알려주는 스코어 산출
 - Q와 K를 곱한 후 소프트맥스 함수를 취해서 이 스코어를 계산

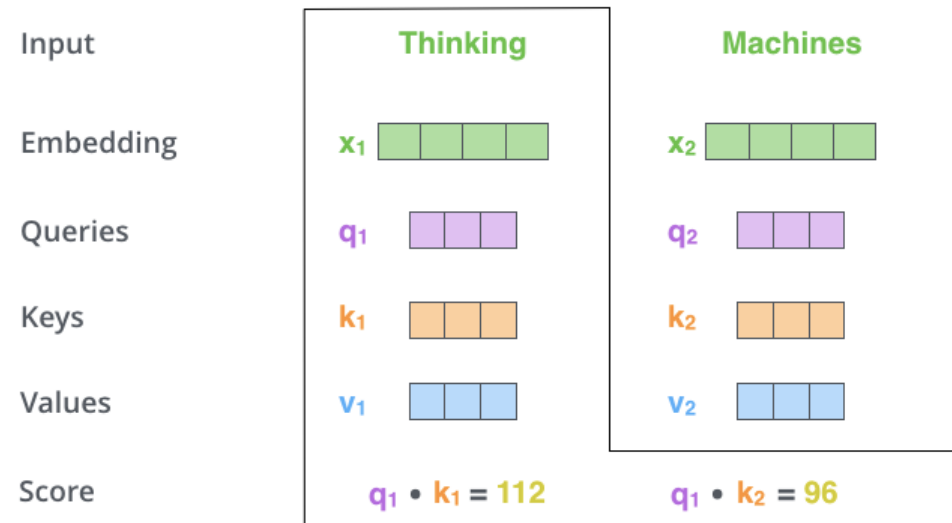


Transformer의 작동 원리

Alamar (Transformer)

- 셀프 어텐션 절차

- ✓ Step 2: 지금 표현하고자 하는 단어(Q)에 대해 어떤 단어들을 고려해야 하는지(K)를 알려주는 스코어 산출
 - Q와 K를 곱한 후 소프트맥스 함수를 취해서 이 스코어를 계산

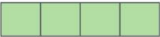
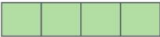
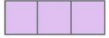
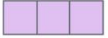
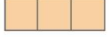
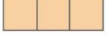
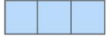
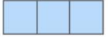


Transformer의 작동 원리

Alamar (Transformer)

- 셀프 어텐션 절차

- ✓ Step 3: Step 2에서 계산한 스코어를 $\sqrt{d_k}$ (= 8 in the original paper since $d_k = 64$)로 나눠줌
 - 이 과정을 통해 그래디언트 전파가 보다 안정적으로 수행됨
- ✓ Step 4: Step 3의 결과물을 이용하여 소프트맥스 함수를 적용하여 해당 단어에 대한 집중도를 산출

Input	Thinking	Machines
Embedding	x_1 	x_2 
Queries	q_1 	q_2 
Keys	k_1 	k_2 
Values	v_1 	v_2 
Score	$q_1 \cdot k_1 = 112$	$q_2 \cdot k_2 = 96$
Divide by 8 ($\sqrt{d_k}$)	14	12
Softmax	0.88	0.12

Transformer의 작동 원리

- 셀프 어텐션 절차

- ✓ Step 5: Step 4에서 산출된 확률값과 해당 단어의 Key 값을 곱함
- ✓ Step 6: Step 5에서 산출된 모든 값들을 더해서 출력으로 반환

Alamar (Transformer)

Input

Embedding

Queries

Keys

Values

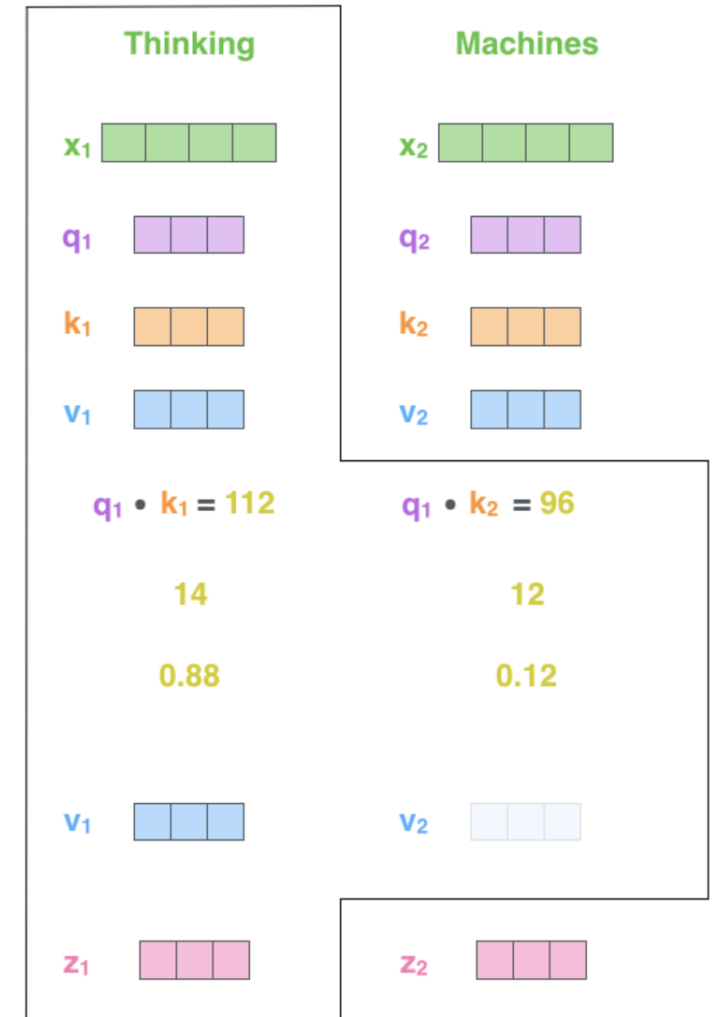
Score

Divide by 8 ($\sqrt{d_k}$)

Softmax

Softmax
X
Value

Sum

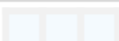
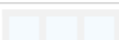
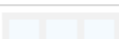
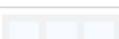
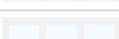


Transformer의 작동 원리

Alamar (Transformer)

- 셀프 어텐션 절차

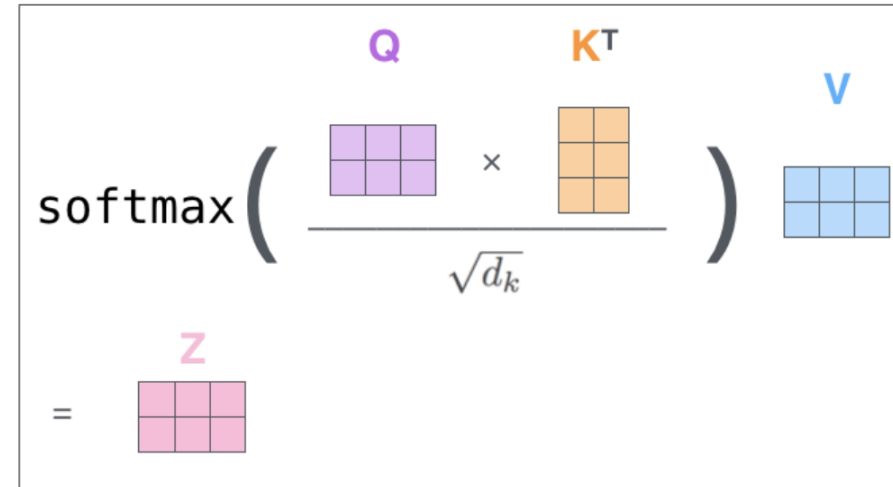
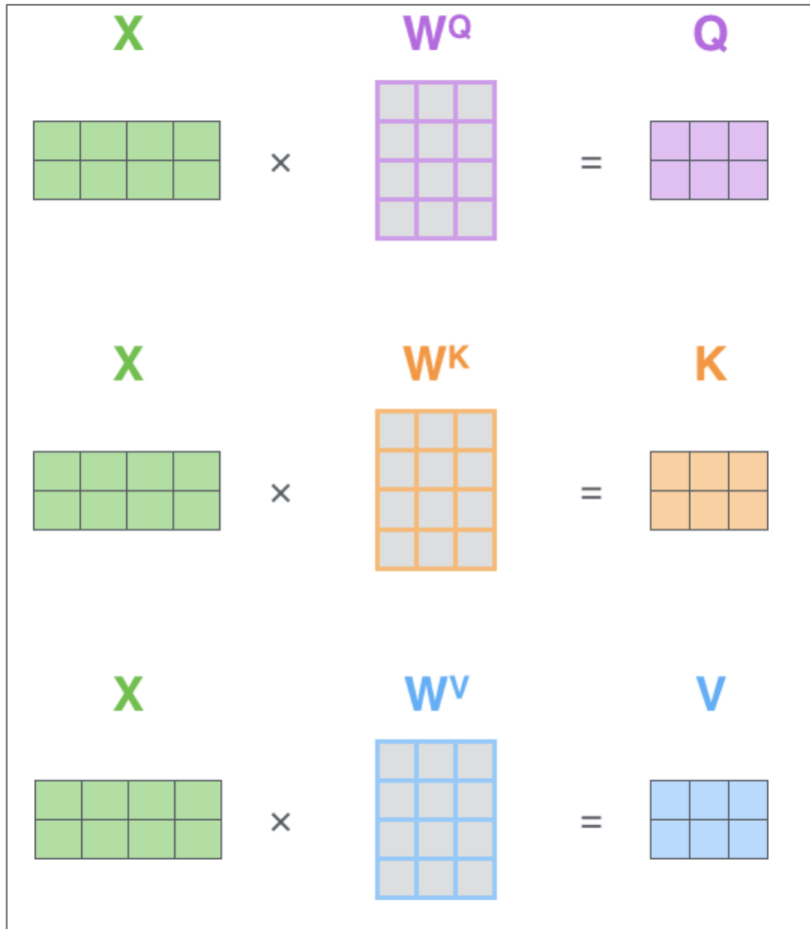
- ✓ Step 5: Step 4에서 산출된 확률값과 해당 단어의 Key 값을 곱함
- ✓ Step 6: Step 5에서 산출된 모든 값들을 더해서 출력으로 반환

Word	Value vector	Score	Value X Score
<S>		0.001	
a		0.3	
robot		0.5	
must		0.002	
obey		0.001	
the		0.0003	
orders		0.005	
given		0.002	
it		0.19	
		Sum:	

Transformer의 작동 원리

Alamar (Transformer)

- 실제로는 이 과정이 모두 행렬 연산으로 수행됨: Matrix calculation of self-attention



Transformer의 작동 원리

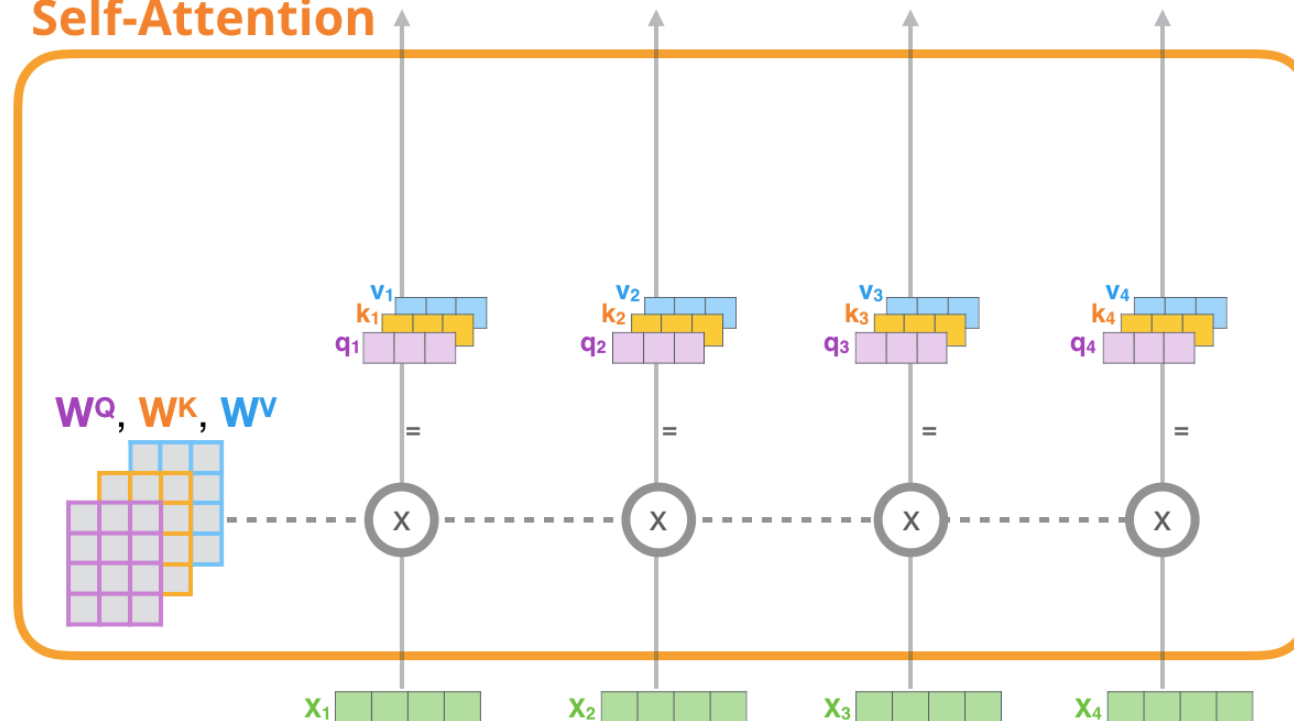
Alamar (GPT-2)

- 셀프 어텐션 예시

- ✓ Create Query, Key, and Value Vectors

1) For each input token, create a **query vector**, a **key vector**, and a **value vector** by multiplying by weight Matrices W^Q , W^K , W^V

Self-Attention



Transformer의 작동 원리

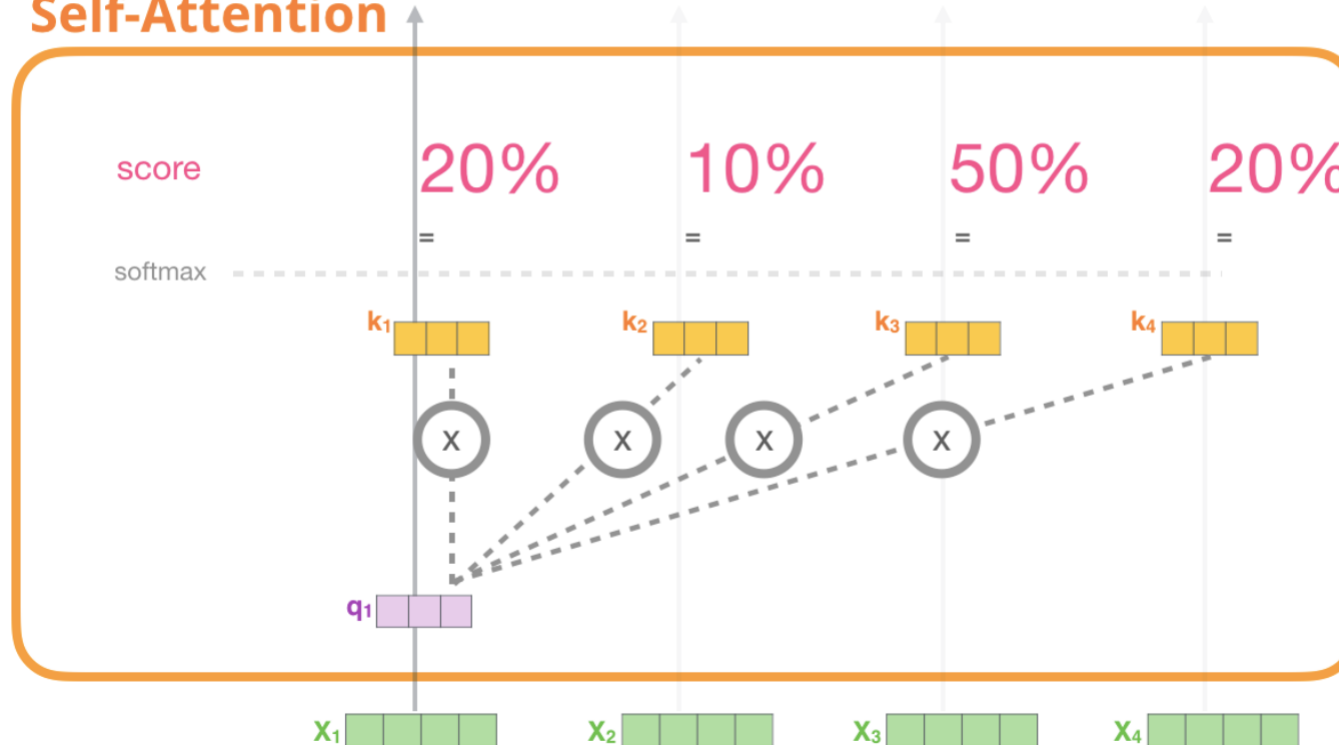
Alamar (GPT-2)

- 셀프 어텐션 예시

- ✓ Score

2) Multiply (dot product) the current **query vector**, by all the **key vectors**, to get a score of how well they match

Self-Attention



Transformer의 작동 원리

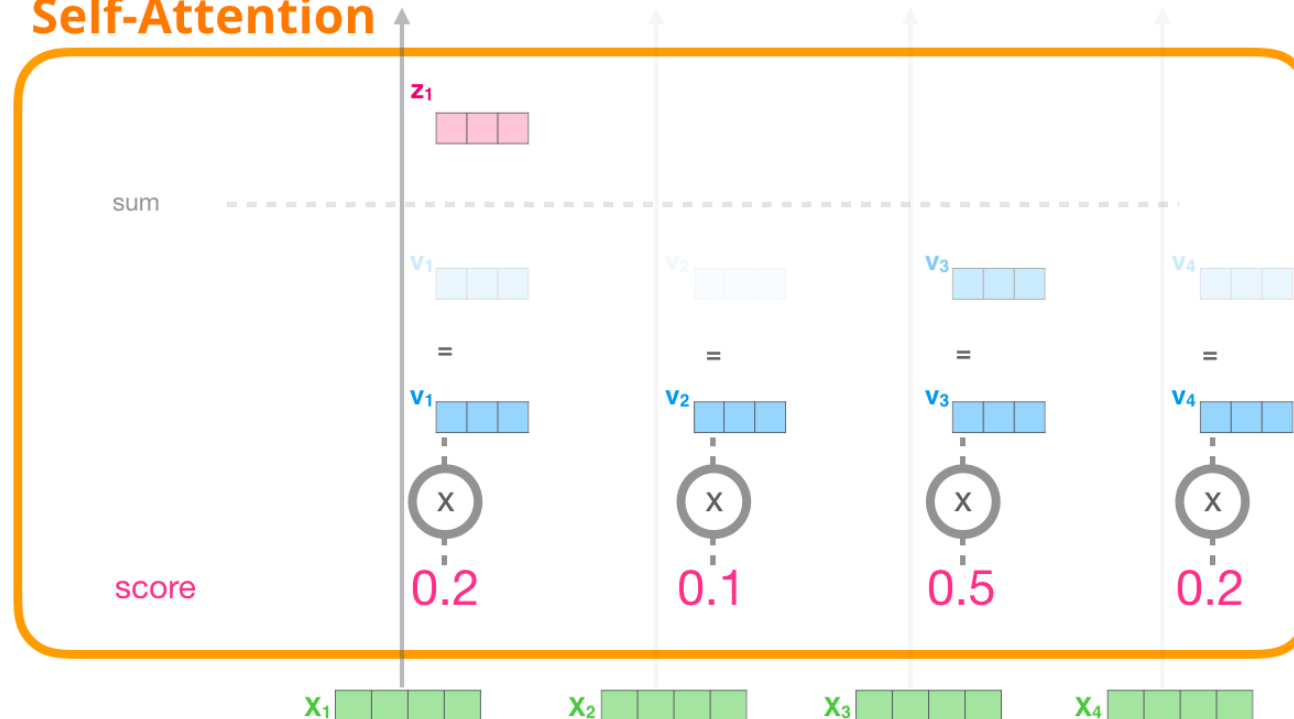
Alamar (GPT-2)

- 셀프 어텐션 예시

- ✓ Sum

3) Multiply the **value vectors** by the **scores**, then sum up

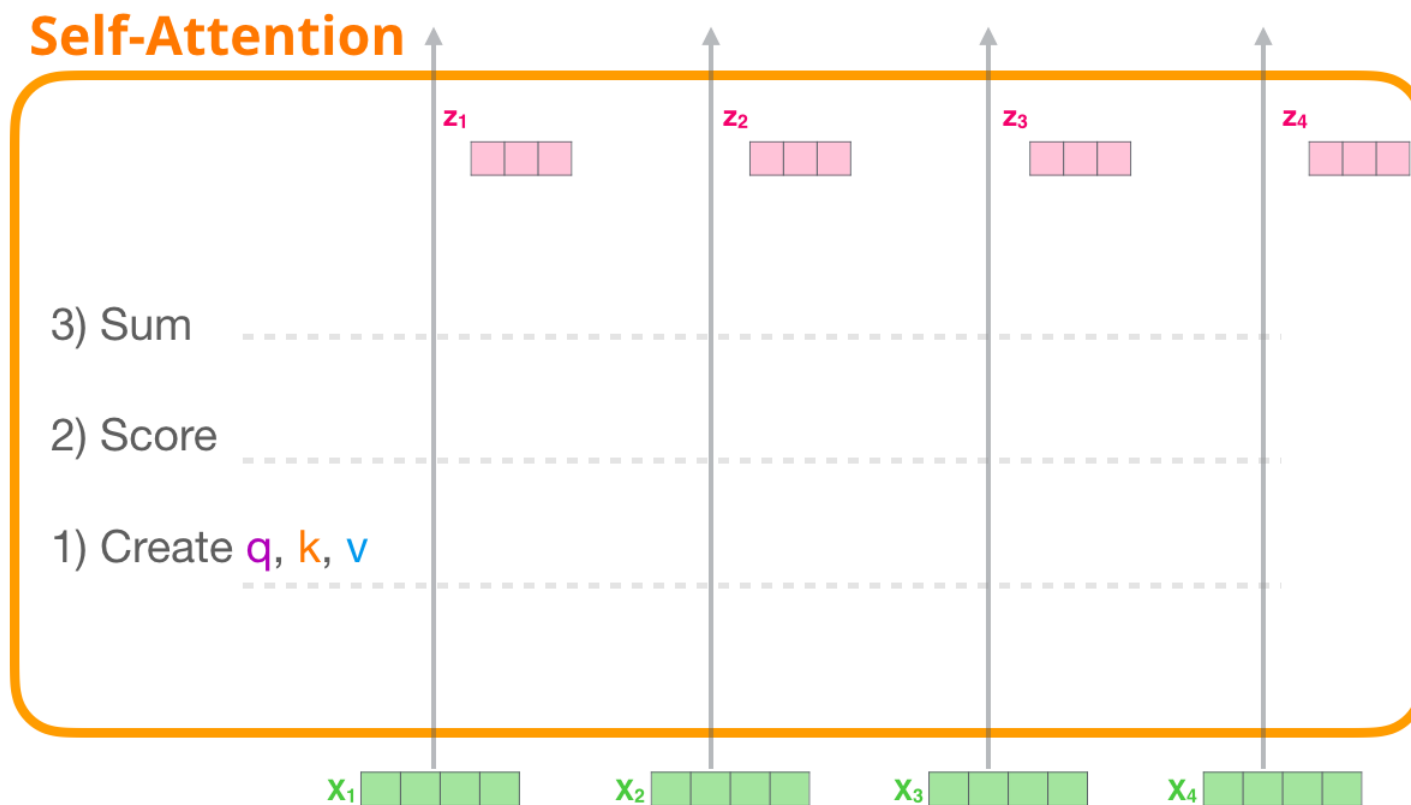
Self-Attention



Transformer의 작동 원리

Alamar (GPT-2)

- 셀프 어텐션 예시
 - ✓ 동일한 작업을 모든 입력 토큰에 대해 각각 수행

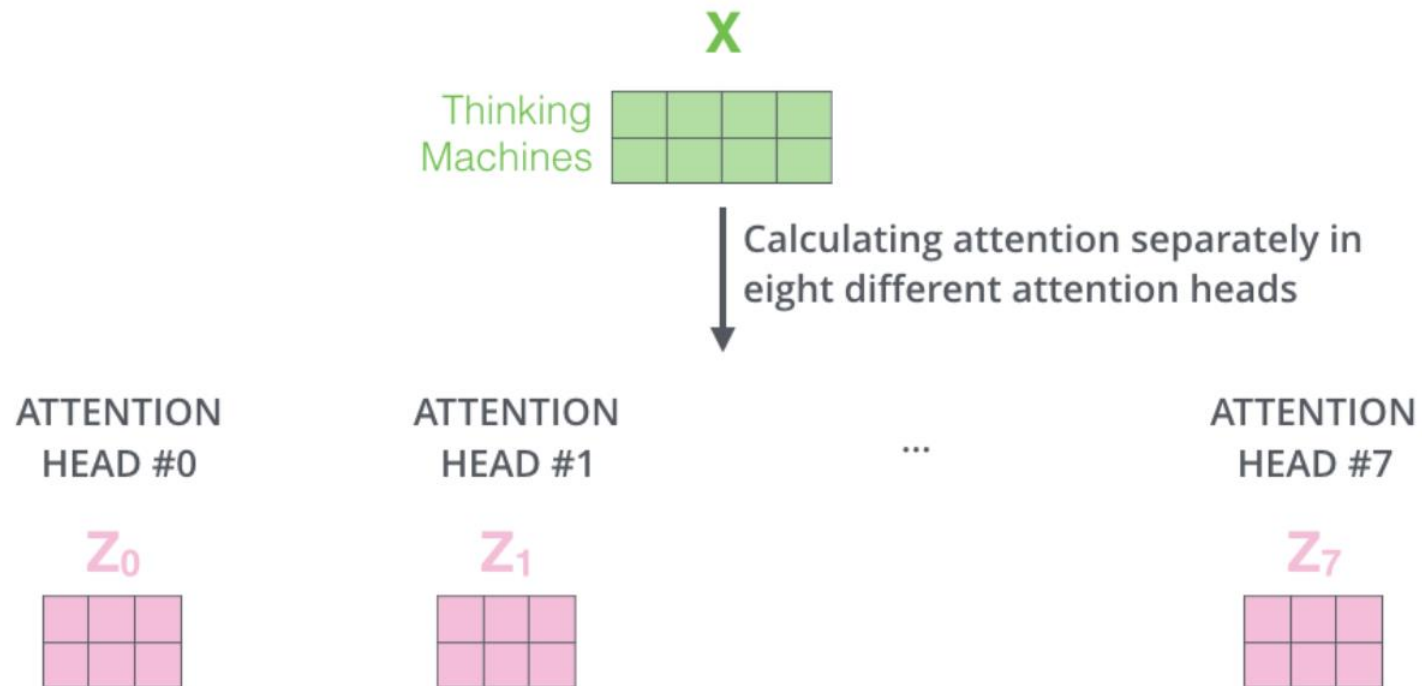


Transformer의 작동 원리

Alamar (Transformer)

- 멀티헤드 어텐션 Multi-headed attention

- ✓ 한 Query 토큰에 대해서 다양한 관점으로 표현할 수 있는 능력을 제공



Transformer의 작동 원리

Alamar (Transformer)

- 멀티헤드 어텐션 Multi-headed attention

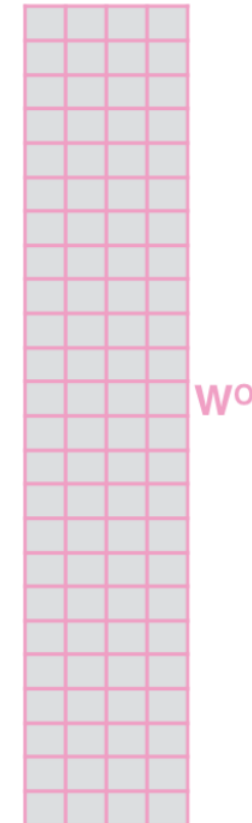
✓ 어텐션 결과물들은 병합(concatenation) 후 가중치 행렬과의 연산을 통해 원래 입력 차원과 동일한 차원의 출력 벡터를 생성하게 됨

1) Concatenate all the attention heads

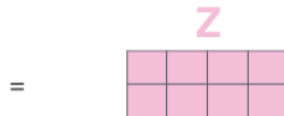


2) Multiply with a weight matrix W^O that was trained jointly with the model

X



3) The result would be the Z matrix that captures information from all the attention heads. We can send this forward to the FFNN



Transformer의 작동 원리

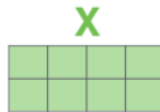
Alamar (Transformer)

- 멀티헤드 어텐션 Multi-headed attention

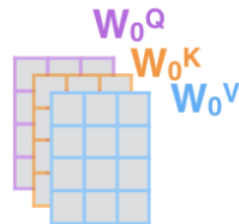
1) This is our input sentence*

Thinking
Machines

2) We embed each word*



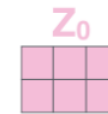
3) Split into 8 heads.
We multiply X or R with weight matrices



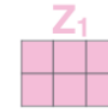
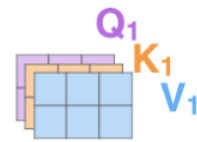
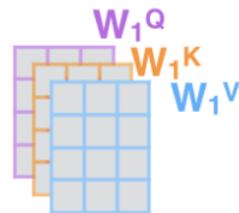
4) Calculate attention using the resulting $Q/K/V$ matrices



5) Concatenate the resulting Z matrices, then multiply with weight matrix W^O to produce the output of the layer



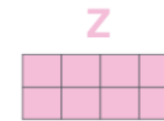
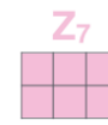
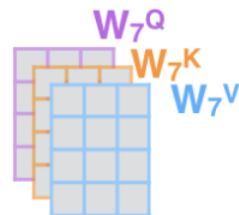
* In all encoders other than #0, we don't need embedding. We start directly with the output of the encoder right below this one



...

...

...

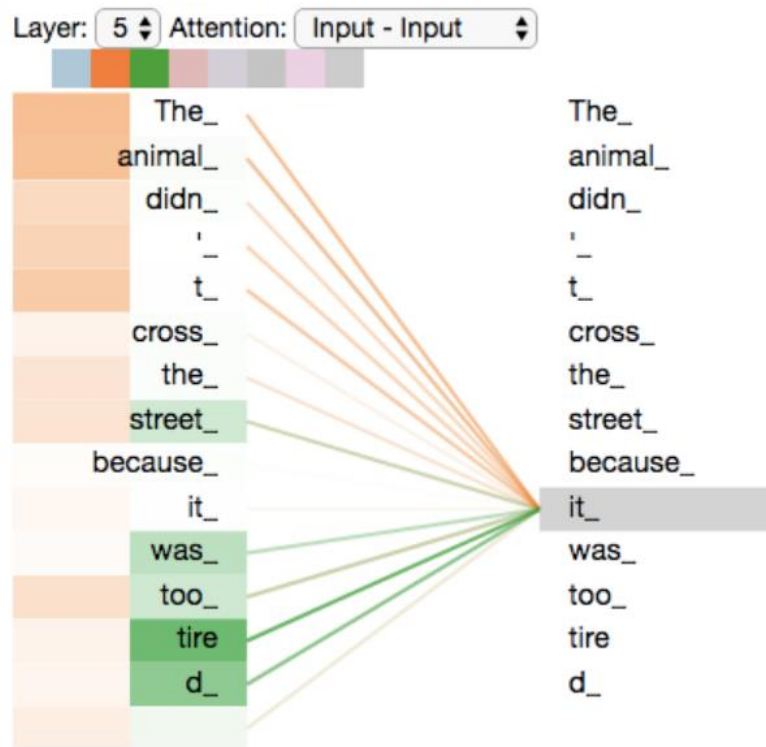


Transformer의 작동 원리

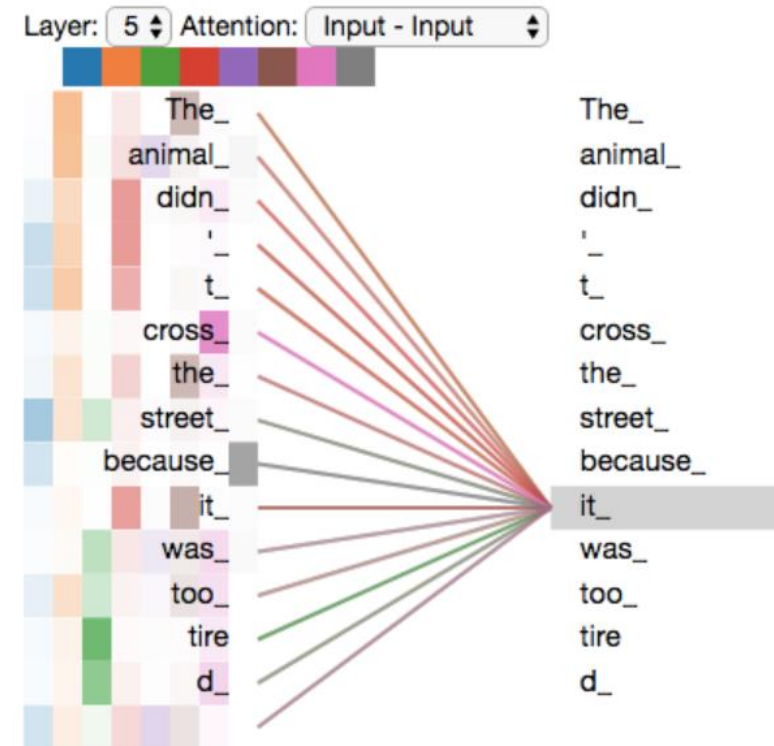
Alamar (Transformer)

- 멀티헤드 어텐션 Multi-headed attention

Attention with two heads



Attention with eight heads

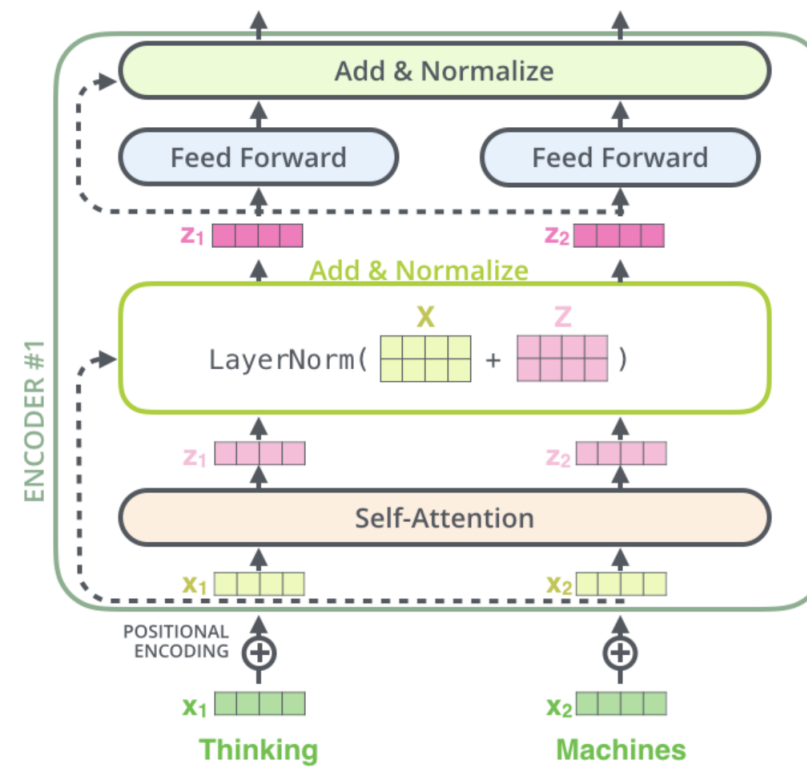
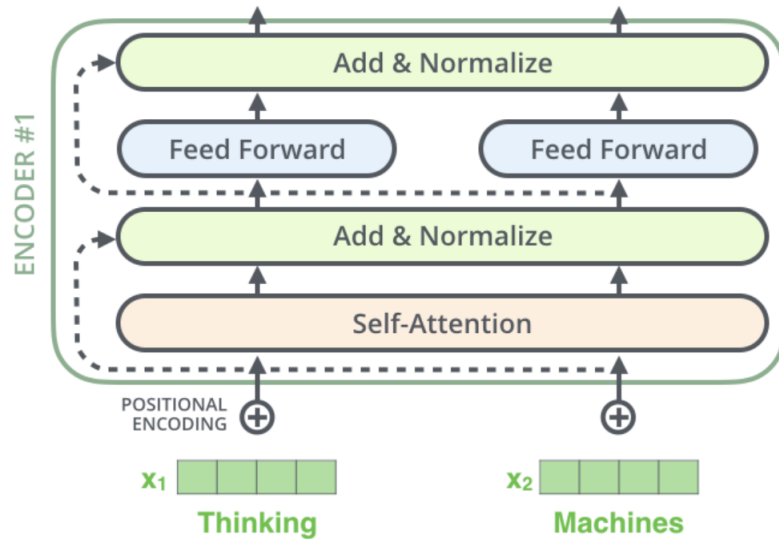


Transformer의 작동 원리

Alamar (Transformer)

- Residual

✓ 각 어텐션 블록에서는 Residual Connection과 Layer Normalization이 수행됨

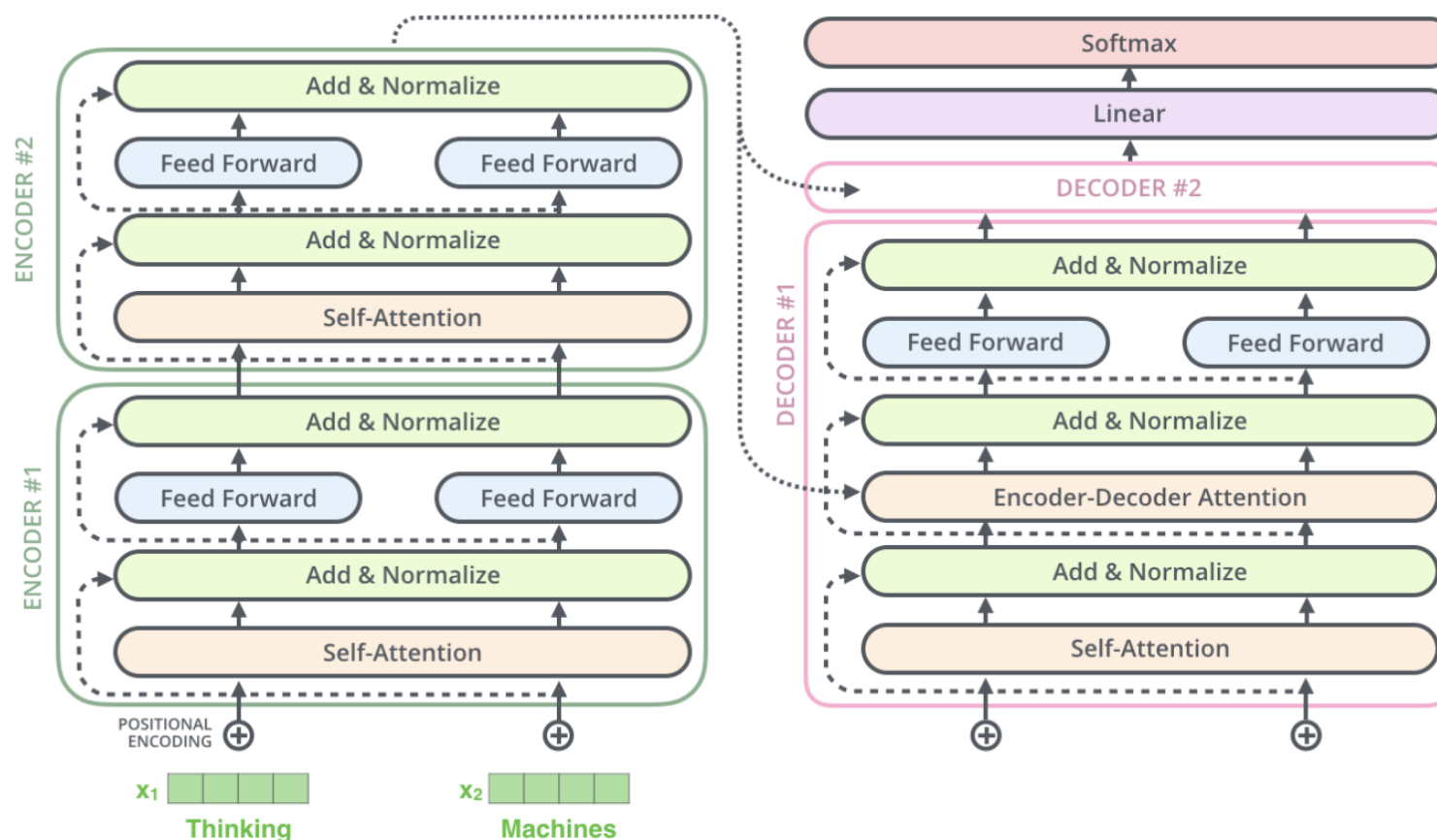


Transformer의 작동 원리

Alamar (Transformer)

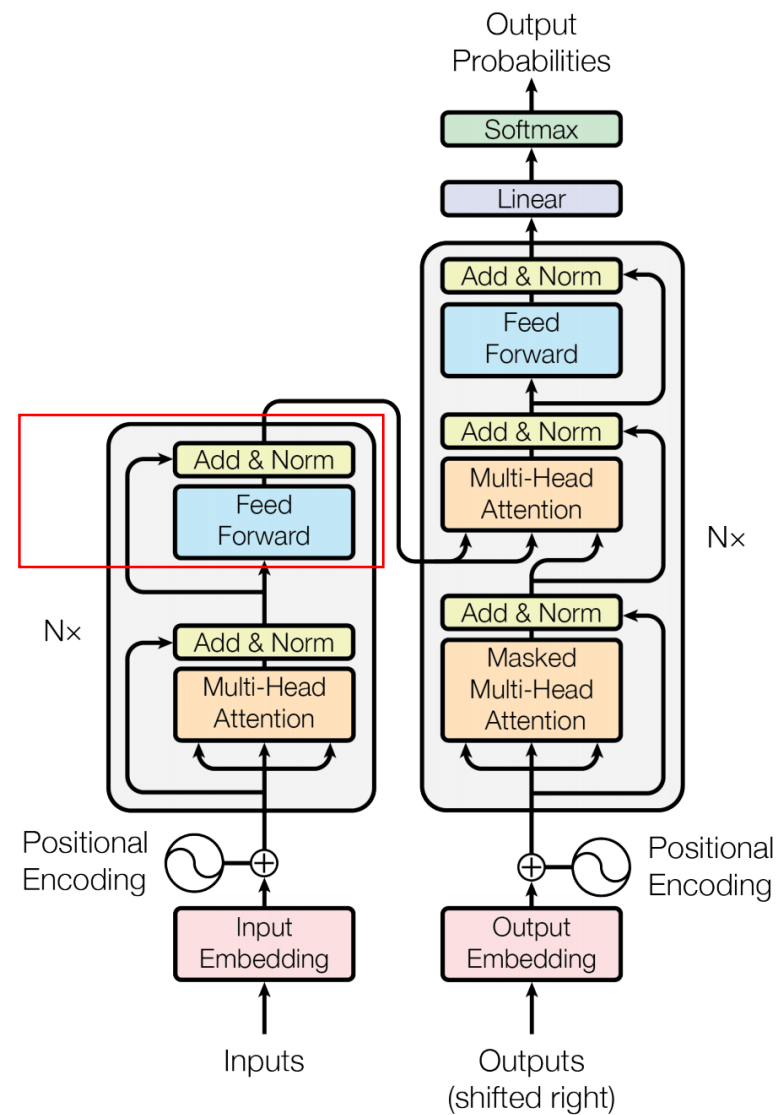
- Residual

✓ 디코더 과정에서도 이 절차는 적용됨



Transformer의 작동 원리

- Position-wise Feed-Forward Networks



Transformer의 작동 원리

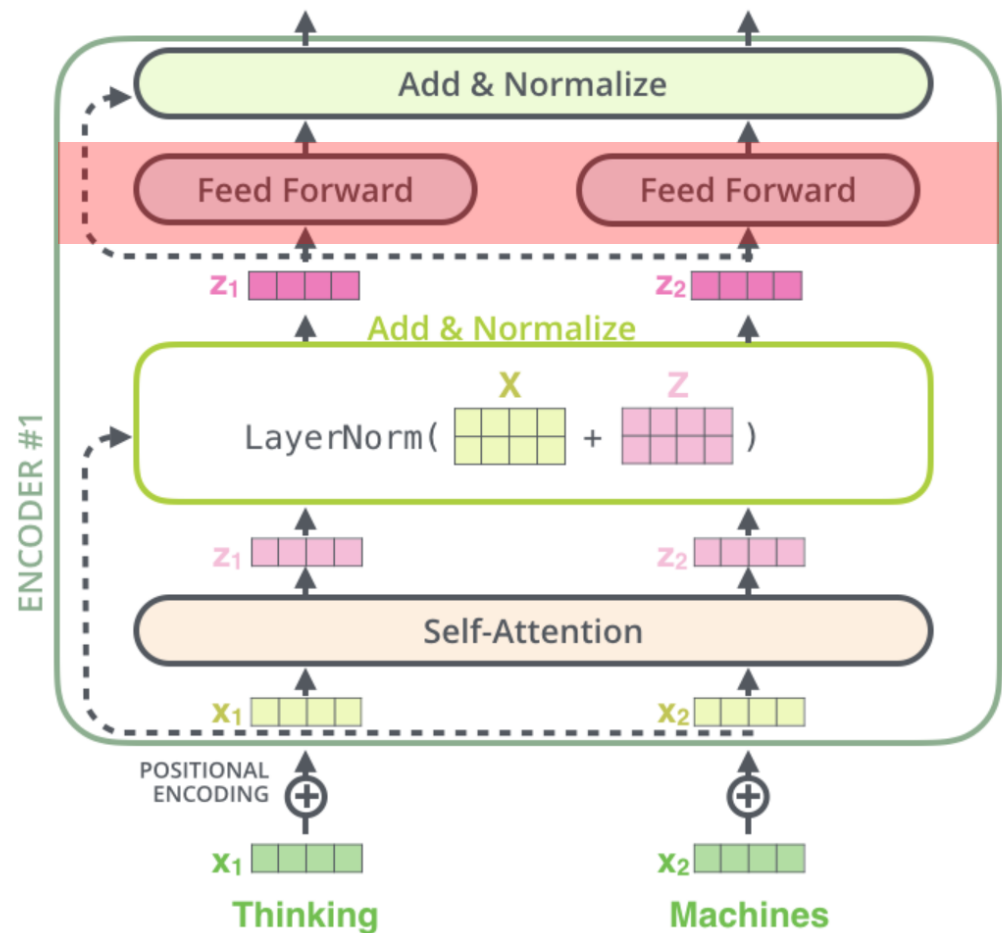
Vaswani et. al (2017), Kim (2019)

- Position-wise Feed-Forward Networks

- ✓ Fully connected feed-forward network
- ✓ 각 포지션에 대해서 독립적으로 수행됨

$$FFN(x) = \max(0, xW_1 + b_1)W_2 + b_2$$

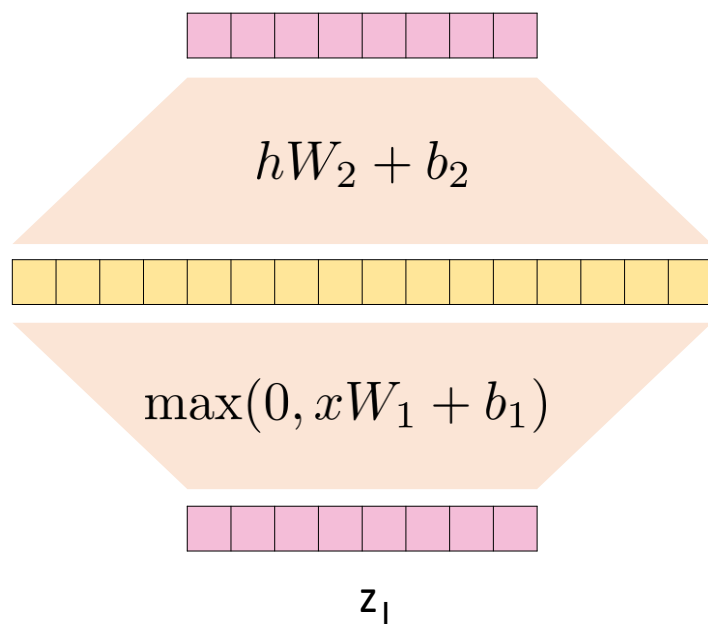
- ✓ 같은 인코더 블록에서는 동일한 가중치를 사용
- ✓ 다른 인코더 블록끼리는 서로 다른 가중치를 학습



Transformer의 작동 원리

Vaswani et. al (2017), Kim (2019)

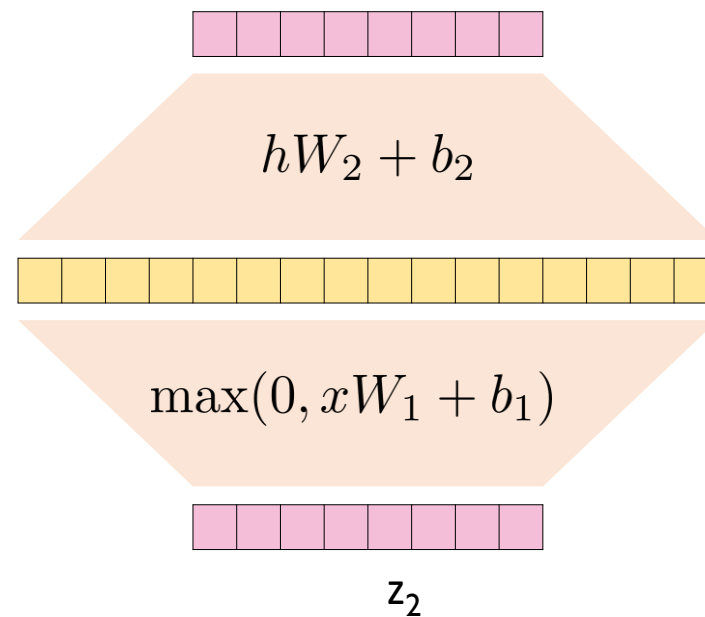
- Position-wise Feed-Forward Networks



512 dim.

2048 dim.

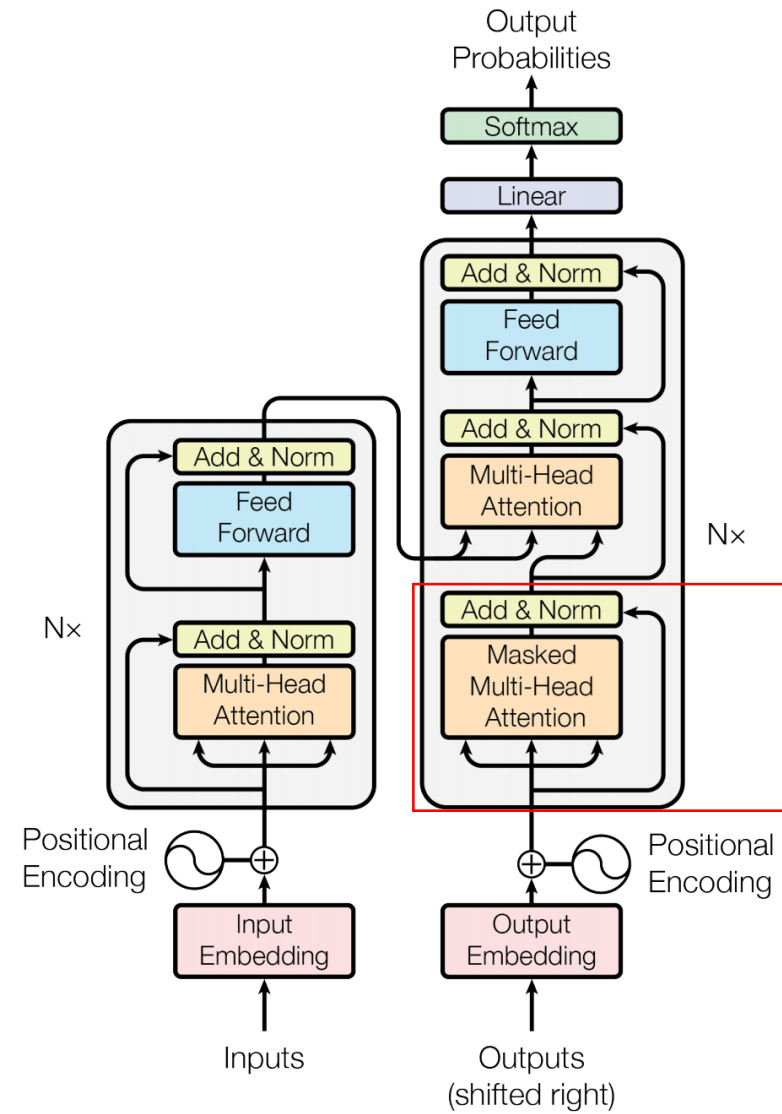
512 dim.



z_2

Transformer의 작동 원리

- Masked Multi-Head Attention

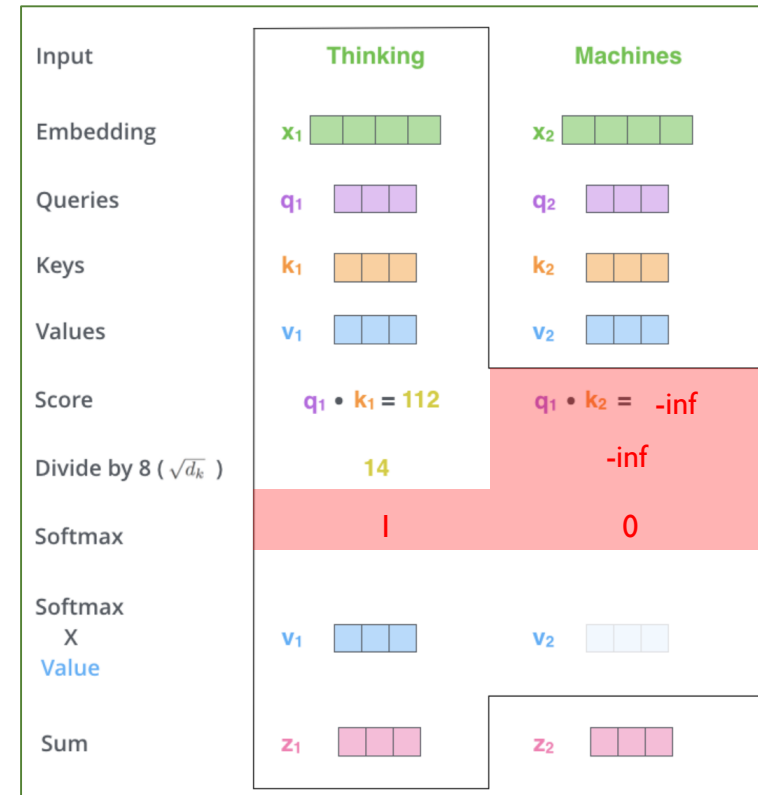
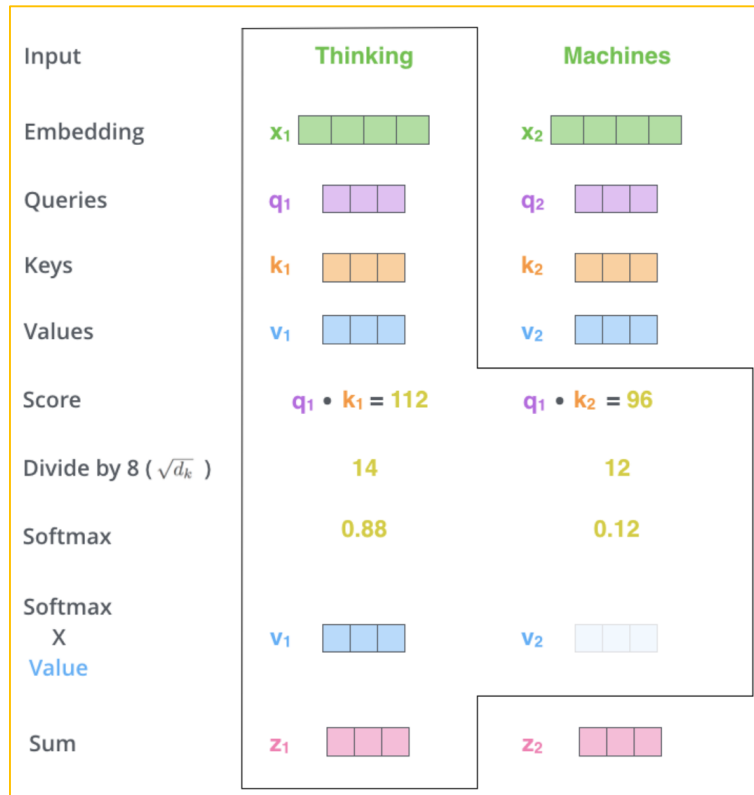


Transformer의 작동 원리

Alamar (Transformer)

- Masked Multi-head Attention

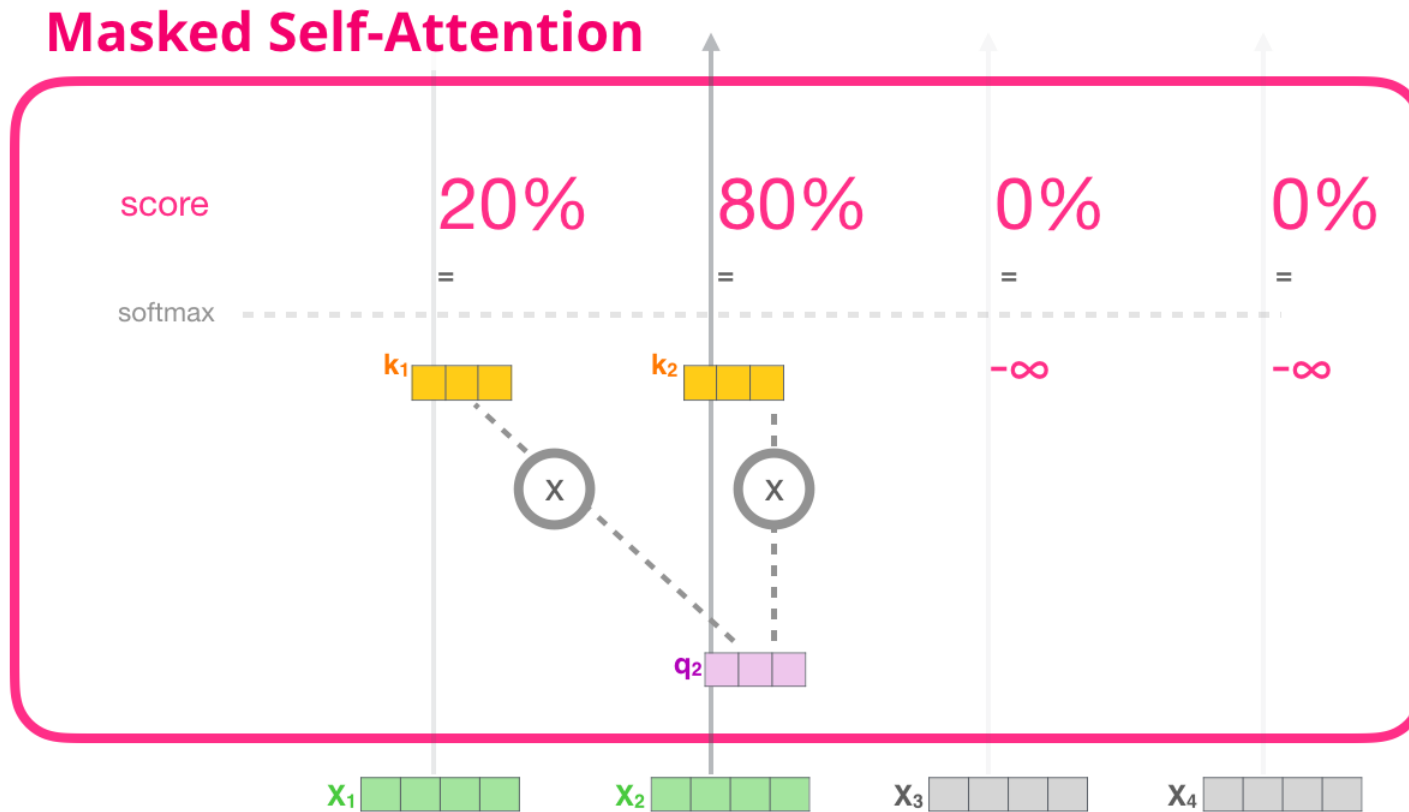
✓ 디코딩 단계에서 셀프 어텐션은 Query 토큰보다 뒤에 위치한 토큰들에 대한 정보는 가용하지 않다고 가정하고 해당 부분을 전부 마스킹(Masking) 처리



Transformer의 작동 원리

Alamar (GPT-2)

- Masked Multi-head Attention



Transformer의 작동 원리

Alamar (GPT-2)

- Masked Multi-head Attention

✓ 순차적으로 수행할 필요 없이 한번에 수행 가능

Features				Labels	
position: 1 2 3 4					
Example:					
1	robot	must	obey	orders	must
2	robot	must	obey	orders	obey
3	robot	must	obey	orders	orders
4	robot	must	obey	orders	<eos>

Transformer의 작동 원리

Alamar (GPT-2)

- Masked Multi-head Attention

Attention

Queries

robot	must	obey	orders
-------	------	------	--------

X

Keys

robot	must	obey	orders
robot	must	obey	orders
robot	must	obey	orders
robot	must	obey	orders

=

Scores
(before softmax)

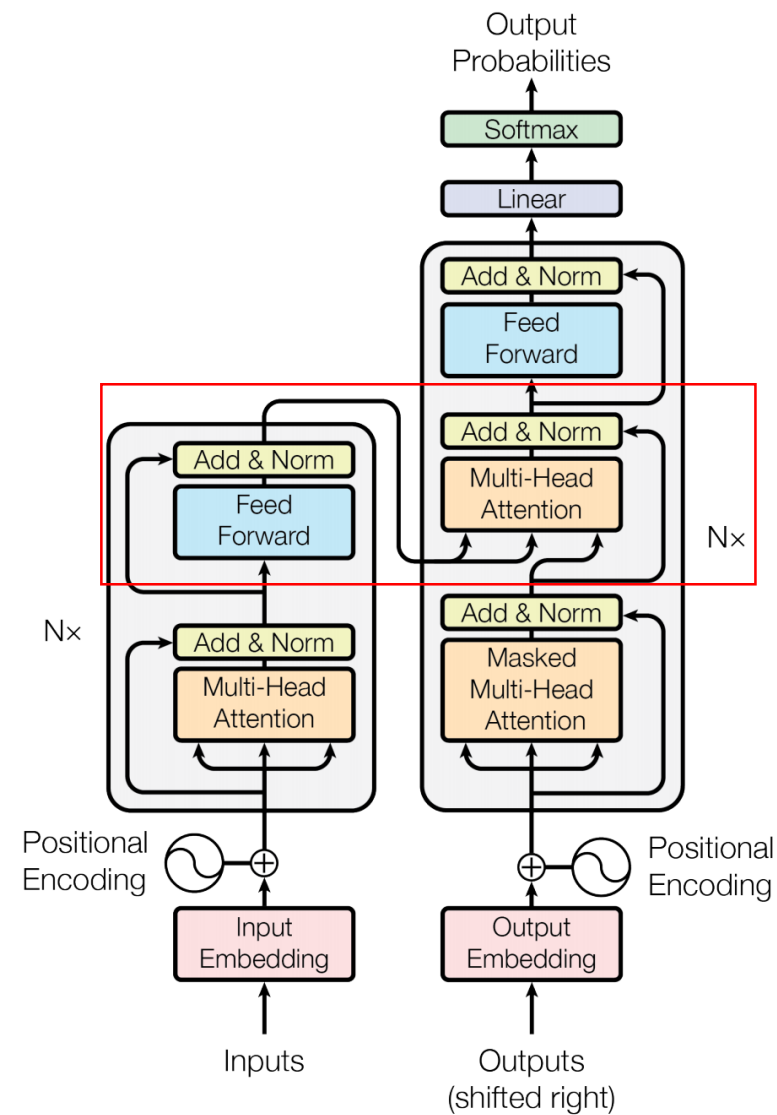
0.11	0.00	0.81	0.79
0.19	0.50	0.30	0.48
0.53	0.98	0.95	0.14
0.81	0.86	0.38	0.90

Scores (before softmax)				Masked Scores (before softmax)			
0.11	0.00	0.81	0.79	Apply Attention Mask			
0.19	0.50	0.30	0.48				
0.53	0.98	0.95	0.14				
0.81	0.86	0.38	0.90				
0.11	-inf	-inf	-inf				
0.19	0.50	-inf	-inf				
0.53	0.98	0.95	-inf				
0.81	0.86	0.38	0.90				

Masked Scores (before softmax)				Scores			
0.11	-inf	-inf	-inf	Softmax (along rows)			
0.19	0.50	-inf	-inf				
0.53	0.98	0.95	-inf				
0.81	0.86	0.38	0.90				
1	0	0	0				
0.48	0.52	0	0				
0.31	0.35	0.34	0				
0.25	0.26	0.23	0.26				

Transformer의 작동 원리

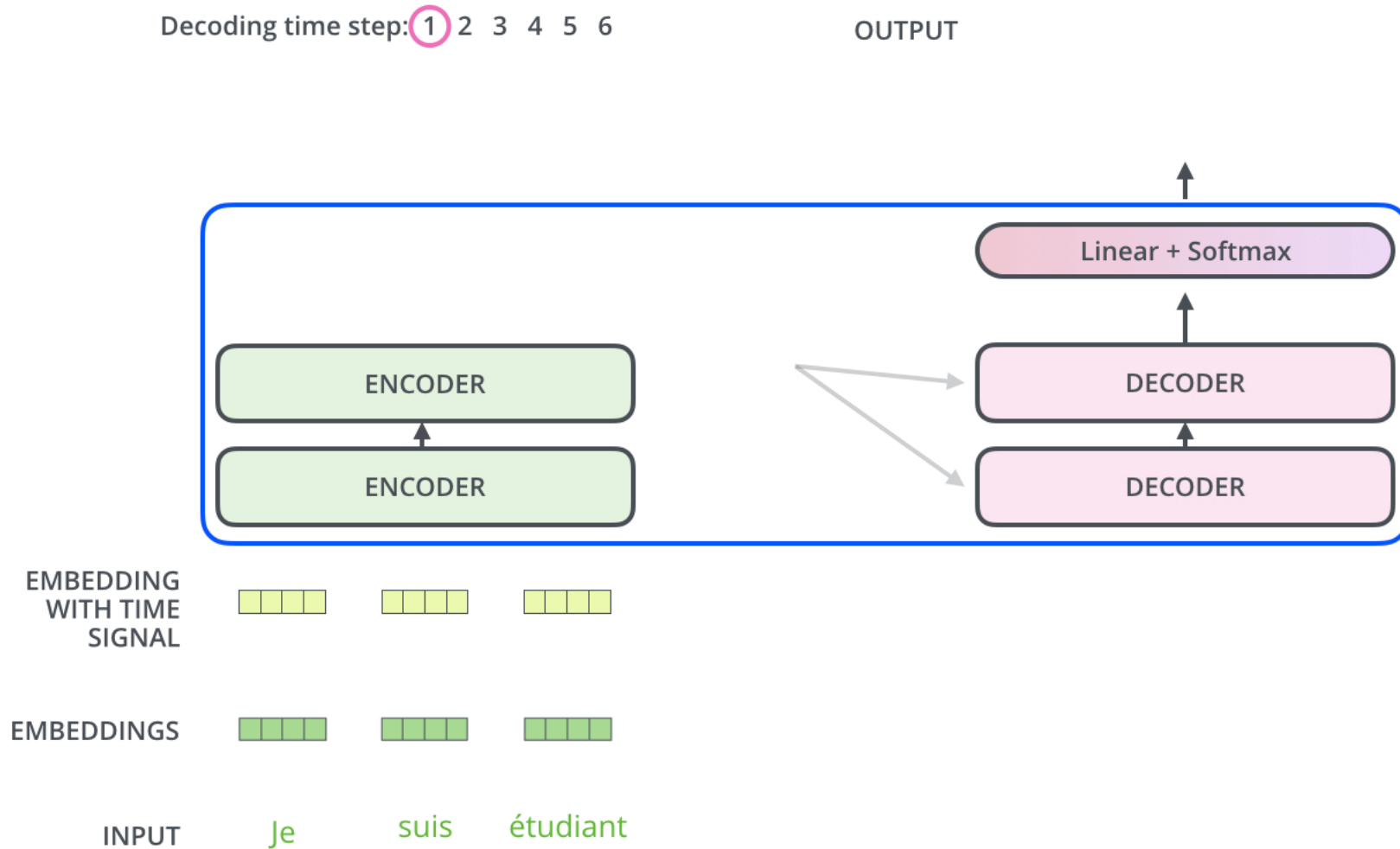
- Multi-Head Attention with Encoder Outputs



Transformer의 작동 원리

Alamar (Transformer)

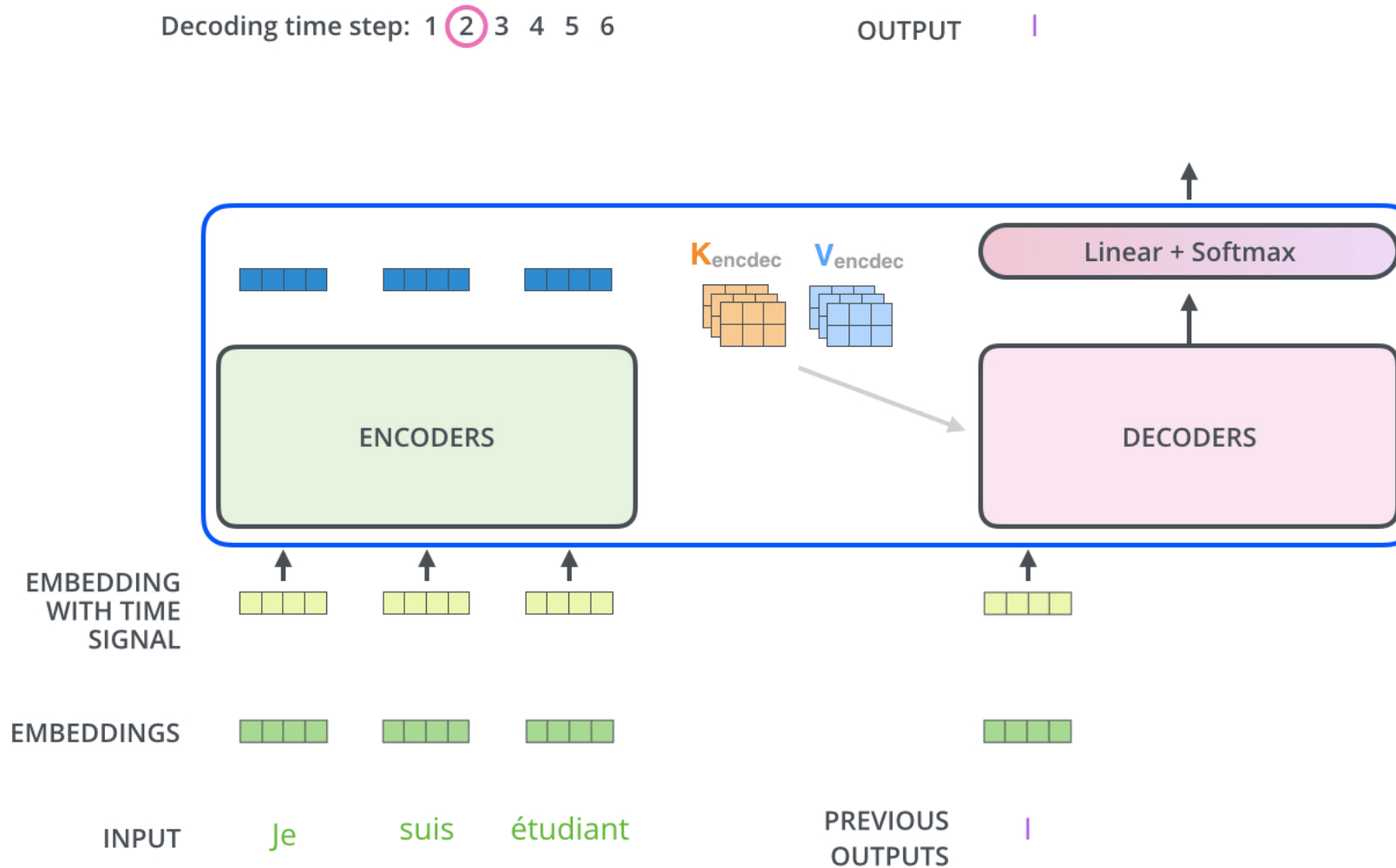
- Decoder Side



Transformer의 작동 원리

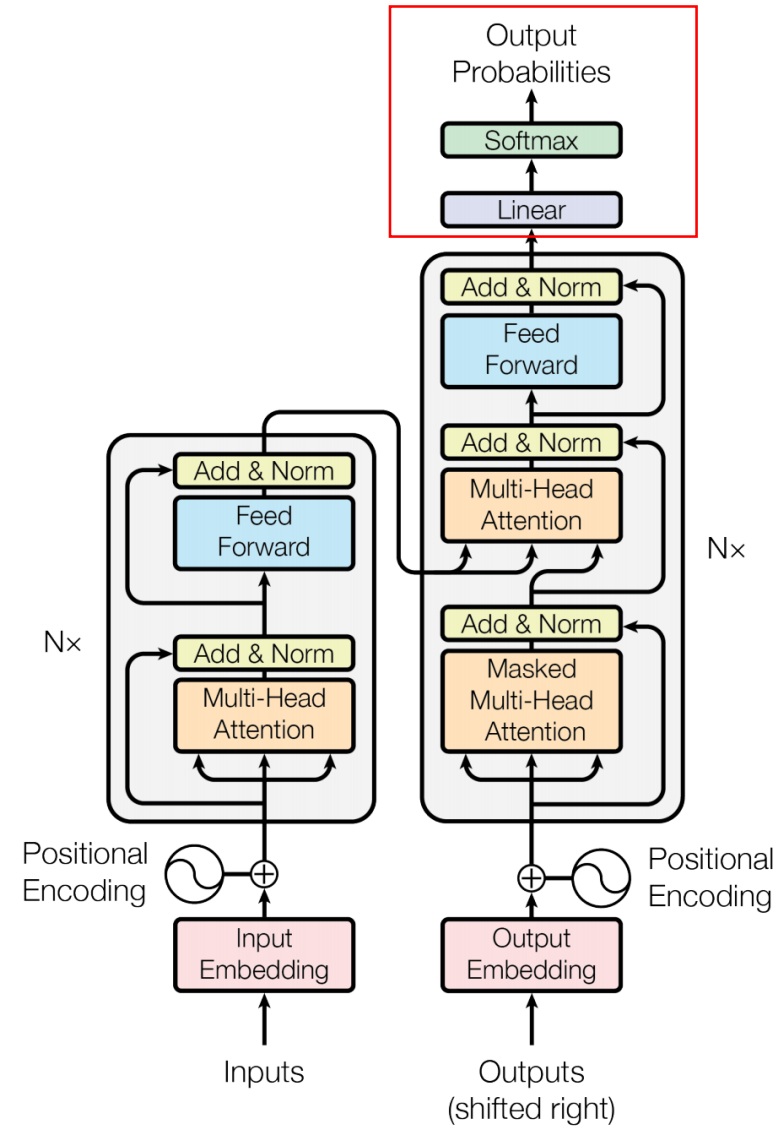
Alamar (Transformer)

- Decoder Side



Transformer의 작동 원리

- The Final Linear and Softmax Layer



Transformer의 작동 원리

Alamar (Transformer)

- The Final Linear and Softmax Layer

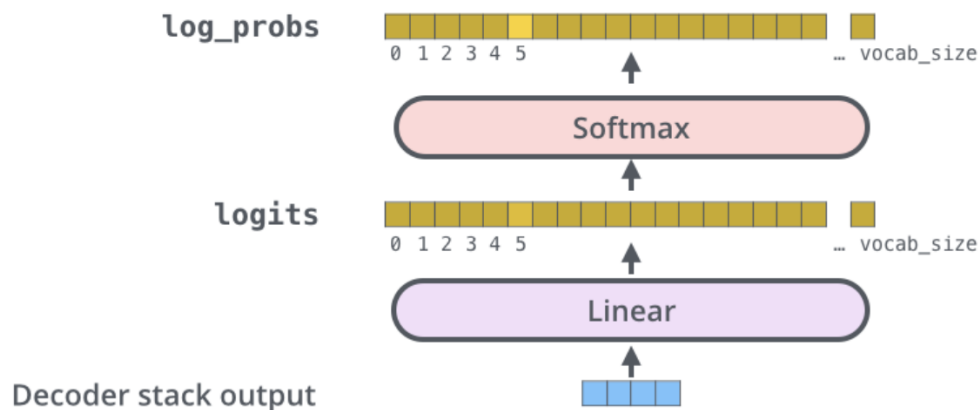
- ✓ Linear layer: 단순 FFNN 형태로서 마지막 디코더의 출력 결과물을 이용하여 모든 단어들의 출력 확률을 산출하기 위해 차원을 늘리는 역할을 수행
- ✓ Softmax layer: 개별 단어들의 출력 확률 반환

Which word in our vocabulary
is associated with this index?

am

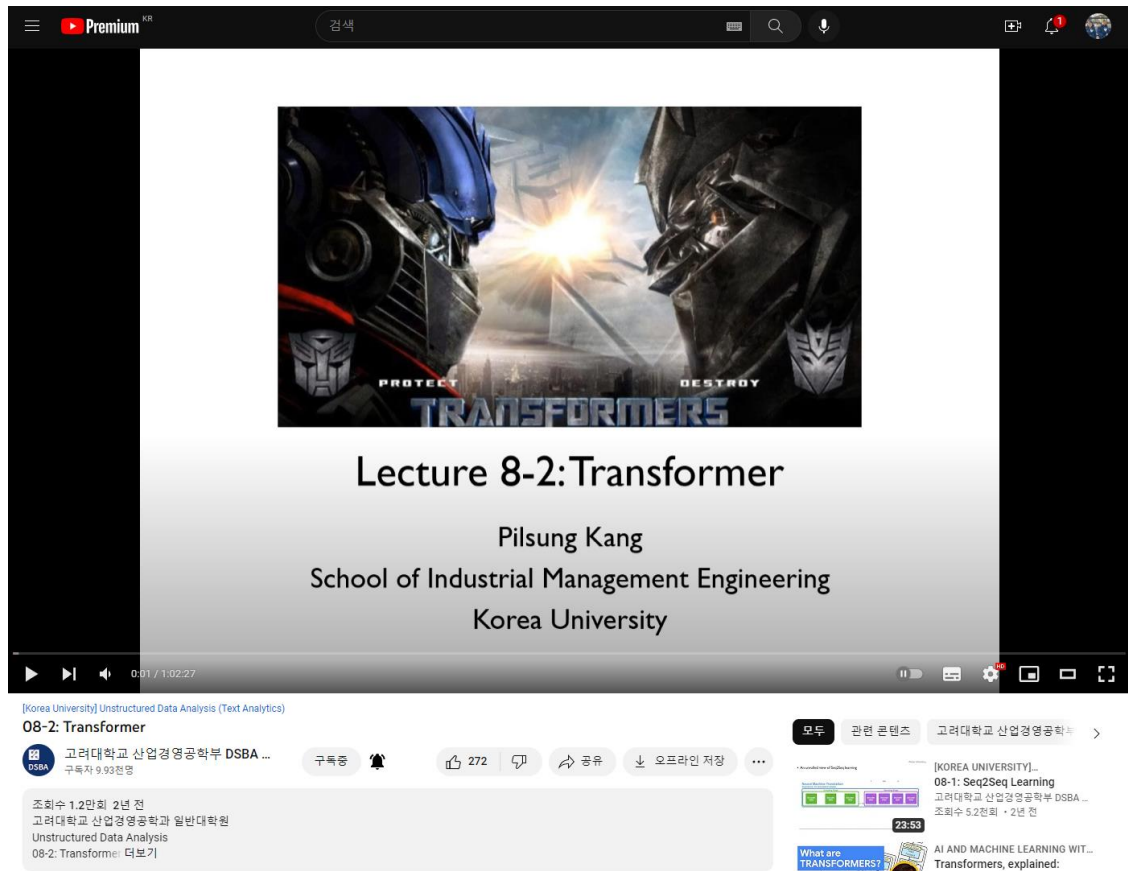
Get the index of the cell
with the highest value
(argmax)

5



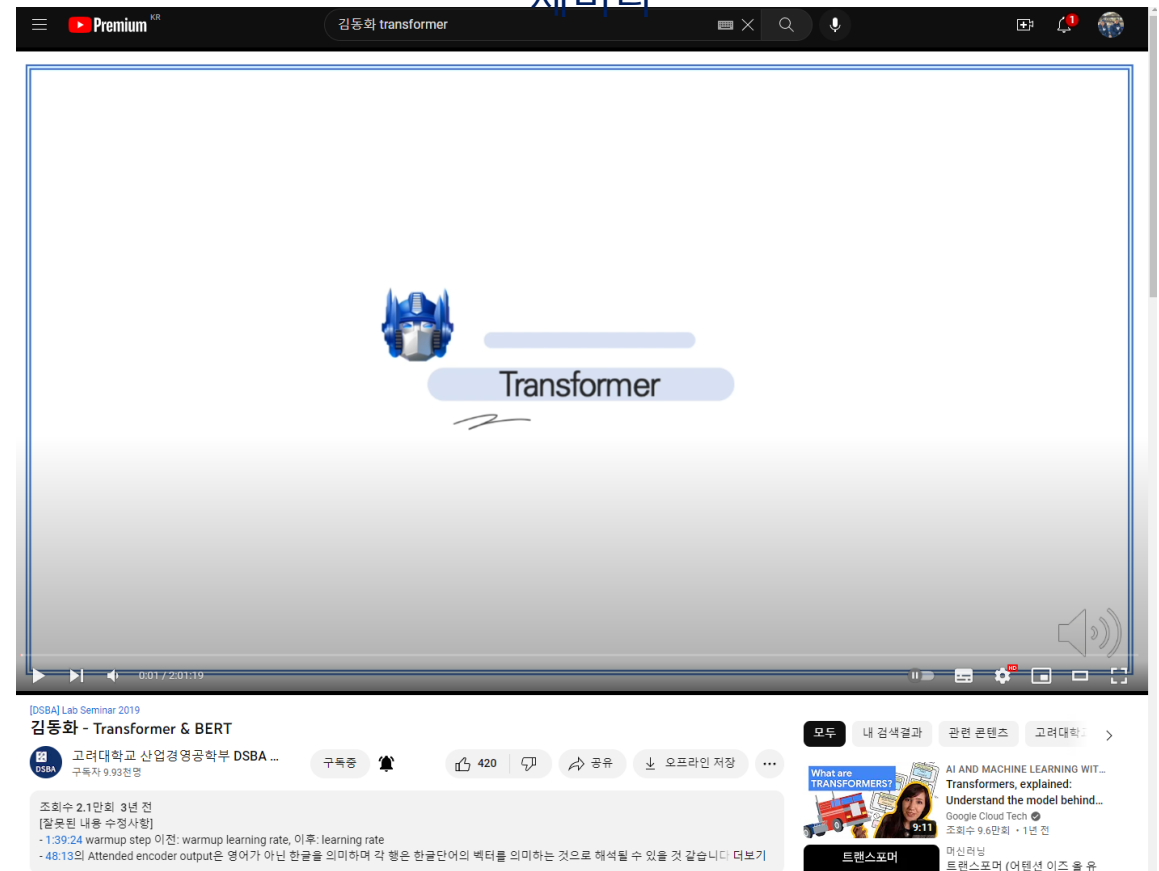
Trasformer 관련 설명 영상

고려대학교 산업경영공학부 강필성 교수 강의



https://www.youtube.com/watch?v=YkItV_cXMMU

고려대학교 DSBA 연구실 김동화 박사(현: 카카오스타일) 세미나



<https://www.youtube.com/watch?v=xhY7m8QVKjo&t=4528s>

Transformer의 시계열 데이터 적용

- Transformer를 다변량 시계열 데이터에 최초로 적용한 논문

Zerveas, G. Zerveas, G., Jayaraman, S., Patel, D., Bhamidipaty, A., & Eickhoff, C. (2021, August). A transformer-based framework for multivariate time series representation learning. In *Proceedings of the 27th ACM SIGKDD Conference on Knowledge Discovery & Data Mining* (pp. 2114-2124).

- ✓ Transformer의 Encoder 구조만 사용
- ✓ Pre-training 과업을 위하여 연속적인 길이의 Input Masking 사용
- ✓ Layer Normalization 대신 Batch Normalization 사용
- ✓ Fine-tuning 단계에서 구조를 어떻게 설계하느냐에 따라 Classification, Regression, Forecasting, Missing Value Imputation 등 다양한 Task에 적용 가능

A TRANSFORMER-BASED FRAMEWORK FOR MULTIVARIATE TIME SERIES REPRESENTATION LEARNING

George Zerveas
Brown University
Providence, RI, USA
george_erveas@brown.edu

Srideepika Jayaraman,
IBM Research
Yorktown Heights, NY, USA
j.srideepika@ibm.com

Dhaval Patel
IBM Research
Yorktown Heights, NY, USA
pateldha@us.ibm.com

Anuradha Bhamidipaty
IBM Research
Yorktown Heights, NY, USA
anubham@us.ibm.com

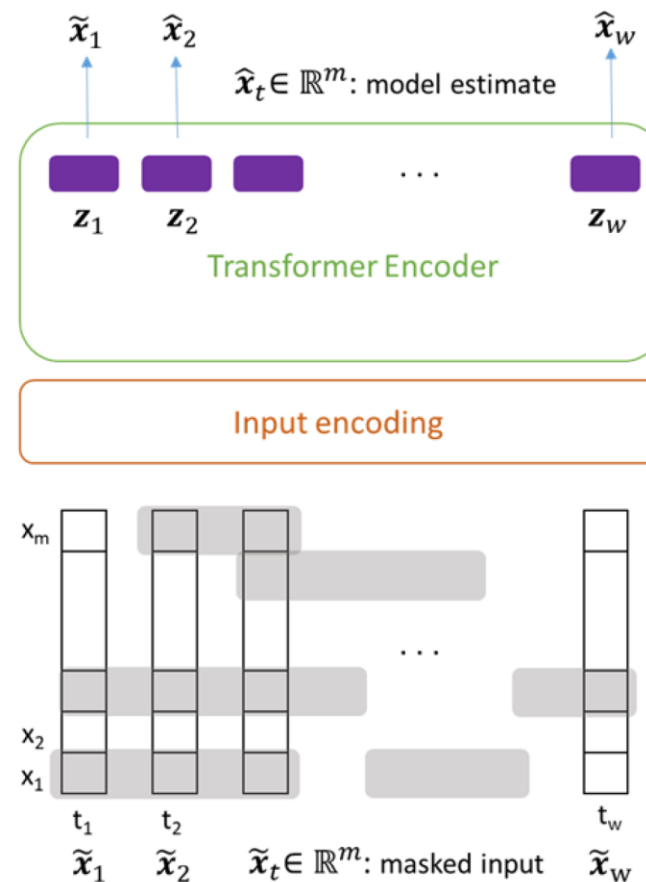
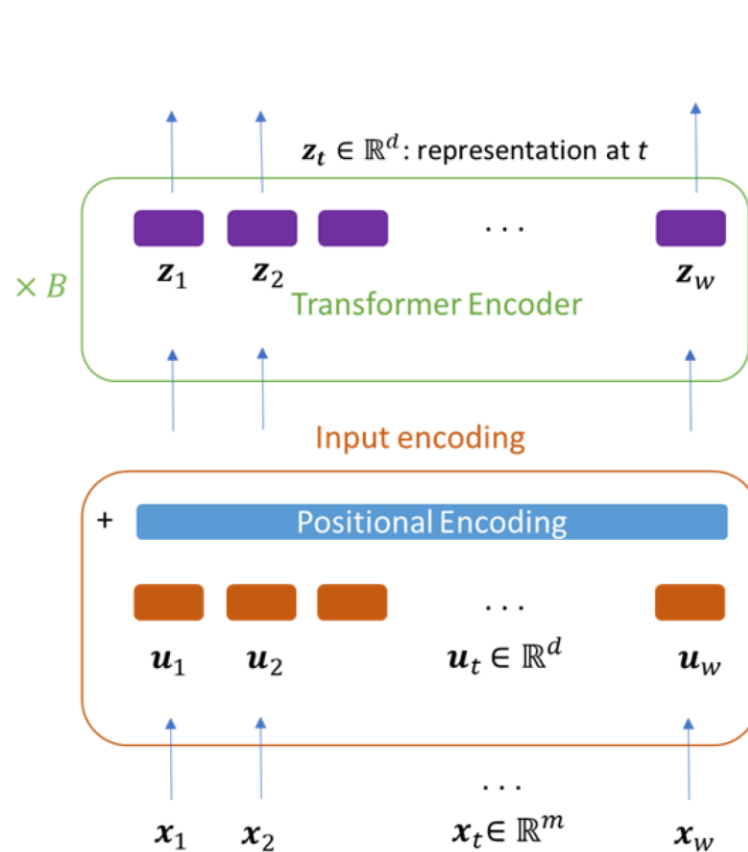
Carsten Eickhoff
Brown University
Providence, RI, USA
carsten@brown.edu

ABSTRACT

In this work we propose for the first time a transformer-based framework for unsupervised representation learning of multivariate time series. Pre-trained models can be potentially used for downstream tasks such as regression and classification, forecasting and missing value imputation. By evaluating our models on several benchmark datasets for multivariate time series regression and classification, we show that our modeling approach represents the most successful method employing unsupervised learning of multivariate time series presented to date; it is also the first unsupervised approach shown to exceed the current state-of-the-art performance of supervised methods. It does so by a significant margin, even when the number of training samples is very limited, while offering computational efficiency. Finally, we demonstrate that unsupervised pre-training of our transformer models offers a substantial performance benefit over fully supervised learning, even without leveraging additional unlabeled data, i.e., by reusing the same data samples through the unsupervised objective.

Time-Series Transformer (TST) 작동 원리

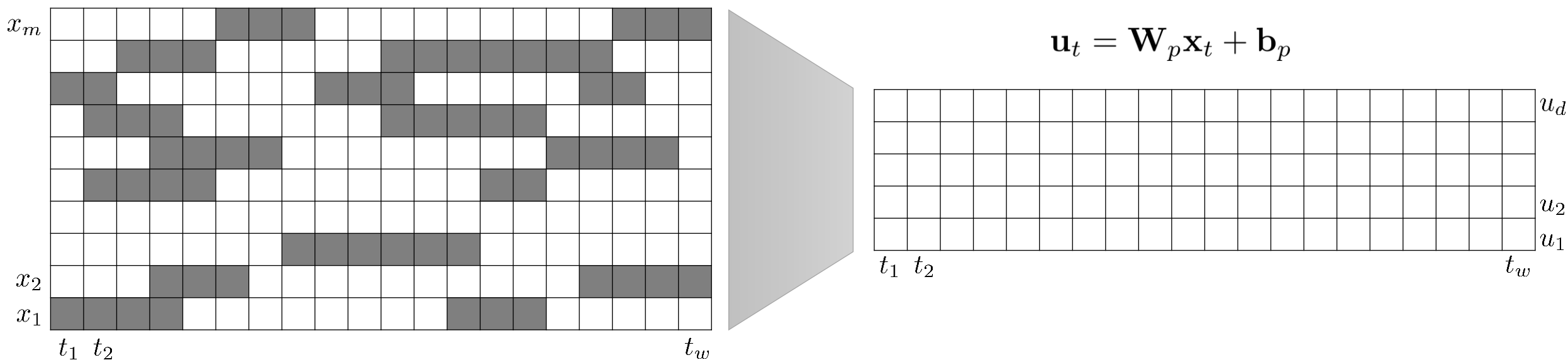
- TST는 크게 Pre-training과 Fine-tuning의 2 Phases로 구분됨



Time-Series Transformer (TST) 작동 원리

- 입력 데이터 변환

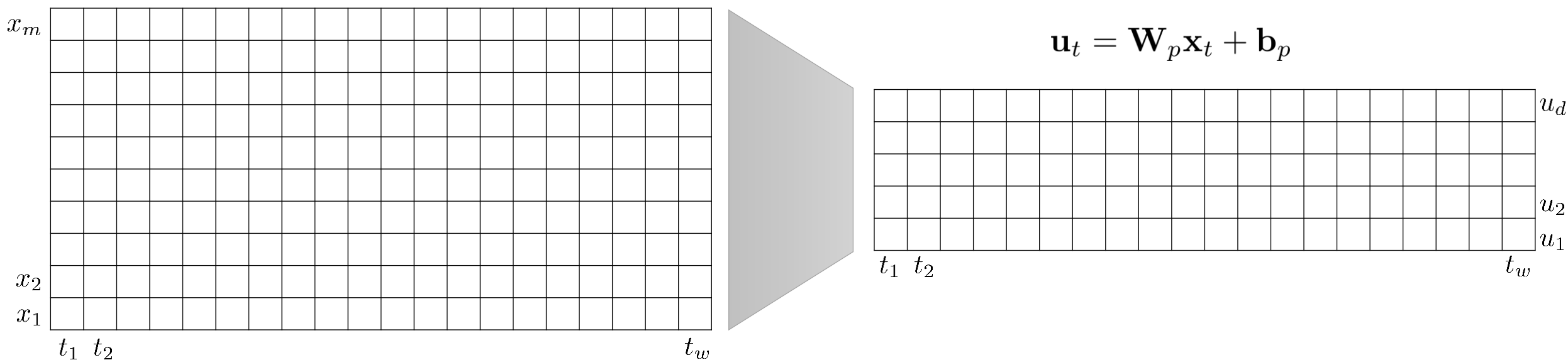
- ✓ 원본 입력 데이터 $\mathbf{x}$ 는 m 개의 변수와 w 개의 time window length를 가짐
- ✓ Pre-training 과정에서는 이 중 일부를 Masking을 한 뒤 Transformer의 Input이 되는 d 차원($d < m$)으로 변환
- ✓ Fine-tuning 과정에서는 Masking 과정 없이 d 차원으로 변환



Time-Series Transformer (TST) 작동 원리

- 입력 데이터 변환

- ✓ 원본 입력 데이터 $\mathbf{x}$ 는 m 개의 변수와 w 개의 time window length를 가짐
- ✓ Pre-training 과정에서는 이 중 일부를 Masking을 한 뒤 Transformer의 Input이 되는 d 차원($d < m$)으로 변환
- ✓ Fine-tuning 과정에서는 Masking 과정 없이 d 차원으로 변환



Time-Series Transformer (TST) 작동 원리

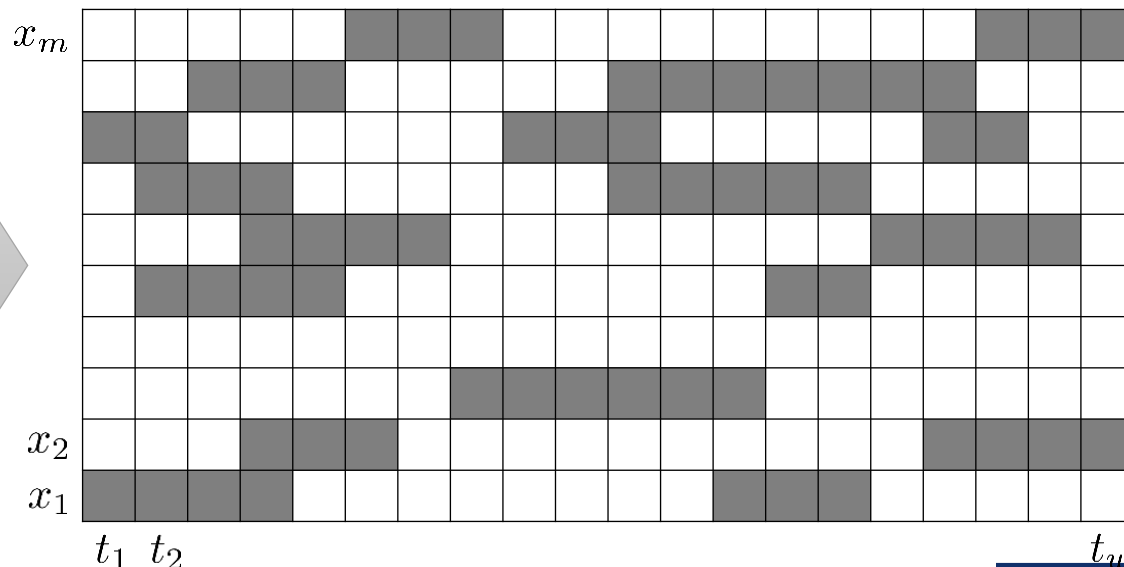
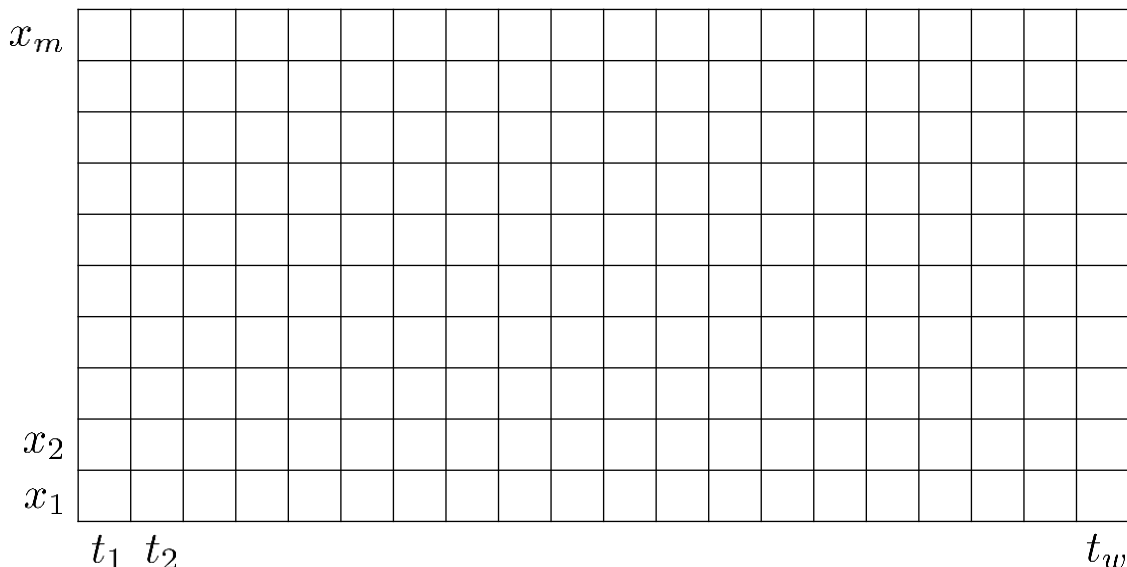
- 입력 데이터 Masking

- ✓ Transformer의 사전 학습 목적은 Masking 된 부분을 정확하게 예측하는 것으로 설계됨

- 각 Cell에 대한 Masking 여부를 독립적으로 결정하게 되면 Trivial Solution으로도 문제를 잘 맞추는 상황이 발생
- Trivial Solution: Masking된 Cell의 이전 시점 혹은 이후 시점 값을 그대로 사용하거나 양쪽 Cell의 평균값을 사용

- ✓ Masking의 길이가 평균 l_m 만큼이 되는 기하분포를 따르도록 Markov Chain을 적용하여 Masking 여부를 결정

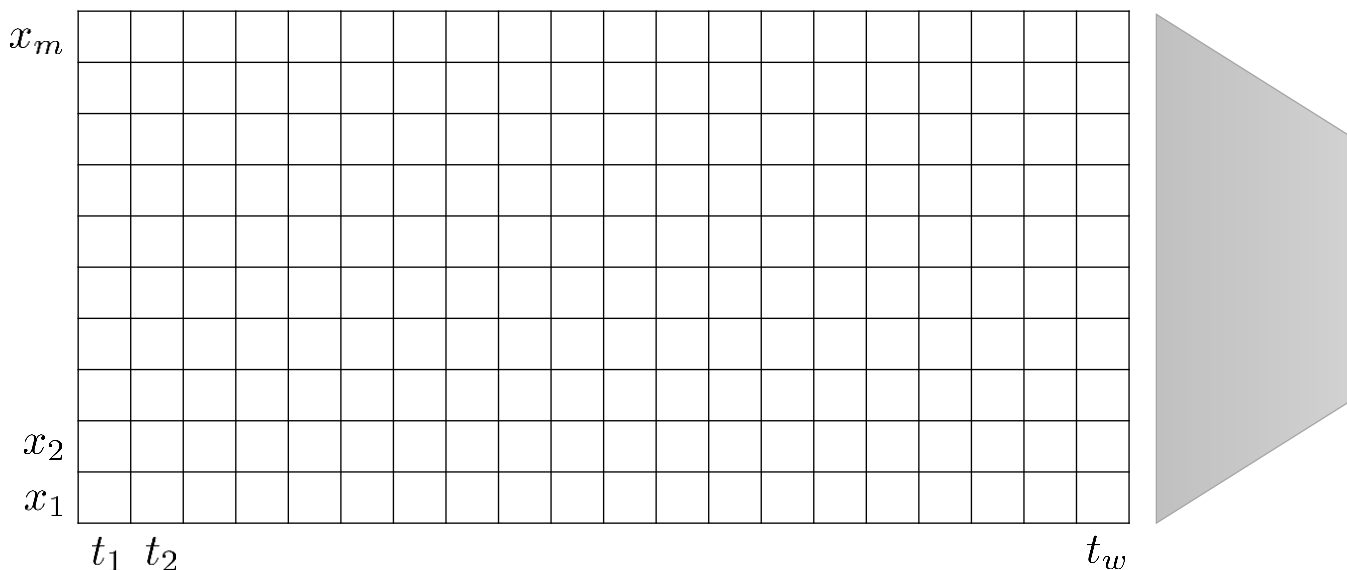
- 이 때, 모든 변수에 대해 동일한 시점을 Masking하는 것보다 변수별로 독립적으로 Masking segment를 결정하는 것이 실험적으로 더 우수한 성능을 나타냄



Time-Series Transformer (TST) 작동 원리

• 포지션 인코딩

- ✓ NLP에서의 Transformer와 마찬가지로 TST에서도 입력 데이터에 포지션 인코딩을 더해서 Transformer 인코더의 입력 값으로 사용
- ✓ 고정된 포지션 인코딩을 사용할 수도 있고, 학습 가능한 포지션 인코딩을 사용할수도 있음



$$U' = U + W_{pos}$$

Diagram illustrating the output matrix U' (yellow grid) with dimensions $t_1, t_2, \dots, t_w$ and $u'_1, u'_2, \dots, u'_d$.

Position Encoding Matrix

$$W_{pos} \in R^{w \times d}$$

Diagram illustrating the Position Encoding Matrix W_{pos} (brown grid).

$$U \in R^{w \times d} = [\mathbf{u}_1, \dots, \mathbf{u}_w]$$

Diagram illustrating the input matrix U (white grid) with dimensions $t_1, t_2, \dots, t_w$ and $u_1, u_2, \dots, u_d$.

$$\mathbf{u}_t = \mathbf{W}_p \mathbf{x}_t + \mathbf{b}_p$$

Time-Series Transformer (TST) 작동 원리

- Self-Attention in Transformer Encoder Block

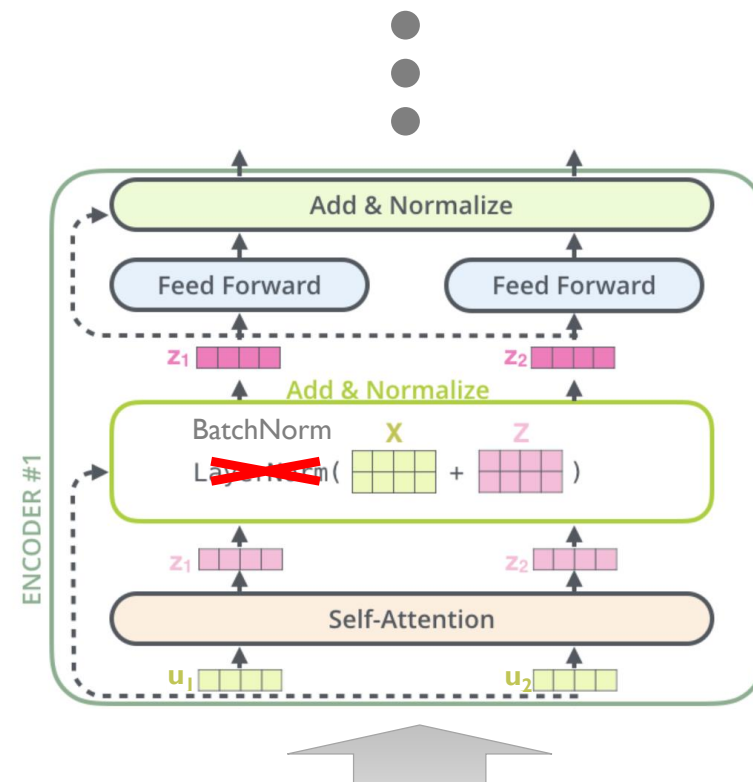
- ✓ 총 3개의 Encoder 블록을 사용

- ✓ NLP에서의 Transformer와는 다르게 Layer

Normalization 대신 Batch Normalization을 사용

- 시계열 데이터에는 NLP Word Embedding에는 없는 이상치(Outlier)가 존재함
 - 시계열 데이터는 각 관측치의 길이 변화가 NLP의 문장 길이의 변화보다 작음
 - 이러한 상황 하에서는 실험적으로 Batch Normalization이 Layer Normalization보다 성능이 우수한 것으로 입증

Dataset	Task (Metric)	LayerNorm	BatchNorm
Heartbeat	Classif. (Accuracy)	0.741	0.776
InsectWingbeat	Classif. (Accuracy)	0.658	0.684
SpokenArabicDigits	Classif. (Accuracy)	0.993	0.993
PEMS-SF	Classif. (Accuracy)	0.832	0.919
BenzeneConcentration	Regress. (RMSE)	2.053	0.516
BeijingPM25Quality	Regress. (RMSE)	61.082	60.357
LiveFuelMoistureContent	Regress. (RMSE)	42.993	42.607



$$U' = U + W_{pos}$$

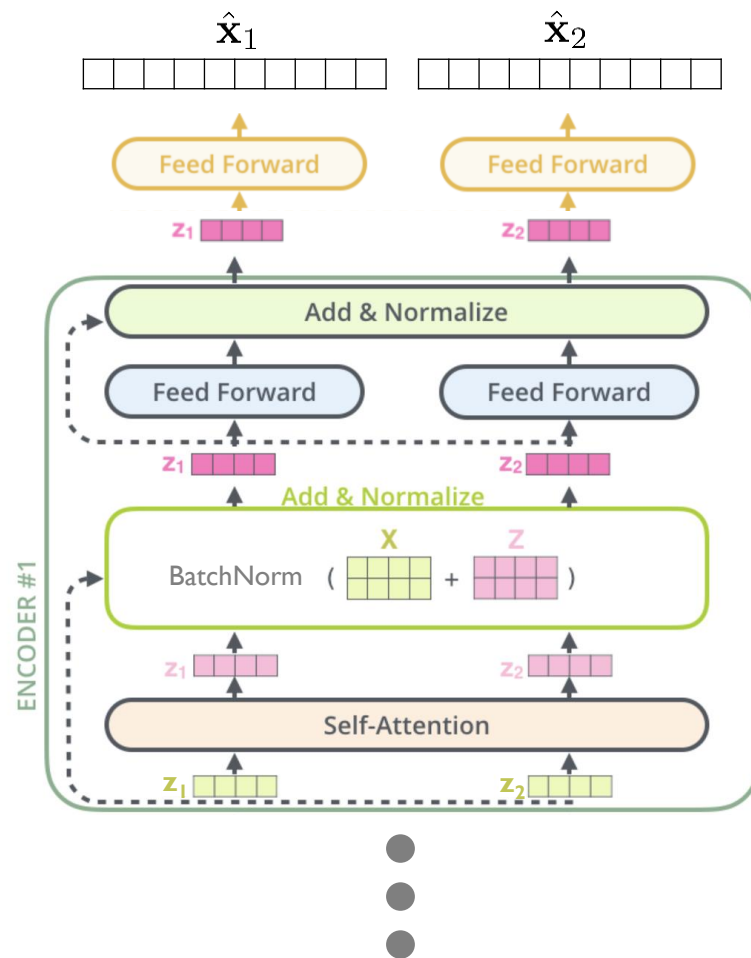
The diagram shows a grid representing the input matrix U with dimensions t_1 to t_w and u'_1 to u'_d . The equation $U' = U + W_{pos}$ is overlaid on the grid.

Time-Series Transformer (TST) 작동 원리

• Pre-training

- ✓ 각 관측치 및 epoch마다 독립적으로 생성한 mask를 input과 elementwise multiplication하여 masked input $\tilde{X} = M \odot X$ 를 도출함
- ✓ $\tilde{X}$ 를 input encoding과 Transformer Encoder에 순차적으로 넣어 latent representation $\bar{z} = [z_1; z_2; \dots; z_w] \in \mathbb{R}^{d \times w}$ 를 도출한 후, $\bar{z}$ 를 linear output layer에 넣어 모든 값이 채워져 있는 각 시점의 데이터를 예측함
 - $\hat{x}_t = W_o z_t + b_o$
- ✓ Masking된 부분의 실제 값과 예측 값의 mean squared error를 기반으로 모델을 학습함

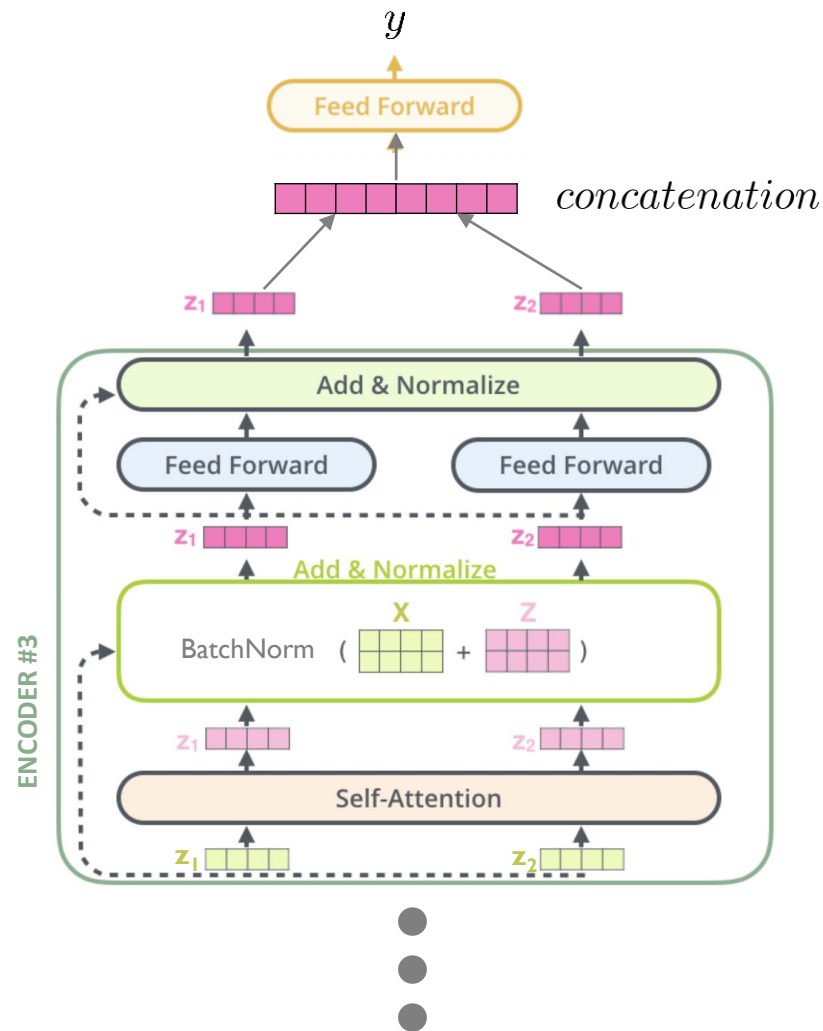
$$L_{MSE} = \frac{1}{|M|} \sum_{(t,i) \in M} (\hat{x}(t,i) - x(t,i))^2$$



Time-Series Transformer (TST) 작동 원리

- Fine-tuning

- ✓ 1단계) Masking를 적용하지 않은 Input Time Window를 Input encoding과 Transformer Encoder에 순차적으로 넣어 Representation을 도출함
- ✓ 2단계) 도출된 모든 시점의 Representation을 Concatenation한 것을 Output Linear Layer에 Input으로 넣어 Regression 또는 Classification의 정답을 예측함
- ✓ 3단계) Task의 실제 정답과 TST가 예측한 값의 차이를 통해 Output Linear Layer를 Fine-tuning 함



Time-Series Transformer (TST) 작동 원리

- Fine-tuning

- ✓ Pre-trained model에서 도출된 representation $\bar{z} = [z_1; z_2; \dots; z_w] \in \mathbb{R}^{d \times w}$ 를 활용해 regression 및 classification을 수행할 수 있음
- ✓ 세부적으로 $\bar{z}$ 를 새로운 linear output layer에 input으로 넣어 regression 및 classification을 수행하며, 해당 layer만 task에 맞는 loss를 통해 학습함
 - $\hat{y} = W_o \bar{z} + b_o$
 - Regression loss: $L_{\text{reg}} = \|\hat{y} - y\|^2$
 - Classification loss: $L_{\text{cls}} = \text{cross-entropy}(\text{softmax}(\hat{y}), y)$
- ✓ 논문에서는 fine-tuning task를 수행하기 위한 output layer를 제외한 pre-trained model은 freezing하고 fine-tuning을 진행함
 - pre-trained model에서 추출된 static feature를 사용하는 것이 속도와 성능의 trade-off에서 가장 적합하다는 것을 실험적으로도 증명함

Time-Series Transformer (TST) 성능

- Experimental Settings - Regression

- ✓ 각각 다른 변수 개수와 길이를 가지는 총 6개의 대표적인 regression 데이터셋을 활용하여 아래와 같이 제안한 TST를 학습 및 검증함
 - 학습: training set으로 TST를 unsupervised pre-training 한 후, pre-trained TST를 training set에 supervised fine-tuning하는 방식으로 모델을 학습함
 - 검증: fine-tuned TST로 test set을 예측 값을 도출한 후, 평가 지표를 도출함
- ✓ 아래 두 평가 지표를 사용하여 모델을 검증하였으며, fully supervised TST도 학습하여 pre-training 사용 여부에 따른 모델의 학습 시간 및 성능을 비교함
 - Root mean squared error (Root MSE)
 - Average Relative Difference from Mean (Avg.Rel.Diff.Mean): 모든 모델의 평균 Root MSE 대비 각 모델의 Root MSE 감소/증가 비율

Time-Series Transformer (TST) 성능

• Results - Regression

- ✓ TST가 총 6개의 데이터셋 중 4개에서 가장 좋은 성능을 도출하였으며, 나머지 2개에서는 두번째로 높은 성능을 도출함
- ✓ 총 6개의 데이터셋 중 3개에서 TST(pretrained)가 TST(sup. only)보다 높은 성능을 도출한 것을 통해 동일한 training set을 unsupervised/supervised way로 중복 사용하여 모델을 학습하는 것이 효과가 있다는 것을 알 수

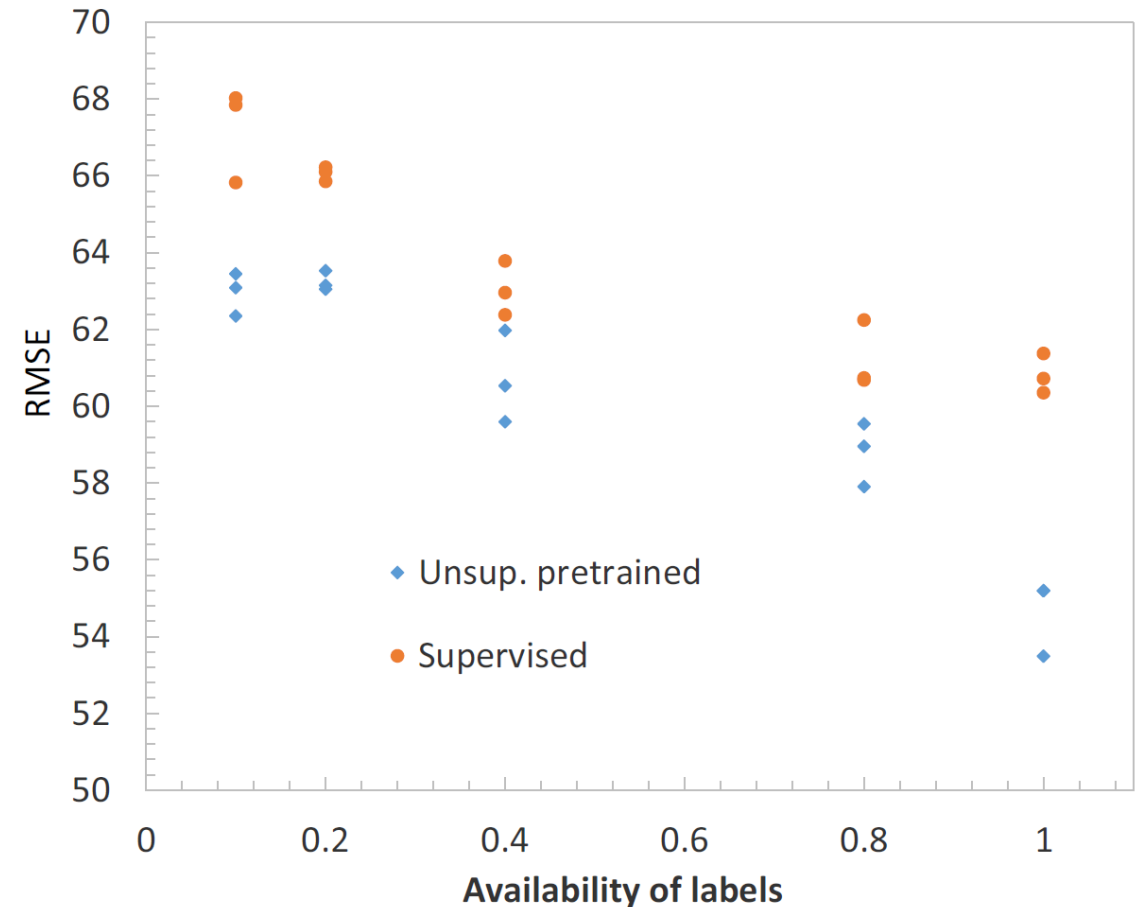
니오

Dataset	Root MSE											Ours	
	SVR	Random Forest	XGBoost	1-NN-ED	5-NN-ED	1-NN-DTWD	5-NN-DTWD	Rocket	FCN	ResNet	Inception	TST (sup. only)	TST (pretrained)
AppliancesEnergy	3.457	3.455	3.489	5.231	4.227	6.036	4.019	<u>2.299</u>	2.865	3.065	4.435	2.228	2.375
BenzeneConcentr.	4.790	0.855	0.637	6.535	5.844	4.983	4.868	<u>3.360</u>	4.988	4.061	1.584	<u>0.517</u>	0.494
BeijingPM10	110.574	94.072	93.138	139.229	115.669	139.134	115.502	120.057	94.348	95.489	96.749	<u>91.344</u>	86.866
BeijingPM25	75.734	63.301	<u>59.495</u>	88.193	74.156	88.256	72.717	62.769	59.726	64.462	62.227	<u>60.357</u>	53.492
LiveFuelMoisture	43.021	44.657	<u>44.295</u>	58.238	46.331	57.111	46.290	41.829	47.877	51.632	51.539	<u>42.607</u>	43.138
IEEEPPG	36.301	32.109	31.487	33.208	27.111	37.140	33.572	36.515	34.325	33.150	23.903	<u>25.042</u>	27.806
Avg Rel. diff. from mean	0.097	-0.172	-0.197	0.377	0.152	0.353	0.124	-0.048	0.021	0.005	-0.108	<u>-0.301</u>	-0.303
Avg Rank	7.166	4.5	3.5	10.833	8	11.167	7.667	5.667	6.167	6.333	5.666	1.333	

Time-Series Transformer (TST) 성능

- Results – Regression

- ✓ Q1. Given partially labeled dataset of a certain size, how will additional labels affect performance?
- ✓ Supervised learning에 사용되는 label의 비율 증가에 따른 TST(pretrained)와 TST(sup. only)의 성능 변화를 통해 아래의 결과를 확인할 수 있음
 - 두 모델 모두 사용 가능한 label의 비율이 증가할수록 성능이 향상됨
 - TST(pretrained)가 TST(sup. only) 보다 모든 비율에서 높은 성능을 도출한 것을 통해 동일한 training set을 중복 사용하여 모델을 학습한 것이 효과가 있다는 것을 알 수 있음



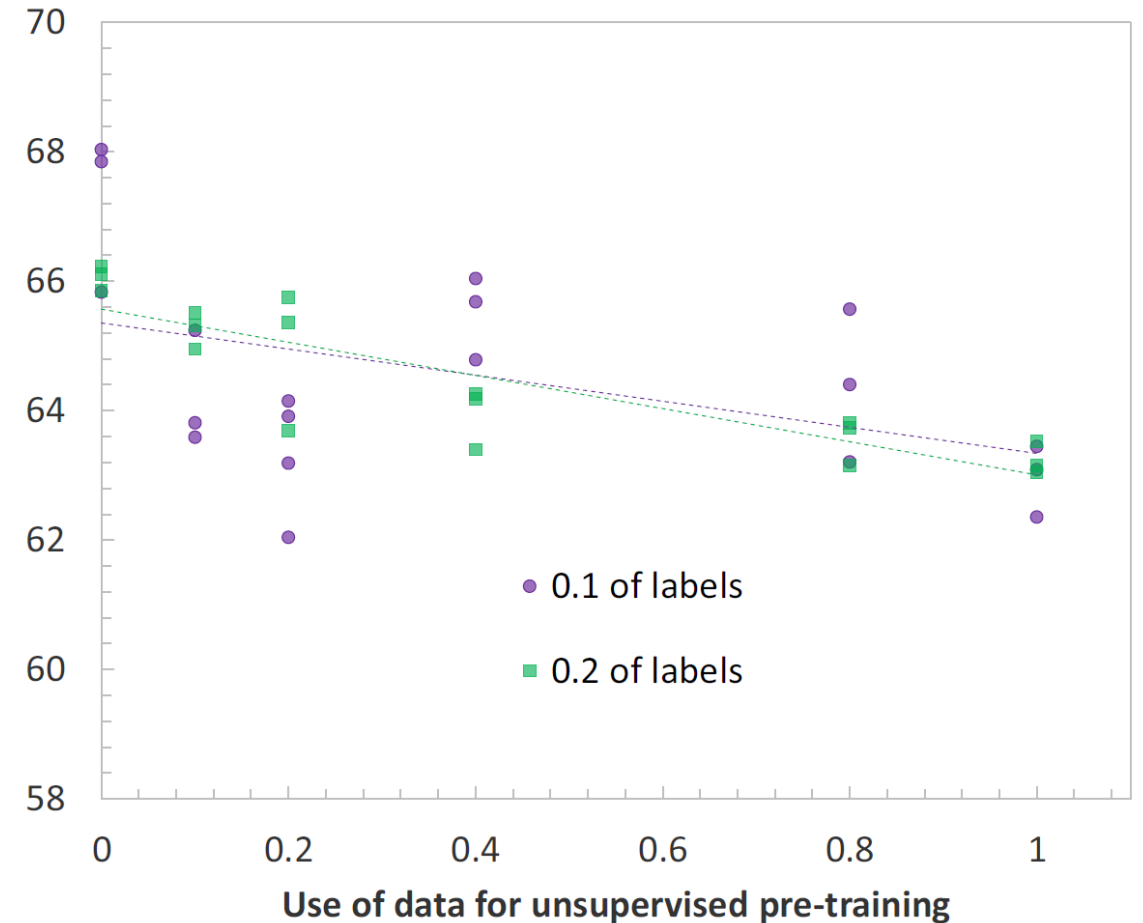
Time-Series Transformer (TST) 성능

- Results – Regression

- ✓ Q2. Given labeled dataset, how will additional unlabeled samples affect performance?

- ✓ Fine-tuning 데이터의 개수가 고정되었을 때, pre-training에 사용되는 데이터의 비율 증가에 따른 TST(pretrained)의 성능 변화를 통해 아래 결과를 확인할 수 있음

- Pre-training에서 많은 데이터를 학습할수록 TST(pretrained)의 성능이 향상됨
- TST(pretrained)의 fine-tuning 데이터의 개수가 적을수록 pre-training에 사용되는 데이터의 비율 증가에 따른 성능 향상 폭이 큼



TST 관련 설명 영상

고려대학교 DSBA 연구실 최희정 박사과정 세미나



<https://www.youtube.com/watch?v=t72DDUcpIzc&t=1602s>

Summary

- Time-series Transformer (TST)

- ✓ Transformer의 Encoder 구조만 사용
- ✓ Pre-training 과업을 위하여 연속적인 길이의 Input Masking 사용
- ✓ Layer Normalization 대신 Batch Normalization 사용
- ✓ Fine-tuning 단계에서 구조를 어떻게 설계하느냐에 따라 Classification, Regression, Forecasting, Missing Value Imputation 등 다양한 Task에 적용 가능
- ✓ Pre-training을 통해 기존의 Supervised Learning만 수행했을 경우보다도 우수한 예측 성능을 내는 것을 확인

